Lab 6: A PWM Controller for an RGB LED

Yuval Zecharia

08/11/2026

1. מטרת המעבדה

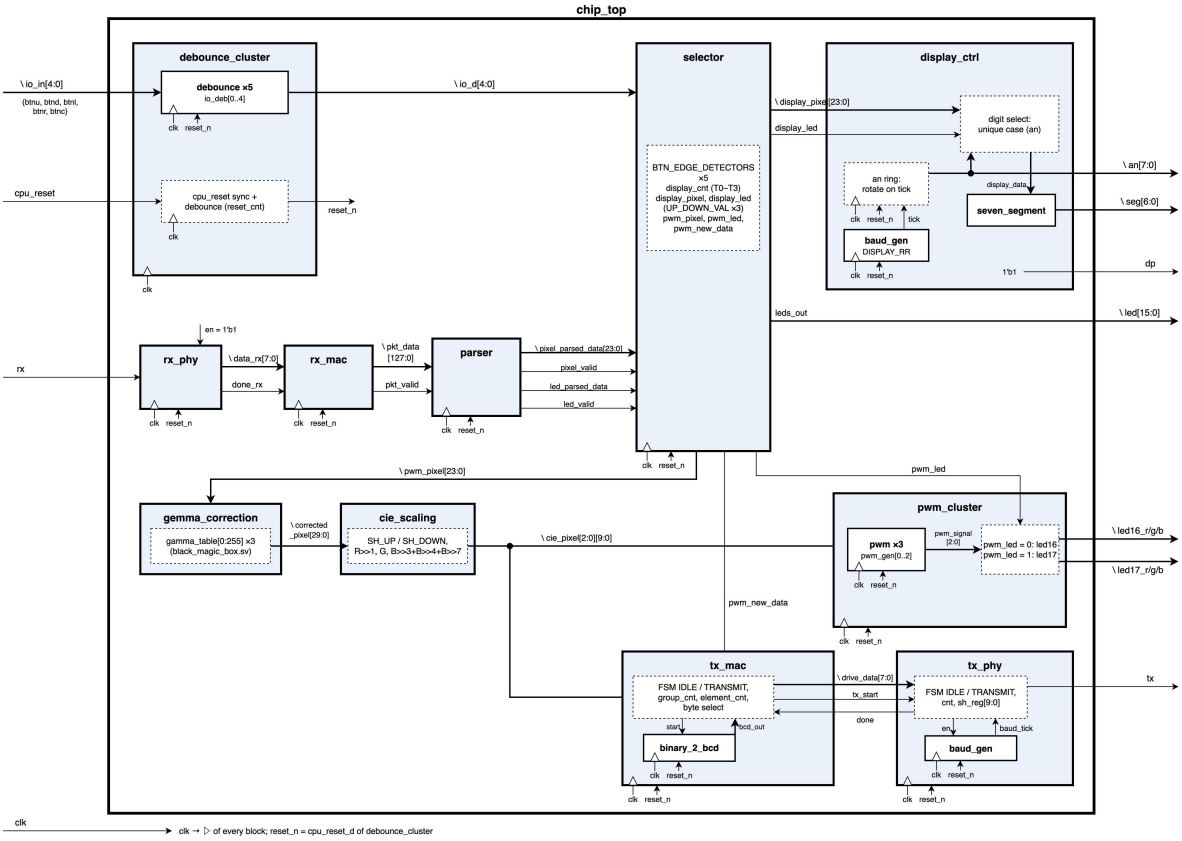
מטרת המעבדה היא לממש ב-SystemVerilog, על לוח Nexys A7-100T, בקר PWM לנורות ה-RGB של הלוח. המערכת מקבלת מה-PC ב-UART פיקסל sRGB או בחירת נורה, ממירה את ערכי ה-sRGB לערכים לינאריים (gamma correction), מאזנת את שלושת הצבעים לפי רגישות העין (CIE scaling) ומדליקה בהתאם את LED16 או את LED17 בשלושה ערוצי PWM. הערכים אחרי ההמרה נשלחים חזרה ל-PC, ותצוגת ה-Seven Segment מציגה את הפיקסל ואת הנורה שנבחרה, שאותם אפשר לערוך גם בלחצנים. הדרישות:

1. UART ב-57600 bps, בשני הכיוונים.
2. שתי הודעות: {Rddd,Gddd,Bddd} עם ערכי הפיקסל, ו-{Lddd} לבחירת הנורה (016 או 017).
3. debounce של 20ms לכל לחצן, בקוד גנרי שנפרש ב-generate.
4. Up ו-Down משנים את השדה הנבחר בלי גלישה, Left ו-Right מזיזים את הבחירה בלי מעבר מעגלי, ו-Center מחיל את הערכים על בקר ה-PWM.
5. תצוגה בלי הבהוב: T0 כחול, T1 ירוק ו-T2 אדום, ב-hex, ו-T3 הנורה (01 ל-LED16, 10 ל-LED17). LED[10:9], LED[8:7], LED[5:4] ו-LED[13:12] מסמנים את השדה הנבחר.
6. בקר PWM של 1024 סלוטים, בפרמטר שתומך גם ב-4096, 2048, 512 ו-8192, אחרי gamma correction במודול הנתון ו-CIE scaling.
7. שליחת הערכים אחרי ההמרה חזרה ל-PC.

2. תהליך התכנון

2.1. ארכיטקטורה

המערכת בנויה משלושה נתיבים שנפגשים ב-selector. נתיב ה-RX (rx_phy, rx_mac ו-parser) מפענח את ההודעות מה-PC. חמשת הלחצנים ו-CPU_RESETN עוברים debounce ב-debounce_cluster, ו-reset_n שיוצא ממנו מאפס את כל שאר המודולים. ה-selector מחזיק שני עותקים של הפיקסל ושל בחירת הנורה: display_pixel ו-display_led, שמוצגים בתצוגה ונערכים בלחצנים, ו-pwm_pixel ו-pwm_led, שמוחלים על הנורה בלחיצה על Center או כשמגיעה הודעה. pwm_pixel עובר gamma correction ו-CIE scaling, והתוצאה, cie_pixel, מזינה גם את pwm_cluster וגם את tx_mac, שמשדר אותה חזרה ל-PC (איור 1). ממשק ה-io של chip_top מפורט בטבלה 1.



איור 1: מבנה chip_top: תת-המודולים והאותות ביניהם.

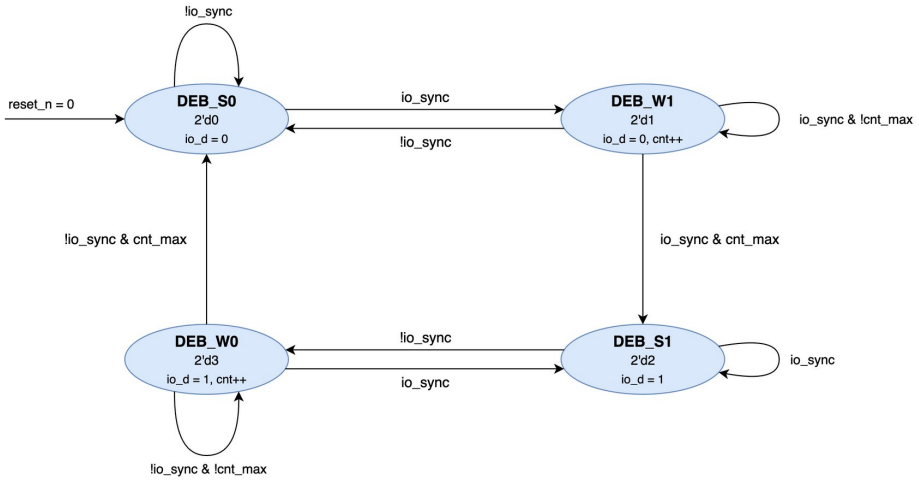
2.2. ממשק chip_top

טבלה 1: ממשק chip_top.

Port name	Direction	Type	Description
clk	Input	logic	100 MHz system clock (E3)
cpu_reset	Input	logic	CPU_RESETN push-button, active low (C12)
btnu, btnd	Input	logic	Up / Down push-buttons: the selected field +1 / −1, no wrap-around (M18, P18)
btnl, btnr	Input	logic	Left / Right push-buttons: move the selection toward T3 / T0, no cyclic shift (P17, M17)
btnc	Input	logic	Center push-button: applies the displayed pixel and LED to the PWM controller (N17)
rx	Input	logic	UART line from the PC (UART_TXD_IN, C4)
tx	Output	logic	UART line to the PC (UART_RXD_OUT, D4); 1 when idle
an	Output	logic [7:0]	Digit anodes, active low, one digit at a time
seg	Output	logic [6:0]	Cathodes {CA … CG} of the active digit, active low
dp	Output	logic	Decimal point: always 1 (off)
led	Output	logic [15:0]	Selected field: LED[5:4] = T0, LED[8:7] = T1, LED[10:9] = T2, LED[13:12] = T3; the other LEDs are 0
led16_r, led16_g, led16_b	Output	logic	LED16 red / green / blue (N15, M16, R12): PWM when LED16 is selected
led17_r, led17_g, led17_b	Output	logic	LED17 red / green / blue (N16, R11, G14): PWM when LED17 is selected

2.3. מכונות המצבים

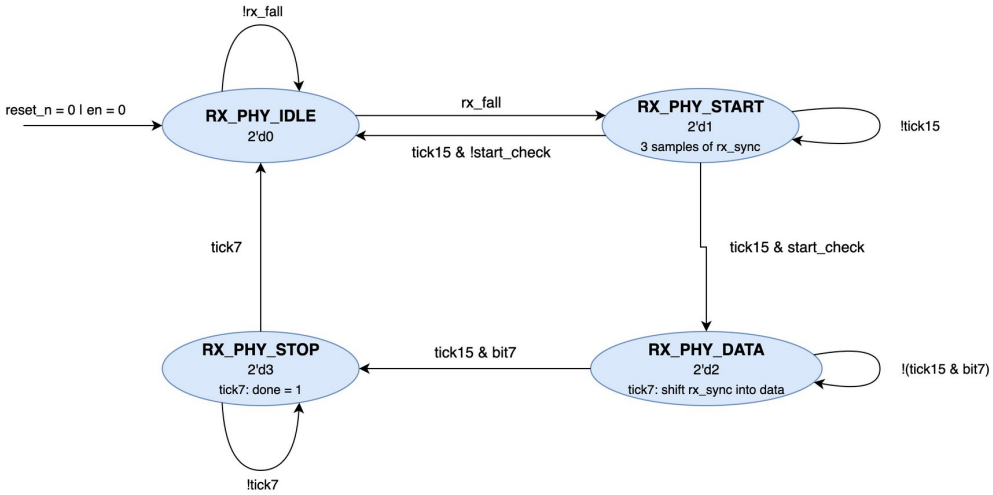
בתכנון חמש מכונות מצבים (איורים 2 עד 6 וטבלאות 2 עד 6). הקודים בדיאגרמות הם ערכי ה-enum שב-lab6_pkg, והסינתזה מקודדת אותם מחדש one-hot (FSM_ENCODING=auto): ב-rx_mac, קידוד בינארי של שני ביטים ב-debounce וב-rx_phy, ו-tx_mac ו-tx_phy, שה-enum שלהם ברוחב ביט אחד, נשארים דלגלג יחיד. כשאף תנאי יציאה אינו מתקיים, המכונה נשארת במצבה (הלולאות העצמיות בדיאגרמות).



איור 2: דיאגרמת המצבים של debounce (מופע אחד לכל לחצן).

טבלה 2: טבלת המעברים של debounce.

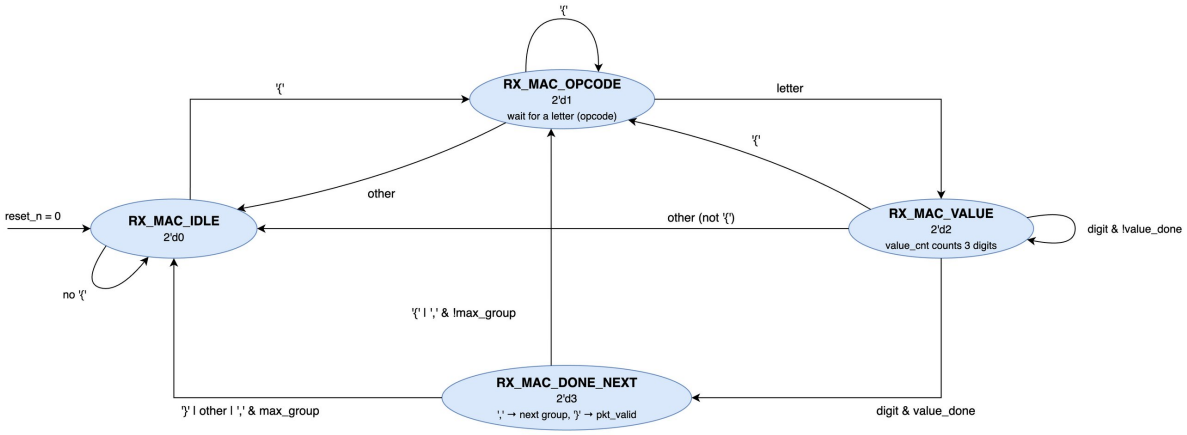
Current state	Condition	Next state	Registered on the transition
DEB_S0 (0)	io_sync	DEB_W1	cnt = 0
DEB_W1 (1)	!io_sync	DEB_S0	cnt = 0 (a glitch)
DEB_W1 (1)	io_sync & cnt_max	DEB_S1	cnt = 0; io_d = 1 from now on
DEB_S1 (2)	!io_sync	DEB_W0	cnt = 0
DEB_W0 (3)	io_sync	DEB_S1	cnt = 0 (a glitch)
DEB_W0 (3)	!io_sync & cnt_max	DEB_S0	cnt = 0; io_d = 0 from now on



איור 3: דיאגרמת המצבים של rx_phy.

טבלה 3: טבלת המעברים של rx_phy.

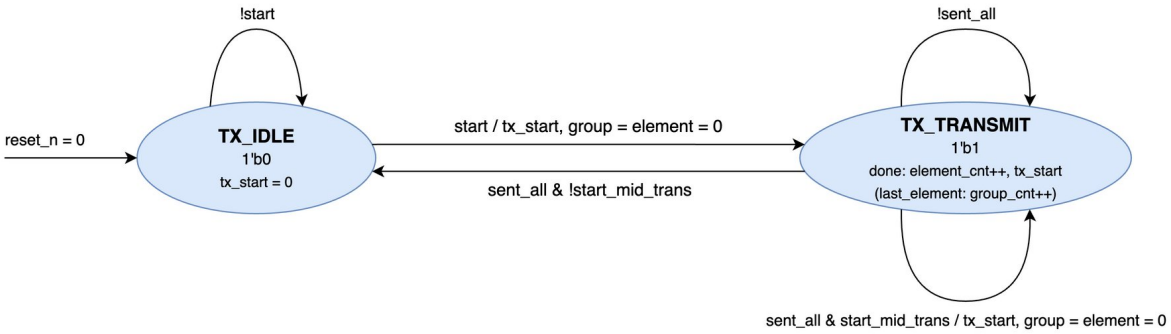
Current state	Condition	Next state	Registered on the transition
RX_PHY_IDLE (0)	rx_fall	RX_PHY_START	tick_cnt = tick16_cnt = 0: the ticks align to the falling edge
RX_PHY_START (1)	tick15 & start_check	RX_PHY_DATA	bit_cnt = 0
RX_PHY_START (1)	tick15 & !start_check	RX_PHY_IDLE	– (a glitch, not a start bit)
RX_PHY_DATA (2)	tick7 (stays)	RX_PHY_DATA	data = {rx_sync, data[7:1]}
RX_PHY_DATA (2)	tick15 & bit7	RX_PHY_STOP	–
RX_PHY_STOP (3)	tick7	RX_PHY_IDLE	done = 1 for one clock
any state	en = 0 (en = 1'b1 in chip_top)	RX_PHY_IDLE	–



איור 4: דיאגרמת המצבים של rx_mac.

טבלה 4: טבלת המעברים של rx_mac (כל מעבר דורש done, כלומר בית שהתקבל).

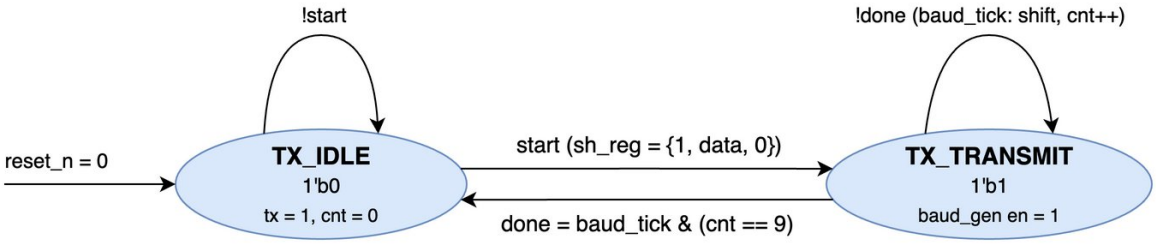
Current state	Condition	Next state	Registered on the transition
RX_MAC_IDLE (0)	'{'	RX_MAC_OPCODE	pkt_data = {0…0, '{'}, group_cnt = 0
RX_MAC_OPCODE (1)	letter (A–Z, a–z)	RX_MAC_VALUE	shift the byte in, value_cnt = 0
RX_MAC_OPCODE (1)	'{' / other	RX_MAC_OPCODE / RX_MAC_IDLE	restart with '{' / drop the packet
RX_MAC_VALUE (2)	digit & !value_done	RX_MAC_VALUE	shift in, value_cnt++
RX_MAC_VALUE (2)	digit & value_done	RX_MAC_DONE_NEXT	shift in (the third digit)
RX_MAC_VALUE (2)	'{' / other	RX_MAC_OPCODE / RX_MAC_IDLE	restart with '{' / drop the packet
RX_MAC_DONE_NEXT (3)	',' & !max_group	RX_MAC_OPCODE	shift in, group_cnt++
RX_MAC_DONE_NEXT (3)	'}'	RX_MAC_IDLE	shift in; pkt_valid = 1 for one clock
RX_MAC_DONE_NEXT (3)	'{'	RX_MAC_OPCODE	restart with '{'
RX_MAC_DONE_NEXT (3)	other, ',' & max_group	RX_MAC_IDLE	– (a fourth group is rejected)



איור 5: דיאגרמת המצבים של tx_mac.

טבלה 5: טבלת המעברים של tx_mac (הודעה של 19 בתים לכל start).

Current state	Condition	Next state	Registered on the transition
TX_IDLE (0)	start	TX_TRANSMIT	tx_start, group_cnt = element_cnt = 0: the '{' byte, binary_2_bcd starts
TX_TRANSMIT (1)	done & !sent_all	TX_TRANSMIT	tx_start, element_cnt++ (after element 5: element_cnt = 0, group_cnt++)
TX_TRANSMIT (1)	start	TX_TRANSMIT	start_mid_trans = 1: one more message waits
TX_TRANSMIT (1)	sent_all & start_mid_trans	TX_TRANSMIT	tx_start, group_cnt = element_cnt = 0, start_mid_trans = 0
TX_TRANSMIT (1)	sent_all & !start_mid_trans	TX_IDLE	– (sent_all = done of the '}' byte)



איור 6: דיאגרמת המצבים של tx_phy.

טבלה 6: טבלת המעברים של tx_phy.

Current state	Condition	Next state	Registered on the transition
TX_IDLE (0)	start	TX_TRANSMIT	sh_reg = {1, data, 0}: tx = 0 (start bit)
TX_TRANSMIT (1)	baud_tick & cnt < 9	TX_TRANSMIT	sh_reg = {1, sh_reg[9:1]}, cnt++
TX_TRANSMIT (1)	done = baud_tick & cnt == 9	TX_IDLE	cnt = 0; tx = 1

2.4. שיקולי תכנון

- קצב השידור (BAUD_RATE=57600, מהדרישות): baud_gen ב-tx_phy סופר COUNT מחזורים ומפיק baud_tick רק כשה-en שלו פעיל, במצב TX_TRANSMIT. COUNT מחושב בחלוקה שלמה, ולכן הקצב בפועל מהיר מעט מ-57600 (משוואה 1), בשגיאה יחסית של 0.0064% (משוואה 2). פריים של 10 ביטים נמשך 173.6μs (משוואה 3).

$$COUNT = \lfloor \frac{100\ MHz}{57600} \rfloor = \lfloor 1736.11 \rfloor = 1736,\ f_{actual} = \frac{100\ MHz}{1736} = 57603.69\ bps \quad (1)$$

$$error = \frac{f_{actual} - BAUD_RATE}{BAUD_RATE} = \frac{57603.69 - 57600}{57600} = +0.0064\% \quad (2)$$

$$T_{frame} = 10 \cdot COUNT \cdot T_{clk} = 10 \cdot 1736 \cdot 10\ ns = 173.6\ \mu s \quad (3)$$

- קליטה (אותו BAUD_RATE): rx_phy מחלק כל ביט ל-16 ticks (oversampling). TICK_CNT מעוגל כלפי מטה ל-108, ולכן ביט במקלט קצר ב-0.47% מהביט הנומינלי (משוואה 4). הירידה של ה-start bit מאפסת את המונים, וכל ביט נדגם ב-tick7, באמצע הביט. ההסטה המצטברת עד ה-stop bit, שנדגם 9.5 ביטים אחרי הירידה, היא 77 מחזורים בלבד, 4.4% מביט (משוואה 5). start bit מאומת רק אחרי 3 דגימות רציפות של 0 באמצע הביט הראשון.

$$TICK_CNT = \lfloor \frac{100\ MHz}{57600 \cdot 16} \rfloor = \lfloor 108.51 \rfloor = 108\ \Rightarrow\ T_{bit,RX} = 16 \cdot 108 = 1728\ clk \quad (4)$$

$$\Delta_{stop} = 9.5 \cdot (1736.11 - 1728) \approx 77\ clk \approx 0.044 \cdot T_{bit} \quad (5)$$

- מבנה ההודעה: rx_mac בודק את המבנה תוך כדי קליטה ({, אות, שלוש ספרות, פסיק בין קבוצות, }) ומכניס כל בית מתאים לרגיסטר הזזה של 128 ביטים, כאורך הודעת פיקסל מלאה של 16 בתים. הודעת נורה באורך 6 בתים (משוואה 6), ואחרי ההזזה היא יושבת ב-48 הביטים התחתונים של pkt_data. אילו אותיות התקבלו בודק ה-parser, ולא rx_mac.

$$N_{pixel} = 2 + 3 \cdot (1 + 3) + (3 - 1) = 16\ bytes = 128\ bit,\ N_{led} = 2 + 1 + 3 = 6\ bytes = 48\ bit \quad (6)$$

- פענוח: ה-parser מזהה הודעת פיקסל כשהאותיות במקומן הן R, G ו-B (גם קטנות), והודעת נורה כשבמקום האות האחרונה יש L ו-80 הביטים העליונים של pkt_data הם 0. כל ערך מומר לבינארי (משוואה 7). ערך פיקסל גדול מ-255, או ערך נורה שאינו 16 או 17, פוסל את ההודעה, וה-selector ממשיך עם הערכים הקודמים. הודעה תקינה מפיקה פולס pixel_valid או led_valid.

$$value = 100 \cdot d_2 + 10 \cdot d_1 + d_0 \le 999\ \Rightarrow\ \lceil log_2 1000 \rceil = 10\ bit \quad (7)$$

- תצוגה (DISPLAY_RR): רענון של 500Hz לכל אחת משמונה הספרות נותן DISPLAY_RR=4000, ו-baud_gen של התצוגה סופר 25000 מחזורים בין tick ל-tick (משוואה 8). בכל tick האנודה הפעילה עוברת לספרה הבאה, ו-unique case על an בוחר את ה-nibble שלה: T0 (AN1 ו-AN0) הכחול, T1 הירוק ו-T2 האדום, ב-hex. T3 מציג 01 כש-display_led=0, כלומר LED16, ו-10 כש-display_led=1, כלומר LED17. הנקודה העשרונית כבויה תמיד (dp=1).

$$DISPLAY_RR = 8 \cdot f_{refresh} = 8 \cdot 500\ Hz = 4000\ \Rightarrow\ COUNT = \frac{f_{clk}}{DISPLAY_RR} = \frac{100\ MHz}{4000} = 25000 \quad (8)$$

- debounce וסנכרון (DEB_COUNT): כל אחד מחמשת הלחצנים עובר מופע debounce משלו (io_deb, ב-generate): סנכרון בשני דלגלגים ומכונת מצבים עם מונה יציבות של 20ms (משוואה 9). גם CPU_RESETN עובר סנכרון ומונה יציבות משלו, ומפיק את n_reset של כל התכנון.

$$DEB_COUNT = f_{clk} \cdot 20\ ms = 2 \cdot 10^6\ \Rightarrow\ \lceil log_2 (2 \cdot 10^6) \rceil = 21\ bit \quad (9)$$

- selector: לכל לחצן גלאי עליה (BTN_EDGE_DETECTORS), ולכן החזקה של לחצן היא צעד אחד. display_cnt בוחר את השדה, מ-T0 ועד T3, ונעצר בקצוות בלי מעבר מעגלי, ו-leds_out

מדליק את זוג הנוריות של השדה. Up ו-Down משנים ב-display_pixel את הערך של השדה הנבחר ונעצרים ב-255 וב-(is_max, is_min) 0. ב-T3 הם מחליפים את display_led: Up מ-LED16 ל-LED17 ו-Down חזרה. Center מעתיק את display_pixel ואת display_led ל-pwm_pixel ול-pwm_led ומפיק פולס pwm_new_data. הודעת פיקסל טוענת את שני העותקים ומפיקה pwm_new_data, והודעת נורה טוענת את שני העותקים של הנורה בלי pwm_new_data, ולכן בלי שידור.

- gamma correction: ב-gemma_correction שלוש קריאות במקביל, אחת לכל צבע, מהטבלה הנתונה (black_magic_box.sv) gamma_table של 256 ערכים ברוחב 10 ביטים. הטבלה ממירה ערך sRGB לעוצמה לינארית בסקאלה של 1023 (משוואה 10).

$$c = \frac{sRGB}{255},\ L = round(1023 \cdot (\frac{c+0.055}{1.055})^{2.4})\ \ (c > 0.04045),\ L = round(1023 \cdot c/12.92)\ \ (c \le 0.04045)$$

(10)

- CIE scaling: העין רגישה לכל צבע בעוצמה אחרת (משקלי Rec.601: 0.299 לאדום, 0.587 לירוק ו-0.114 לכחול). כדי שערכים שווים ייראו בהירים באותה מידה, cie_scaling מחליש כל צבע ביחס הפוך לרגישות העין, מנורמל לאדום: האדום נשאר בעוצמה מלאה, והירוק מוכפל ב-0.299 חלקי 0.587 (משוואה 11). הכחול היה צריך להיות מוכפל ב-2.62, אבל מחזור עבודה לא יכול לעבור 100%, ולכן המקדם שלו נחתך ל-1. המשקל של הירוק ממומש בשתי הזזות ובמחבר אחד, בלי כפל. בערך מלא (1023) מתקבלים 518 ,1023 ו-1023 (משוואה 12), והשגיאה מול המשקל המדויק של הירוק היא 0.3%−. סעיף 2.5 משווה בין הנרמול הזה לנרמולים האחרים ומסביר למה הוא נבחר.

$$s_R = 1,\ s_G = \frac{0.299}{0.587} = 0.509 \approx \frac{1}{2} + \frac{1}{128} = 0.5078,\ s_B = min(1, \frac{0.299}{0.114} = 2.62) = 1 \quad (11)$$

$$R = 1023,\ G = (1023 \gg 1) + (1023 \gg 7) = 518,\ B = 1023 \quad (12)$$

- SLOTS: PWM (SLOTS) הוא פרמטר של chip_top (ברירת מחדל 1024) שעובר בשם cie_scaling- ל-pwm_cluster ול-tx_mac, ו-SLOTS_WIDTH הוא localparam שנגזר ממנו (משוואה 13). הטבלה ברוחב קבוע של 10 ביטים (GAMMA_WIDTH בחבילה), ולכן cie_scaling מתאים את ערכיה ל-SLOTS_WIDTH בבלוק generate: הזזה שמאלה (SH_UP) כש-SLOTS_WIDTH>10, והזזה ימינה (SH_DOWN) אחרת. הזזה בקבוע היא חיווט בלבד, ומחזור העבודה זהה בכל גודל: ערך 600 בטבלה נותן 58.6% ב-512, ב-1024 וב-8192 סלוטים. ב-512 סלוטים הולך לאיבוד ה-LSB של הטבלה, ולכן 1024 הוא הגודל הקטן ביותר בלי איבוד. טבלה 7 מסכמת את חמשת הגדלים.

$$SLOTS_WIDTH = \lceil log_2 SLOTS \rceil = \lceil log_2 1024 \rceil = 10\ bit,\ f_{PWM} = \frac{f_{clk}}{SLOTS} = \frac{100\ MHz}{1024} = 97.66\ kHz$$

(13)

- PWM: כל pwm סופר cnt מ-0 עד SLOTS−1, ו-pwm_signal=1 כל עוד cnt<value_reg, כלומר מחזור העבודה הוא value_reg/SLOTS (משוואה 14). value_reg נדגם רק בסוף מחזור (cnt_wrap), כך ששינוי ערך באמצע מחזור אינו קוטע אותו. pwm_cluster מחבר את שלושת ה-pwm לנורה שב-pwm_led, והנורה השנייה כבויה.

$$duty = \frac{value_reg}{SLOTS},\ 0 \le value_reg \le SLOTS-1\ \Rightarrow\ 0 \le duty \le \frac{1023}{1024} = 99.9\% \quad (14)$$

- PC: tx_mac משדר את cie_pixel בהודעה {Rdddd,Gdddd,Bdddd} של 19 בתים (משוואה 15). ארבע ספרות עשרוניות מספיקות גם ב-8192 סלוטים, שבהם הערך הגדול ביותר הוא 8191. binary_2_bcd ממיר כל ערך בשיטת double dabble, הזזה אחת לכל ביט, ומתחיל עם בית הפתיחה של כל קבוצה. הספרות נדרשות רק שני בתים אחר כך, ולכן ההמרה מסתיימת הרבה לפני שהן נשלחות (משוואה 16). start שמגיע באמצע הודעה נשמר ב-start_mid_trans, והודעה נוספת יוצאת מיד אחרי בית הסגירה, אחת בלבד גם אם הגיעו כמה.

$$N_{tx} = 3\cdot(1+1+4)+1 = 19\ bytes \ \Rightarrow\ T_{msg} = 19\cdot T_{frame} = 19\cdot 173.6\ \mu s = 3.3\ ms \quad (15)$$

$$T_{bcd} = SLOTS_WIDTH + 2 = 12\ clk \ \ll\ 2\cdot T_{frame} = 2\cdot 10\cdot 1736 = 34720\ clk \quad (16)$$

- אילוצי תזמון: בתכנון שעון אחד (create_clock על clk). הכניסות מסונכרנות בתוך התכנון והיציאות איטיות ביחס לשעון, ולכן קובץ ה-XDC מגדיר להן set_false_path.
- סימולציה: כשההגדרה SIM קיימת (רק בסימולציה), lab6_definitions.svh נותן ערכים שמקצרים את הריצה: BAUD_RATE הוא 6.25MHz, ביט של 16 מחזורים; DEB_COUNT הוא 10; DISPLAY_RR הוא 10MHz, tick כל 10 מחזורים; SLOTS הוא 64, מחזור PWM של 64 מחזורים.

טבלה 7: גודלי ה-PWM הנתמכים (משוואה 13).

SLOTS	SLOTS_WIDTH	cie_scaling block	Shift of the table value	PWM frequency	Full-scale R, B	Full-scale G
512	9	SH_DOWN	>> 1 (LSB lost)	195.31 kHz	511	258
1024 (default)	10	SH_DOWN	none	97.66 kHz	1023	518
2048	11	SH_UP	<< 1	48.83 kHz	2046	1038
4096	12	SH_UP	<< 2	24.41 kHz	4092	2077
8192	13	SH_UP	<< 3	12.21 kHz	8184	4155

2.5. השוואה בין הנרמולים של ה-CIE

נקודת המוצא היא נוסחת הבהירות של Rec.601 (משוואה 17). המקדם של כל צבע אומר כמה הוא תורם לבהירות שהעין רואה כשהוא בעוצמה מלאה. לבן, שבו שלושת הצבעים בעוצמה מלאה, נותן Y=1, והירוק לבדו תורם ממנו 0.587. כלומר באותו ערך הירוק נראה בהיר פי 1.96 מהאדום ופי 5.15 מהכחול.

$$Y = 0.299\cdot R + 0.587\cdot G + 0.114\cdot B, \quad R=G=B=1 \ \Rightarrow\ Y = 0.299+0.587+0.114 = 1 \quad (17)$$

נרמול לצבע מסוים מכפיל כל צבע במקדם s, כך שבעוצמה מלאה כל הצבעים יתרמו ל-Y בדיוק כמו צבע הנרמול. לכן המקדם של כל צבע הוא המקדם של צבע הנרמול בנוסחה (K_X במשוואה) חלקי המקדם שלו, וצבע הנרמול עצמו מקבל 1 (משוואה 18). מקדם גדול מ-1 אי אפשר לממש, כי מחזור עבודה לא עובר 100%, ולכן הוא נחתך ל-1, והצבע הזה נשאר חלש מצבע הנרמול.

$$0.299\cdot s_R = 0.587\cdot s_G = 0.114\cdot s_B = K_X \ \Rightarrow\ s_R = \frac{K_X}{0.299},\ s_G = \frac{K_X}{0.587},\ s_B = \frac{K_X}{0.114} \quad (s \le 1) \quad (18)$$

כדי לממש את המקדמים בלי כפל, כל מקדם קטן מ-1 נכתב כסכום של חזקות של ½. כל איבר מהצורה 1/2 בחזקת n הוא הזזה ימינה ב-n ביטים, ולכן הכפל הופך לכמה הזזות ולמחברים ביניהן (משוואה 19). טבלה 8 מראה את החישוב לכל נרמול: המקדם של כל צבע, הקירוב שלו בחזקות של ½, ההזזות ב-RTL והחלק של הצבע ב-Y כשהוא בעוצמה מלאה. בשורה של הלבן מופיע Y של הלבן, כלומר סכום החלקים.

$$0.509 \approx \frac{1}{2} + \frac{1}{128} = 0.5078 \ \Rightarrow\ 0.509\cdot G \approx (G \gg 1) + (G \gg 7) \quad (19)$$

בנרמול לכחול שלושת הצבעים תורמים ל-Y אותו חלק, 0.114, ולכן ערכים שווים נראים בהירים באותה מידה, אבל הלבן מקבל רק 34% מהבהירות. בנרמול לאדום האדום והירוק תורמים 0.299 כל אחד, והכחול, שהמקדם שלו נחתך ל-1, תורם 0.114 ונראה חלש מהם פי 2.62. הלבן מקבל 71% מהבהירות. בנרמול לירוק כל המקדמים נחתכים ל-1, כלומר נרמול לירוק לא משנה דבר. הגרסה הראשונה הכפילה כל צבע במקדם שלו חלקי המקדם של הירוק, הפוך מהנרמול: הצבעים החלשים נחלשו עוד, והירוק תרם פי 26.5

מהכחול, יותר מאשר בלי CIE בכלל. מכאן הלבן הירקרק שנמצא בבדיקה הראשונה על הלוח (סעיפים 7.3 ו-8.2).

טבלה 8: הנרמול לכל צבע מתוך הנוסחה של Rec.601: המקדם, ההזזות והחלק של כל צבע ב-Y.

Normalised to	Colour	Scale s	As powers of 1/2	RTL (shifts)	Part of Y
Blue (0.114)	R	0.114/0.299 = 0.381	1/4+1/8+1/128 = 0.383	(R>>2)+(R>>3)+(R>>7)	0.114
	G	0.114/0.587 = 0.194	1/8+1/16+1/128 = 0.195	(G>>3)+(G>>4)+(G>>7)	0.114
	B	0.114/0.114 = 1	1	B	0.114
	White				Y = 0.342
Red (0.299), final	R	0.299/0.299 = 1	1	R	0.299
	G	0.299/0.587 = 0.509	1/2+1/128 = 0.508	(G>>1)+(G>>7)	0.299
	B	0.299/0.114 = 2.62 → 1	1	B	0.114
	White				Y = 0.712
Green (0.587)	R	0.587/0.299 = 1.96 → 1	1	R	0.299
	G	0.587/0.587 = 1	1	G	0.587
	B	0.587/0.114 = 5.15 → 1	1	B	0.114
	White				Y = 1
First version	R	0.299/0.587 = 0.509	1/2	R>>1	0.152
	G	0.587/0.587 = 1	1	G	0.587
	B	0.114/0.587 = 0.194	1/8+1/16+1/128 = 0.195	(B>>3)+(B>>4)+(B>>7)	0.022
	White				Y = 0.761

טבלה 9 מראה מה כל נרמול נותן במימוש, ב-1024 סלוטים: התשובה ב-TX ללבן, כמה רמות שונות נשארות לכל צבע מתוך 256 ערכי ה-sRGB, מאיזה ערך sRGB כל צבע נדלק, וכמה מחברים צריך.

טבלה 9: הנרמולים כפי שהם ממומשים, ב-1024 סלוטים.

Normalised to	TX reply for white	Levels R / G / B	First lit sRGB R / G / B	Adders
Blue	{R0389,G0197,B1023}	183 / 127 / 238	12 / 21 / 2	4
Red (final)	{R1023,G0518,B1023}	238 / 217 / 238	2 / 5 / 2	1
Green (= no CIE)	{R1023,G1023,B1023}	238 / 238 / 238	2 / 2 / 2	0
First version	{R0511,G1023,B0197}	217 / 238 / 127	5 / 2 / 21	2

איזה נרמול עדיף: הגרסה הראשונה מגדילה את חוסר האיזון במקום להקטין אותו, ונרמול לירוק לא עושה דבר, ולכן הבחירה היא בין הכחול לאדום. הכחול הוא הנרמול היחיד שבו כל הצבעים מאוזנים, ואם הדרישה היא צבע מדויק, הוא הנכון. המחיר שלו גבוה: הלבן חשוך פי 2.1 מאשר בנרמול לאדום, לאדום נשארות 183 רמות ולירוק 127, והירוק נדלק רק מ-21 sRGB.

כדי לראות למה דווקא האדום, אפשר להסתכל על הנרמול כעל רמה: כל צבע מוחלש כך שיתרום ל-Y לכל היותר את הרמה הזאת, וצבע שהמקדם שלו קטן ממנה נשאר בעוצמה מלאה. נרמול לכחול הוא רמה 0.114, נרמול לאדום רמה 0.299 ונרמול לירוק רמה 0.587. משוואה 20 מראה את Y של הלבן בכמה רמות. מ-0.114 עד 0.299 גם האדום וגם הירוק עולים, ולכן Y עולה פי 2 מהרמה: 0.370 על עלייה של 0.185. מעל 0.299 האדום כבר בעוצמה מלאה, רק הירוק ממשיך לעלות, ו-Y עולה רק כמו הרמה: 0.288 על עלייה של

0.288. בנוסף, מעל 0.299 האדום והירוק כבר לא שווים. כלומר הנרמול לאדום הוא הבהיר ביותר שעדיין מאזן את האדום והירוק, ומעבר לו כל תוספת בהירות קונה חצי ממה שקנתה קודם, ובמחיר של איזון.

$$Y(0.114) = 0.342,\quad Y(0.2) = 0.514,\quad Y(0.299) = 0.712,\quad Y(0.4) = 0.813,\quad Y(0.587) = 1 \qquad (20)$$

לכן, לפי הקריטריון שנבחר בתכנון, בהירות לפני איזון מושלם, הנרמול לאדום עדיף: הלבן בהיר פי 2.1 מאשר בנרמול לכחול, האדום, הירוק והצבעים שביניהם, כמו צהוב וכתום, מאוזנים, לאדום ולכחול נשארות 238 רמות ולירוק 217, והמימוש דורש מחבר אחד בלבד. המחיר הוא הכחול, שנראה חלש פי 2.62 מהאדום ומהירוק, ולכן לפי הנוסחה הלבן נוטה לצהוב, וכל צבע שיש בו כחול מקבל פחות כחול ממה שצריך. על הלוח הלבן בנרמול לאדום נראה דווקא הקרוב ביותר ללבן מבין שלוש הגרסאות (סעיף 7.3).

2.6. קוד המקור

קוד כל המודולים, החבילה lab6_pkg, קובץ ההגדרות ושורות ה-XDC הפעילות מובאים בנספח א'. הקוד הוא של הגרסה הסופית, שבה cie_scaling מנורמל לאדום (סעיף 2.5).

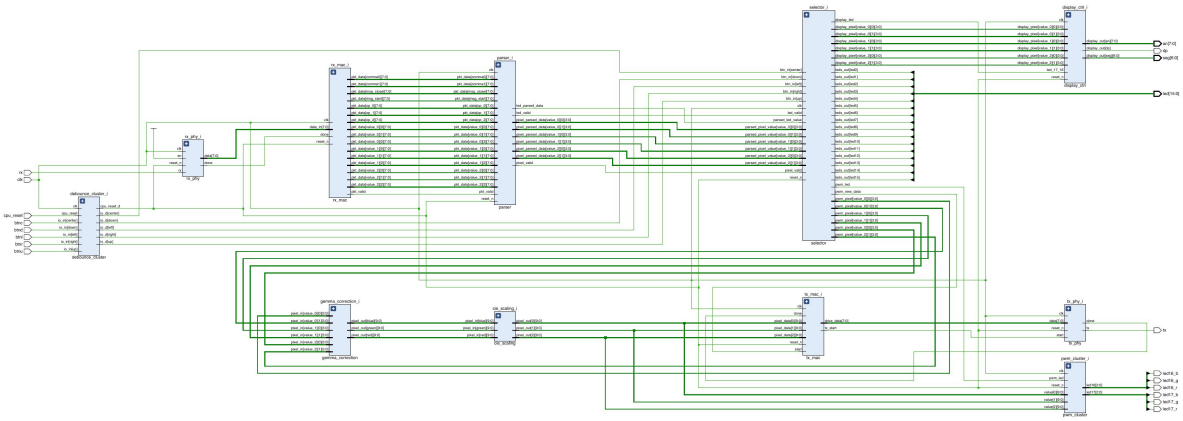
3. בדיקת Lint

טבלה 10: תוצאות בדיקות ה-Lint וה-Elaborated Design.

Check	Scope	Result
RTL Linter (Run Linter)	chip_top + all sub-modules	0 violations
Elaborated design – Report DRC	chip_top	0 checks
Elaborated design – Report Methodology	chip_top	0 checks

ה-Linter רץ על chip_top עם כל תת-המודולים, ודוחות ה-DRC וה-Methodology של ה-Elaborated Design נקיים (טבלה 10). סכמת ה-RTL מוצגת באיור 7.

3.1. סכמת RTL (Elaborated Design)



איור 7: סכמת ה-RTL של chip_top: אחד-עשר תת-המודולים ופורטי ה-struct שלהם מפורקים לשדות.

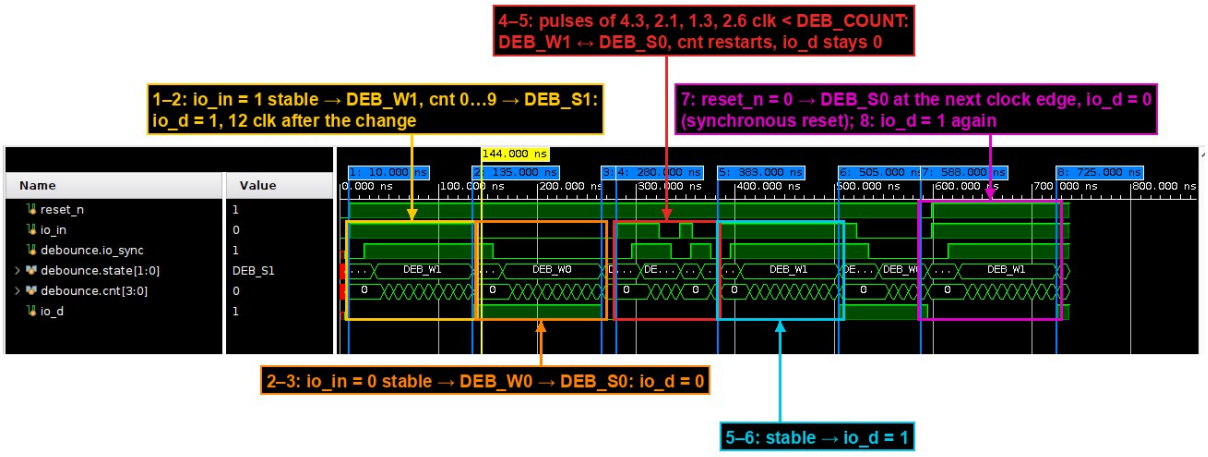
4. תוצאות סימולציה

כל הטסטבנצ'ים רצו עם ההגדרה SIM, ו-assert שנכשל מודפס בלוג כ-Error. בכל סעיף מובאות תוצאת הריצה וצורות הגל, וההסבר מופיע על גבי האיור.

בשל מגבלת הזמן, ומכיוון שהמעבדה בוצעה ביחידות, ארבעה טסטבנצ'ים נכתבו במלואם בעזרת AI, והם tb_selector, tb_pwm_cluster, tb_tx_mac ו-tb_chip_top.

4.1. בדיקת debounce – tb_debounce

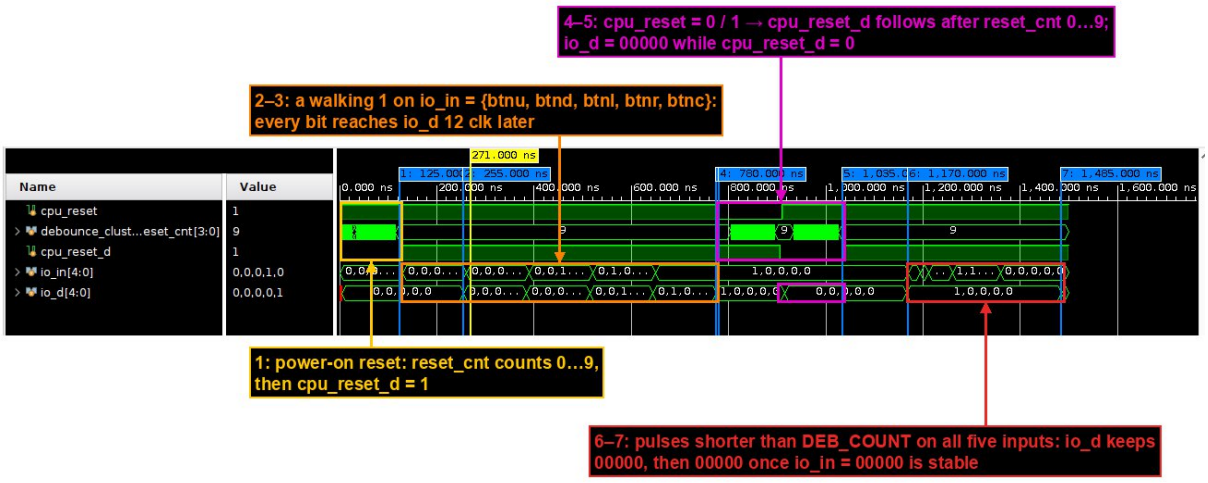
הריצה הסתיימה ב-738ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 8.



איור 8: tb_debounce – כל הריצה: שינוי יציב, פולסים קצרים ואיפוס.

4.2. בדיקת debounce_cluster – tb_debounce_cluster

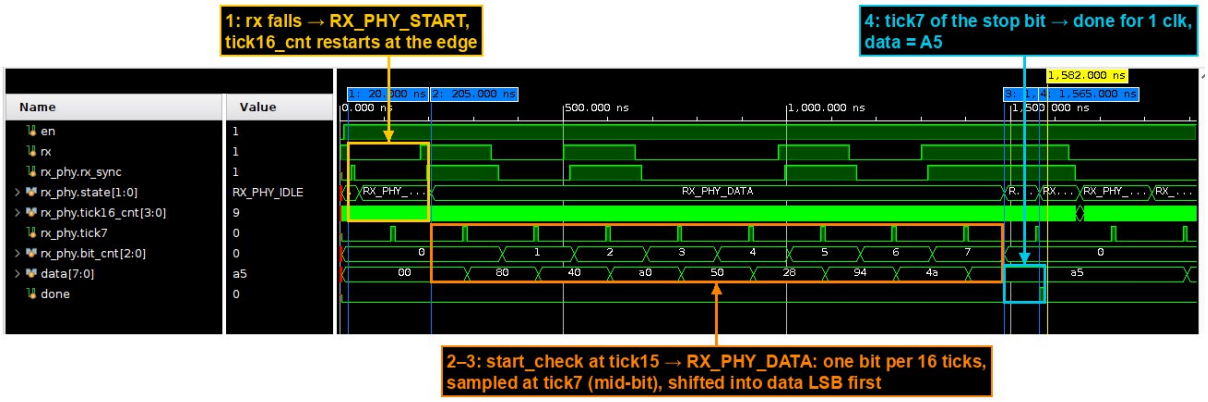
הריצה הסתיימה ב-1500ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 9.



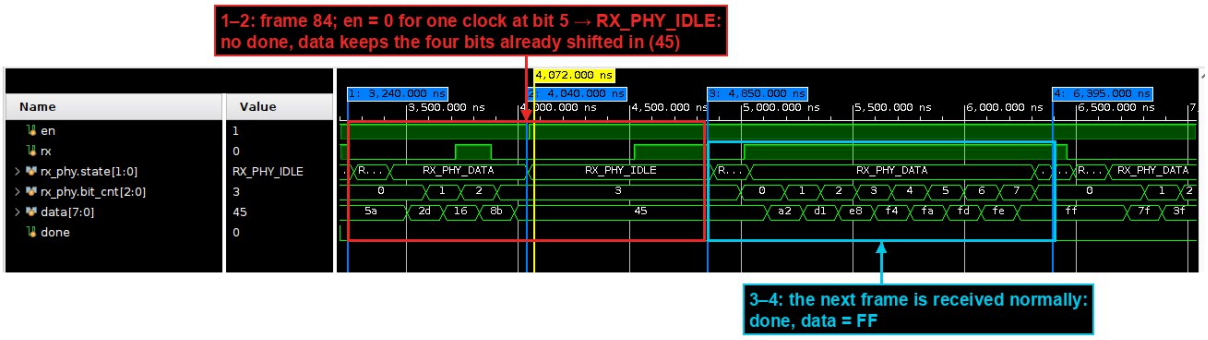
איור 9: tb_debounce_cluster – כל הריצה: power-on reset, לחצן אחר בכל פעם, איפוס ופולסים קצרים.

4.3. בדיקת rx_phy – tb_rx_phy

הריצה הסתיימה ב-8070ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 10 ו-11.



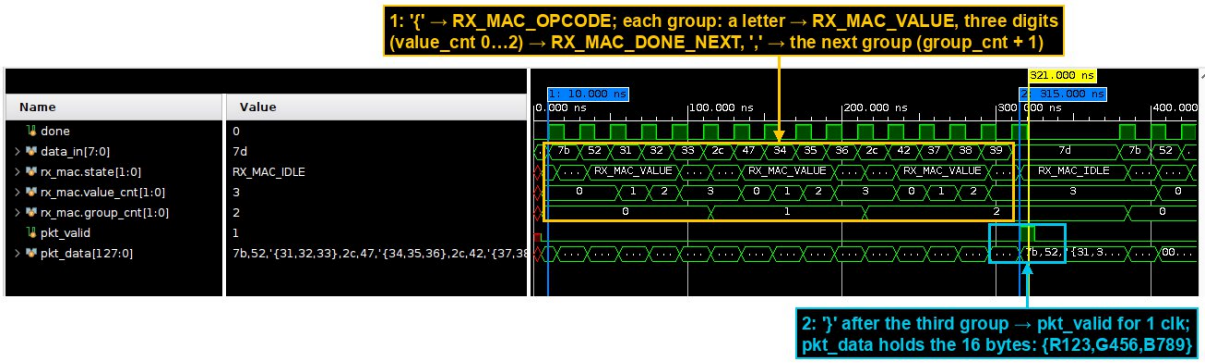
איור 10: tb_rx_phy – זום: קליטת הבית 0xA5.



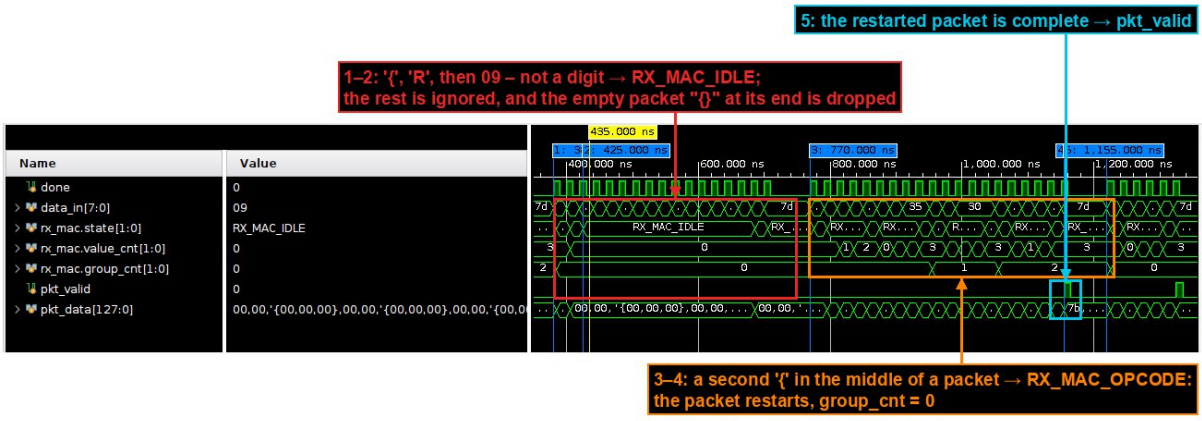
איור 11: tb_rx_phy – זום: en=0 באמצע הפריים 0x84.

4.4. בדיקת rx_mac – tb_rx_mac

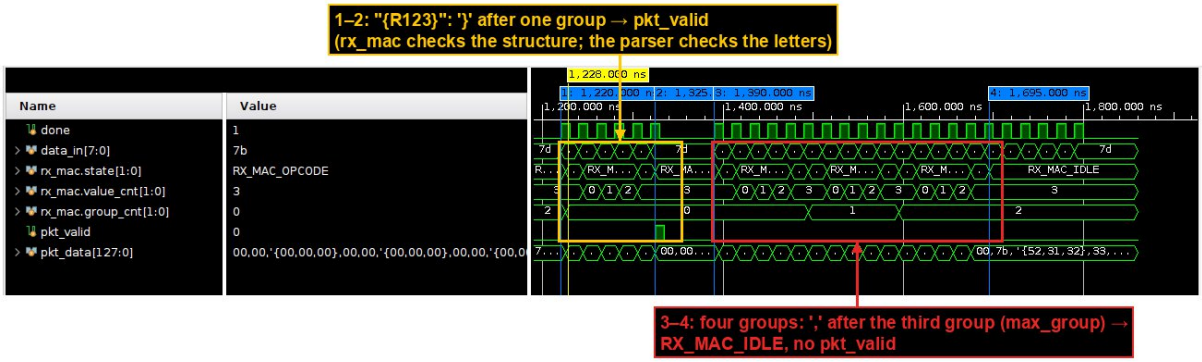
הריצה הסתיימה ב-1860ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 12 עד 14.



איור 12: tb_rx_mac – זום: ההודעה {R123,G456,B789}.



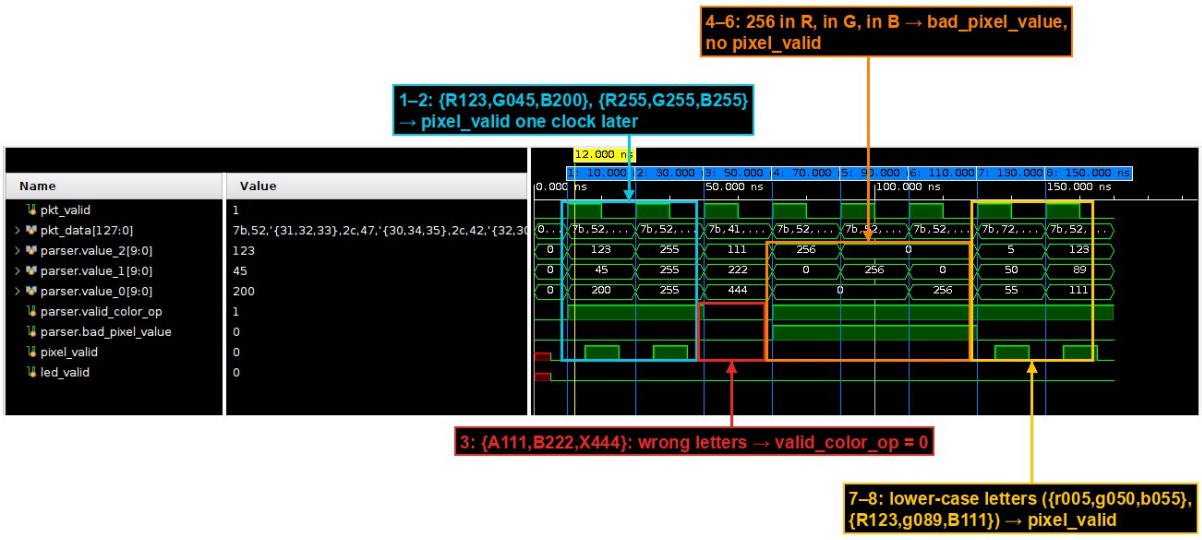
איור 13: tb_rx_mac – זום: תו שאינו ספרה, הודעה ריקה ו-{ באמצע הודעה.



איור 14: tb_rx_mac – זום: הודעה של קבוצה אחת והודעה של ארבע קבוצות.

4.5. tb_parser – בדיקת parser

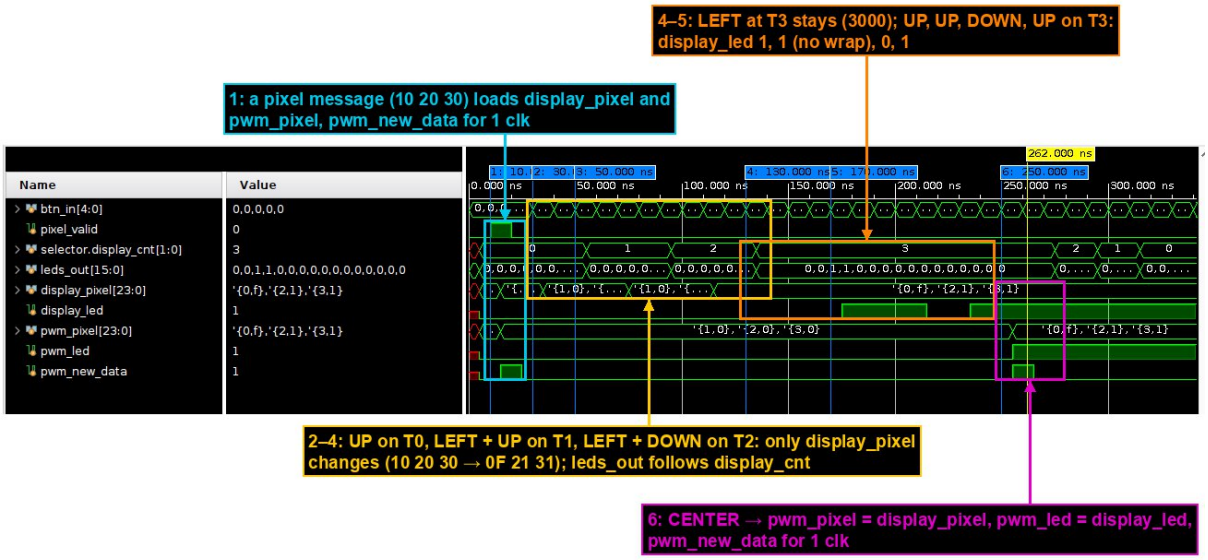
הריצה הסתיימה ב-170ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 15.



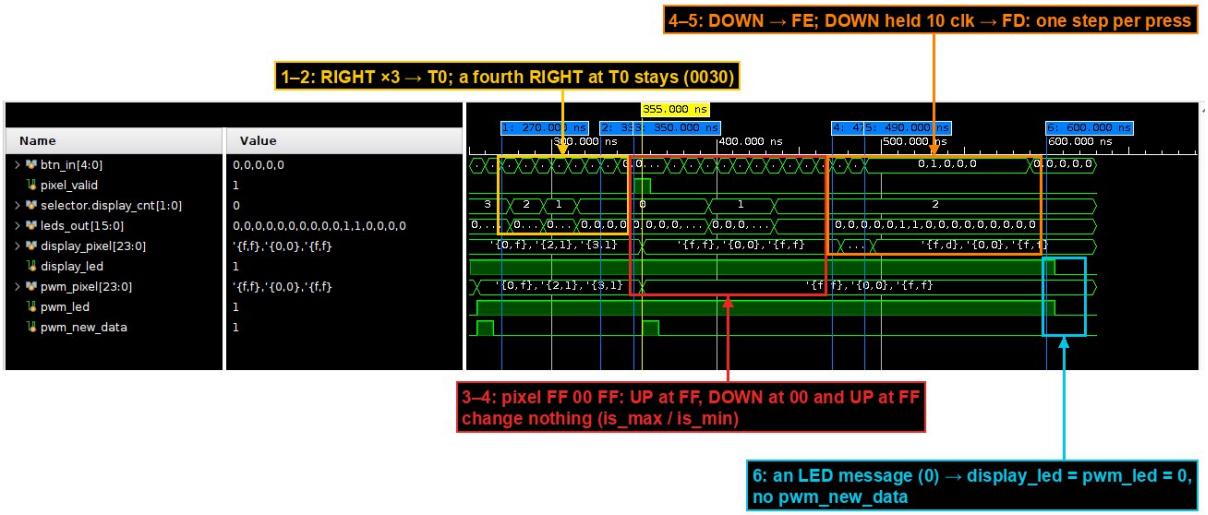
איור 15: tb_parser – כל הריצה: שמונה הודעות.

4.6. tb_selector – בדיקת selector

הריצה הסתיימה ב-630ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 16 ו-17.



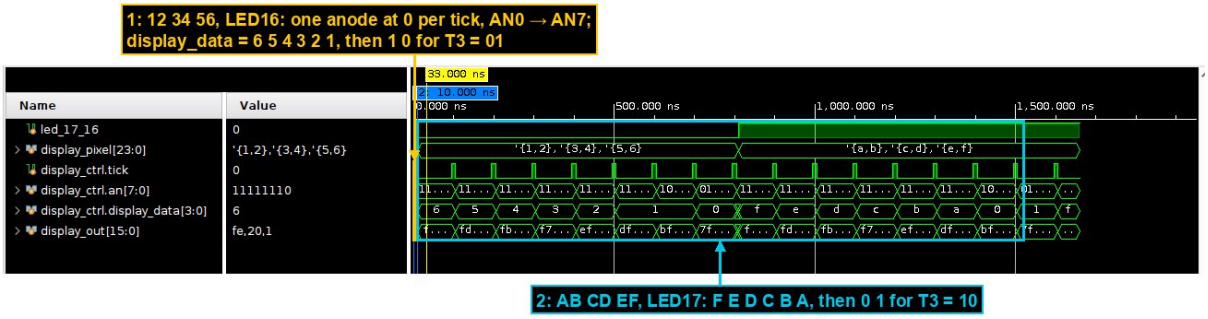
איור 16: tb_selector – זום: הודעת פיקסל, עריכה בלחצנים ו-Center.



איור 17: tb_selector – זום: קצוות הבחירה והערכים, החזקה של לחצן והודעת נורה.

4.7. tb_display_ctrl – בדיקת display_ctrl

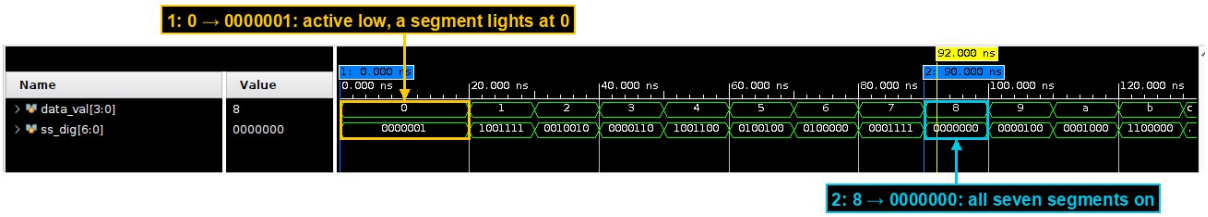
הריצה הסתיימה ב-1660ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 18.



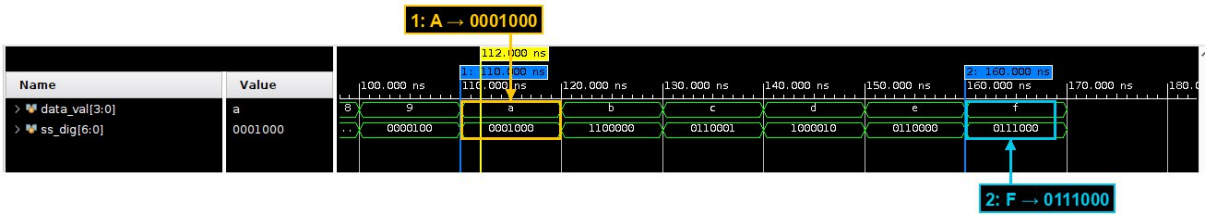
איור 18: tb_display_ctrl – זום: סריקה של שמונה הספרות עם LED16 ועם LED17.

4.8. tb_seven_segment – בדיקת seven_segment

בטסטבנץ' זה אין assertions, ולכן התוצאה נבדקת בצורות הגל באיורים 19 ו-20, והיא כצפוי. הריצה הסתיימה ב-170ns.



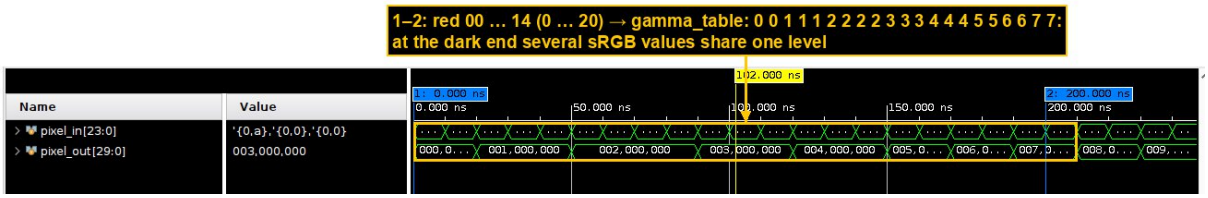
איור 19: tb_seven_segment – זום: הספרות 0 עד 9.



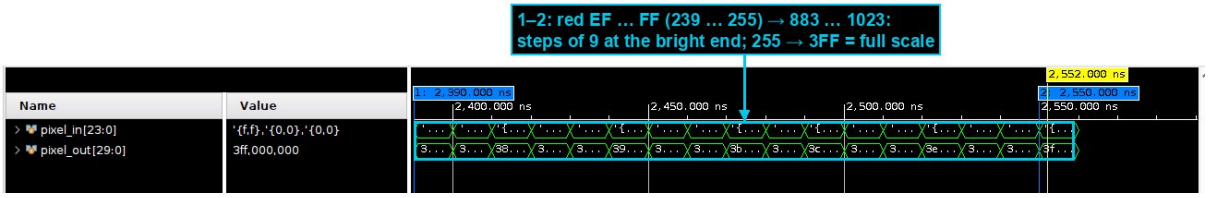
איור 20: tb_seven_segment – זום: האותיות A עד F.

4.9. tb_gemma_correction – בדיקת gemma_correction

הריצה הסתיימה ב-2560ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 21 ו-22.



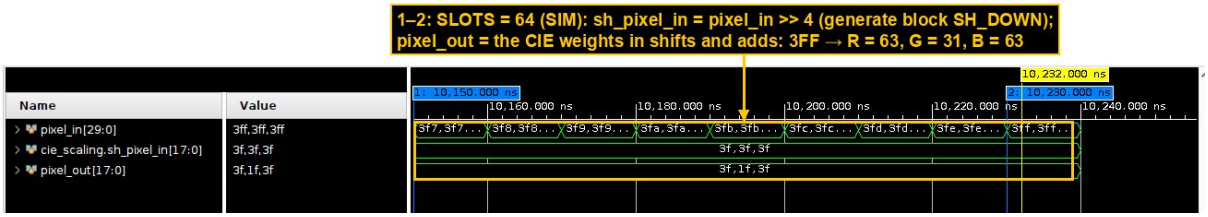
איור 21: tb_gemma_correction – זום: ערכי האדום הנמוכים.



איור 22: tb_gemma_correction – זום: ערכי האדום הגבוהים.

4.10. tb_cie_scailing – בדיקת cie_scaling

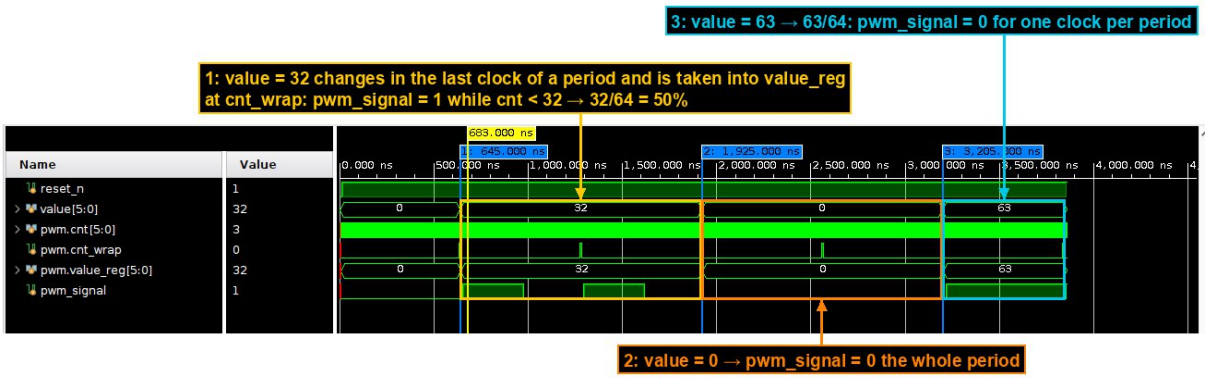
הריצה הסתיימה ב-10240ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 23.



איור 23: tb_cie_scailing – זום: הערכים הגבוהים, SLOTS=64, נרמול לאדום.

4.11. tb_pwm – בדיקת pwm

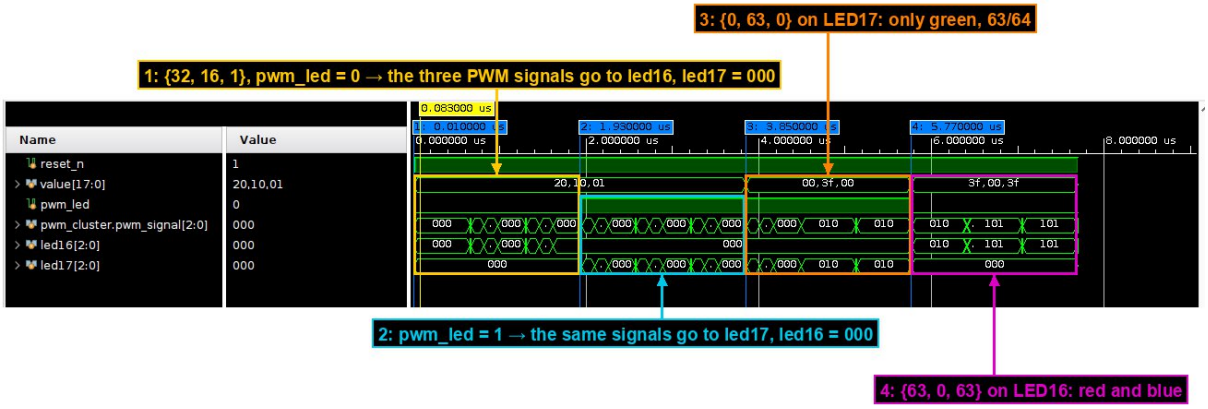
הריצה הסתיימה ב-3860ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 24.



איור 24: tb_pwm – זום: שלושת הערכים, SLOTS=64.

4.12. tb_pwm_cluster – בדיקת pwm_cluster

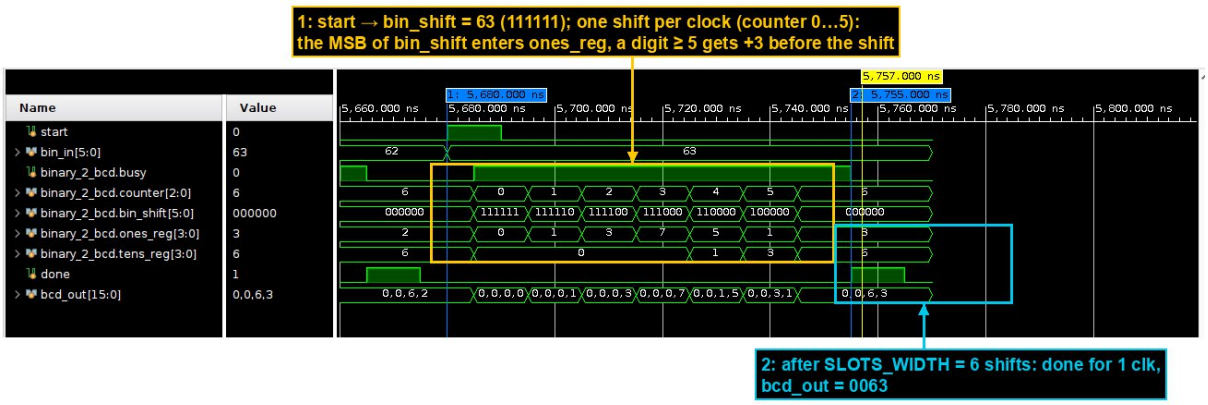
הריצה הסתיימה ב-7700ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 25.



איור 25: tb_pwm_cluster – כל הריצה: שלושה ערכים ב-LED16, אותם ערכים ב-LED17 והקצוות.

4.13. tb_binary_2_bcd – בדיקת binary_2_bcd

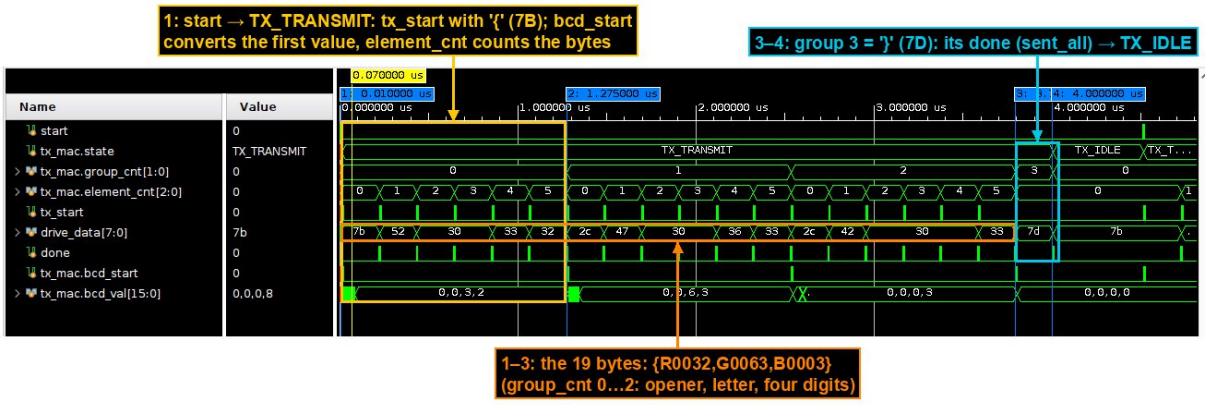
הריצה הסתיימה ב-5770ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 26.



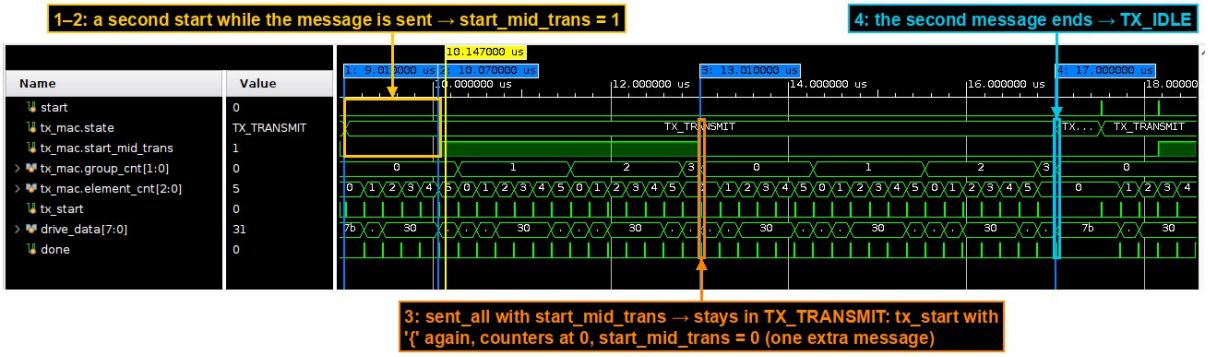
איור 26: tb_binary_2_bcd – זום: המרה של הערך 63.

4.14. tb_tx_mac – בדיקת tx_mac

הריצה הסתיימה ב-26030ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 27 ו-28.



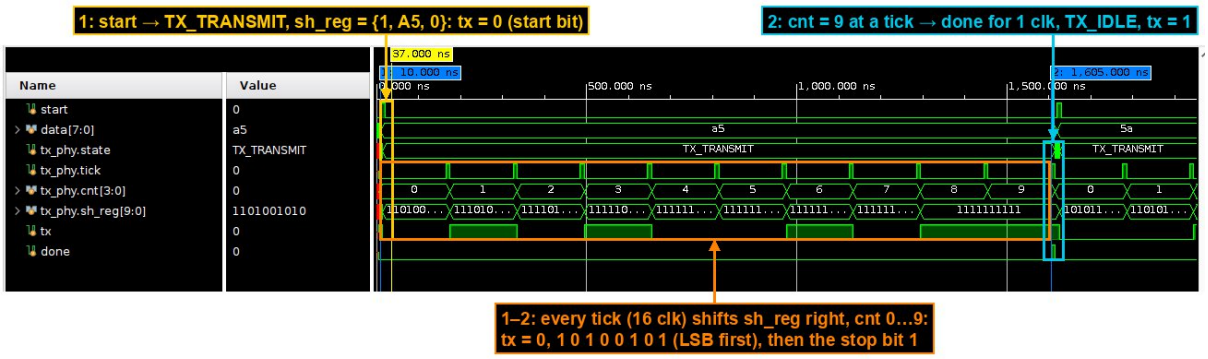
איור 27: tb_tx_mac – זום: הודעה אחת של 19 בתים.



איור 28: tb_tx_mac – זום: start באמצע הודעה והודעה נוספת אחריה.

4.15. tb_tx_phy – בדיקת tx_phy

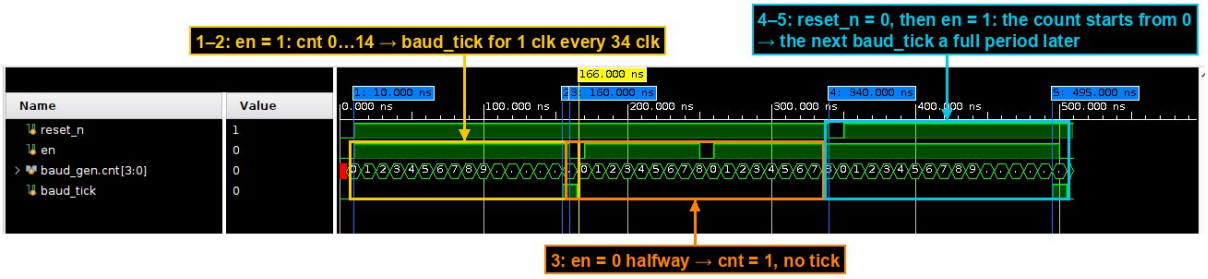
הריצה הסתיימה ב-7450ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 29.



איור 29: tb_tx_phy – זום: שידור הבית 0xA5.

4.16. tb_baud_gen – בדיקת baud_gen

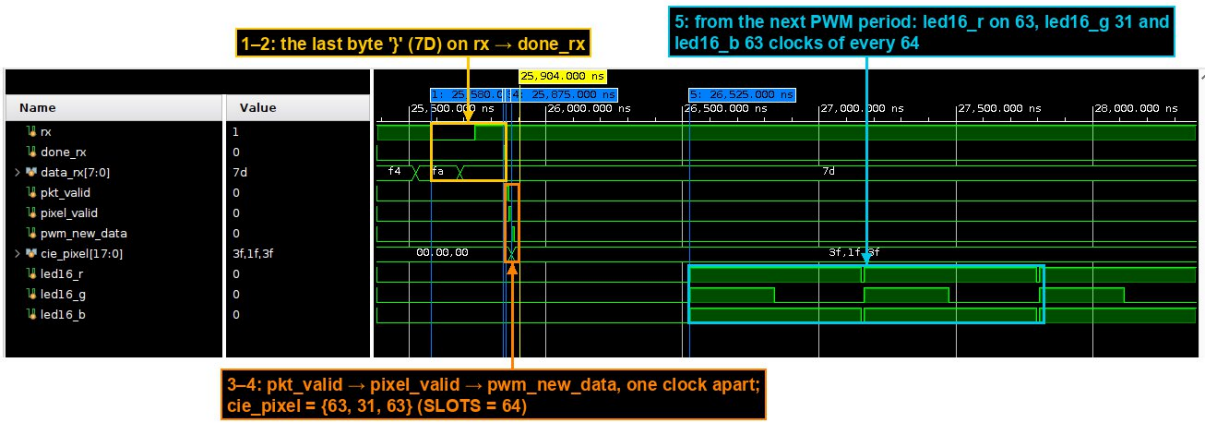
הריצה הסתיימה ב-510ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 30.



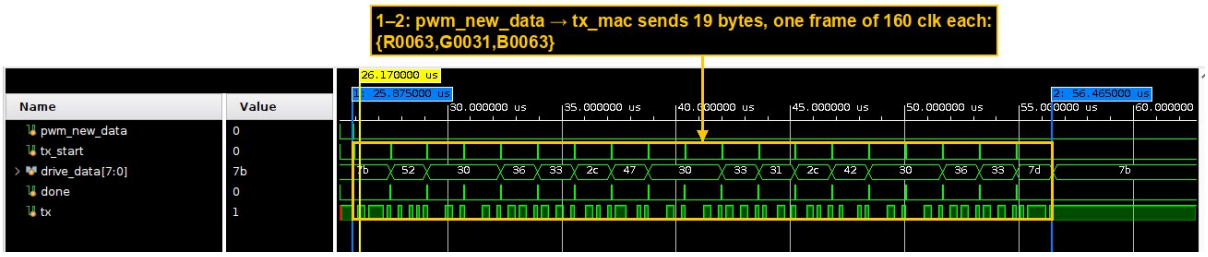
איור 30: tb_baud_gen – כל הריצה: ספירה מלאה, en=0 באמצע ואיפוס.

4.17. tb_chip_top – בדיקת המערכת המלאה

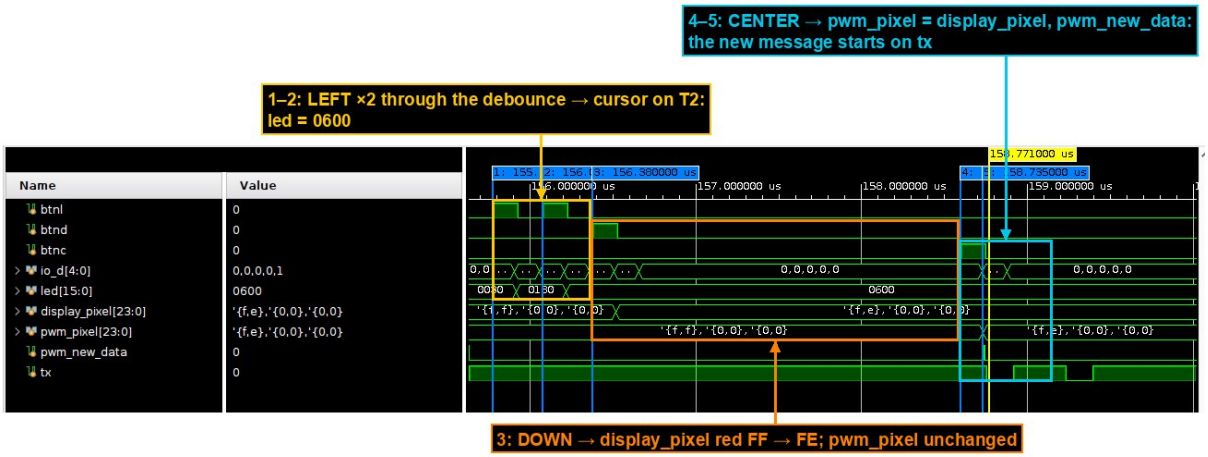
הריצה הסתיימה ב-259100ns בלי Error בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 31 עד 33.



איור 31: tb_chip_top – זום: ההודעה {R255,G255,B255} עד ה-PWM של LED16.



איור 32: tb_chip_top – זום: הודעת המשוב ב-tx.



איור 33: tb_chip_top – זום: Left, Down ו-Center, וההודעה שנשלחת.

5. סינתזה ומימוש (Synthesis & Implementation)

5.1. סינתזה

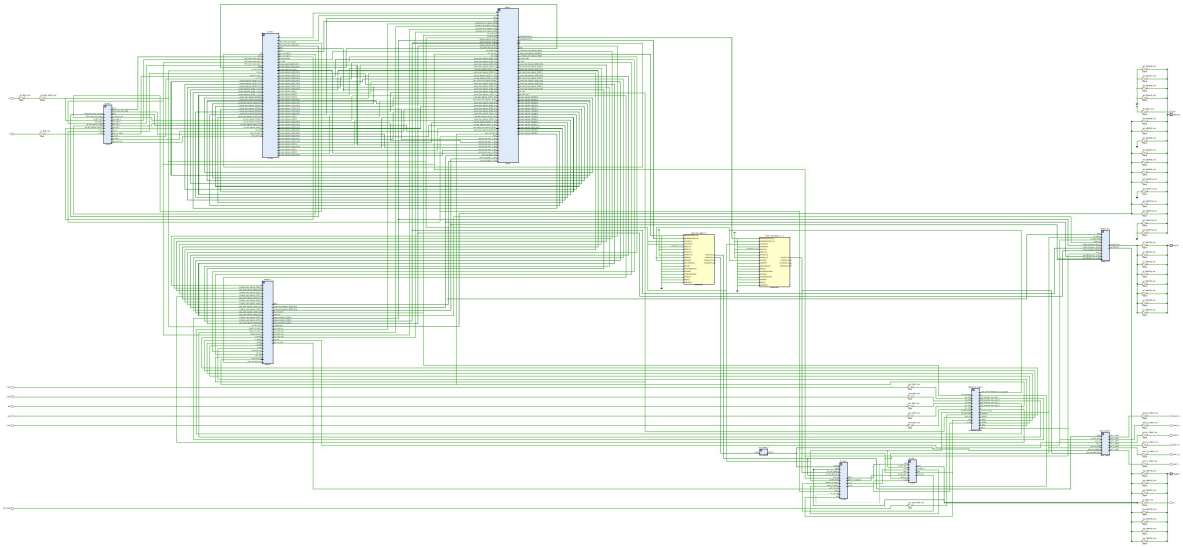
טבלה 11: תוצאות הסינתזה ודוחות ה-Synthesized Design.

Report	Result
Messages (synthesis.log)	0 errors, 0 critical warnings, 9 warnings
Timing, worst case (timing_summary_synth.rpt)	100 MHz: setup WNS +2.288 ns, hold WHS +0.079 ns – met
DRC (drc_synth.rpt)	0 checks
Methodology (methodology_synth.rpt)	2 × SYNTH-6 (Warning: Timing of a RAM block might be sub-optimal)
Clock networks / interaction	one clock: sys_clk_pin 100 MHz on clk (1 BUFG); only sys_clk_pin → sys_clk_pin paths
Noise (noise_synth.rpt)	39 output pins PASS, 0 FAIL
Power (power_synth.rpt)	0.116 W total on-chip (estimate)

28 אזהרות מסוג Synth 8-7129 עוסקות בביטים של pkt_data שה-parser אינו קורא: הפסיק השני (comma1), בית הסגירה (msg_close) וה-nibble העליון של שלוש הספרות ב-value_0. אלה כניסות של ה-parser בלי עומס, ואת המבנה שלהן כבר בדק rx_mac. 9 אזהרות מסוג Synth 8-3917 עוסקות ביציאות קבועות: dp=1, והביטים led[11], led[6], led[3:0] ו-led[15:14], שאינם מסמנים שדה, הם 0.

שתי אזהרות ה-Methodology מסוג SYNTH-6 עוסקות בטבלת הגאמה. ב-gemma_correction שלוש קריאות קומבינטוריות מהטבלה, ובסינתזה של המודול לבדו היא ממופה ל-LUTs. בתכנון המלא הסינתזה משטחת את ההיררכיה ורואה שהכתובת של הטבלה יוצאת ישר מהרגיסטר pwm_pixel שב-selector. Block RAM הוא רכיב סינכרוני, שדוגם את הכתובת ברגיסטר פנימי, ולכן הסינתזה מכניסה את pwm_pixel לתוך ה-RAM כרגיסטר הכתובת וממפה את שלוש הקריאות ל-2 בלוקים של RAMB18E1, שלכל אחד מהם שני פורטי קריאה. ה-WE של הבלוקים מחוברים ל-0, כלומר הבלוקים הם ROM, ו-pwm_pixel כבר לא קיים בנטליסט (פרק 6).

ל-RAMB18E1 יש גם רגיסטר יציאה אופציונלי, אבל אחרי הטבלה באים המחבר של הירוק ב-cie_scaling ושני יעדים שונים, value_reg ב-pwm ו-bin_shift ב-binary_2_bcd, ואין דלגלג אחד שאפשר להכניס לבלוק (DOA_REG=0). האזהרה, אחת לכל בלוק, אומרת שבלי רגיסטר היציאה זמן ה-clock-to-out של הבלוק ארוך יותר, 2.454ns במסלול הגרוע. זו הערה על ביצועים ולא על נכונות: המסלול הגרוע ביותר שעובר דרך הבלוקים עומד במרווח של 3.565ns, והמסלול הגרוע ביותר בתכנון כולו, במרווח של 2.839ns, לא עובר בהם. ההחלטה להשאיר את המיפוי מוסברת בסעיף 8.1.



איור 34: סכמת ה-post-synthesis של chip_top.

5.2. מימוש (Implementation)

טבלה 12: תוצאות המימוש ודוחות ה-Implemented Design.

Report	Result
Messages (implementation.log)	0 errors, 0 critical warnings, 0 warnings
Timing, worst case (timing_summary_routed.rpt)	setup WNS +2.839 ns on rx_mac_i → parser_i (6.945 ns, 9 logic levels); hold WHS +0.092 ns – met
Route status (route_status.rpt)	1450 logical nets, 962 routable, 962 fully routed, 0 with routing errors
DRC (drc_routed.rpt)	0 checks
Methodology (methodology_drc_routed.rpt)	2 × SYNTH-6 (Warning: Timing of a RAM block might be sub-optimal)
Power (power_routed.rpt)	0.113 W total: 0.097 W static + 0.016 W dynamic; junction 25.5 °C
I/O (io_placed.rpt)	47 of 47 ports placed (FIXED) on the XDC pins, LVCMOS33

תוצאות המימוש מרוכזות בטבלה 12. סכמות ה-post-implementation תואמות את סכמת ה-post-synthesis, ולכן אינן מוצגות.

6. ניצול משאבים

הצפי למספר הדלגלגים נספר מה-RTL לפני הסינתזה, עם ערכי החומרה: בכל מודול נספרו כל הרגיסטרים לפי הרוחב שלהם. רוחב מונה הוא $clog2 של מספר הערכים שהוא מקבל (משוואות 21 ,13 ,9 ו-22). מכונת מצבים תופסת את רוחב ה-enum שלה, וכל כניסה מסונכרנת תופסת שני דלגלגים. בטבלה 13 רגיסטר בלי רוחב הוא ביט אחד, והצפי מושווה לסינתזת ה-OOC של כל מודול ולתכנון המלא.

$$W_{baud} = \lceil log_2 1736 \rceil = 11,\ W_{tick} = \lceil log_2 108 \rceil = 7,\ W_{display} = \lceil log_2 25000 \rceil = 15 \quad (21)$$

$$W_{bcd} = \lceil log_2 (10+1) \rceil = 4,\ W_{element} = \lceil log_2 6 \rceil = 3 \quad (22)$$

טבלה 13: צפי מול בפועל: דלגלגים בכל תת-מודול של chip_top ובתכנון המלא.

Module	Registers (name × width)	Expected FF	Actual FF (OOC)	Δ
debounce_cluster	5 × debounce (cnt×21, io_meta, io_sync, state×2); reset_meta, reset_sync, reset_prev, reset_cnt×21, cpu_reset_d	150	150	0
rx_phy	tick_cnt×7, tick16_cnt×4, bit_cnt×3, data×8, start_samples×3, rx_meta, rx_sync, rx_prev, state×2, start_check, done	32	34	+2
rx_mac	pkt_data×128, value_cnt×2, group_cnt×2, state×2, pkt_valid	135	137	+2
parser	pixel_parsed_data×24, pixel_valid, led_valid, led_parsed_data	27	27	0
selector	btn_prev×5, display_cnt×2, display_pixel×24, pwm_pixel×24, pwm_new_data, display_led, pwm_led	58	58	0
display_ctrl	an×8; baud_gen: cnt×15	23	23	0
gemma_correction	– (gamma_table: a ROM, combinational)	0	0	0
cie_scaling	– (shifts and adds, combinational)	0	0	0
pwm_cluster	3 × pwm (cnt×10, value_reg×10, pwm_signal)	63	63	0
tx_mac	group_cnt×2, element_cnt×3, state, start_mid_trans, tx_start; binary_2_bcd: bin_shift×10, ones/tens/hundreds/thousands_reg×4 each, counter×4, busy, done	40	39	−1
tx_phy	sh_reg×10, cnt×4, state; baud_gen: cnt×11	26	25	−1
Sum of modules		554	556	+2
Full design (synth / placed)		554	527 / 535	−27 / −19

ההפרשים:

- rx_phy, +2: הסינתזה משאירה שני עותקים זהים של רגיסטר המצבים, עם אותה לוגיקת מצב הבא. העותק שזיהוי ה-FSM קידד (FSM_sequential_state_reg) מזין את לוגיקת המצב הבא, והעותק השני (state_reg) מזין את ה-enable של המונים ושל רגיסטרי ההזזה.
- rx_mac, +2: מכונת המצבים קודדה ב-one-hot, ארבעה דלגלגים לארבעה מצבים במקום שניים (Synth 8-3354 בלוג הסינתזה).
- tx_mac, −1: היציאה done של binary_2_bcd מחוברת ב-tx_mac לחוט bcd_done שאין לו עומס, ולכן הרגיסטר שלה נמחק. ב-binary_2_bcd לבדו (OOC) הוא קיים.

- tx_phy, −1: הביט העליון של sh_reg הוא תמיד 1: ב-reset, בטעינה ({1, data, 0}) ובכל הזזה נכנס אליו 1, ולכן הסינתזה מחליפה אותו בקבוע.
- התכנון המלא אחרי הסינתזה, 29− מול סכום המודולים: ב-selector חסרים 24 הדלגלגים של pwm_pixel, שנכנסו לשני בלוקי ה-RAMB18 כרגיסטר הכתובת (סעיף 5.1). ב-rx_mac נשארים 132 במקום 137, כי pkt_data הוא רגיסטר הזזה של בתים, והסינתזה ממפה ארבע שרשראות שלו ל-SRL (LUT שמשמש כרגיסטר הזזה).
- אחרי המימוש, 8+ מול הסינתזה: בשלב opt_design נוצר עותק לכל אחד משמונת הביטים של an_reg ב-(an_reg_lopt_replica) display_ctrl. הרגיסטר מזין גם את פיני האנודות וגם את המוקס שבוחר את הספרה.

הצפי ל-IOB הוא 47, כמספר הפינים בקובץ ה-XDC (8 כניסות ו-39 יציאות), והצפי ל-BUFG הוא אחד, לשעון היחיד. שניהם התקבלו כצפוי (טבלה 14). ל-LUTs לא נקבע צפי מדויק, רק הערכה: הרבה LUTs צפויים בטבלת הגאמה אם היא ממומשת בלוגיקה, במוקסים ובמחברים של העריכה ב-selector, ובהשוואות ה-ASCII ב-parser. ב-OOC המודולים הגדולים הם gemma_correction (108), selector (114) ו-parser (87). בתכנון המלא הטבלה של gemma_correction ממופה לשני בלוקי RAMB18 ולא ל-LUTs, וזה חלק מההפרש בין סכום המודולים לתכנון המלא בטבלה 15; השאר מאופטימיזציה בין המודולים אחרי שטיחת ההיררכיה. התכנון כולו תופס פחות מ-1% מה-LUTs ומהדלגלגים של ה-FPGA ונכנס בו בקלות.

הערכים בטבלה 14 לקוחים מדוחות ה-utilization של התכנון המלא (utilization_synth.rpt ו-utilization_placed.rpt), והערכים בטבלה 15 מסינתזה out-of-context של כל מודול בנפרד, עם ערכי החומרה.

טבלה 14: ניצול התכנון המלא אחרי הסינתזה ואחרי המימוש.

Site type	Synthesis	Placed	Available	Util % (placed)
Slice LUTs	439	424	63,400	0.67
of them LUT as memory (SRL)	–	4	19,000	0.02
Slice Registers (flip-flops)	527	535	126,800	0.42
Register as Latch	0	0	126,800	0.00
Slices (placed: 133 SLICEL + 80 SLICEM)	–	213	15,850	1.34
F7 / F8 Muxes	0	0	31,700	0.00
Block RAM Tile	1	1	135	0.74
DSPs	0	0	240	0.00
Bonded IOB (8 inputs + 39 outputs)	47	47	210	22.38
BUFGCTRL	1	1	32	3.13
Unique control sets	32	32	15,850	0.20

טבלה 15: ניצול כל תת-מודול של chip_top בסינתזה out-of-context (כולל תת-המודולים שבתוכו).

Module	Contains	LUTs	CARRY4
debounce_cluster	debounce ×5	60	36
rx_phy	–	43	0
rx_mac	–	39	0
parser	–	87	31
selector	–	114	0
display_ctrl	baud_gen, seven_segment	31	4
gemma_correction	–	108	0
cie_scaling	–	3	3
pwm_cluster	pwm ×3	80	6
tx_mac	binary_2_bcd	68	0
tx_phy	baud_gen	36	0
Sum of modules		669	80
Full design (synth / placed)		439 / 424	80 / 80

7. בדיקה על הלוח

7.1. יצירת ה-bitstream וטעינתו ללוח

טבלה 16: סיכום יצירת ה-bitstream.

Item	Result
Bitstream	chip_top.bit – 0 errors, 0 critical warnings, 0 warnings
Device	xc7a100tcsg324-1 (Digilent Nexys A7-100T)
Clock constraint	create_clock sys_clk_pin, period 10.000 ns on clk (E3)
Programming	JTAG over the on-board USB port; DONE LED on
Terminal	CoolTerm, 57600 bps, 8N1, no flow control (USB-UART of the board)

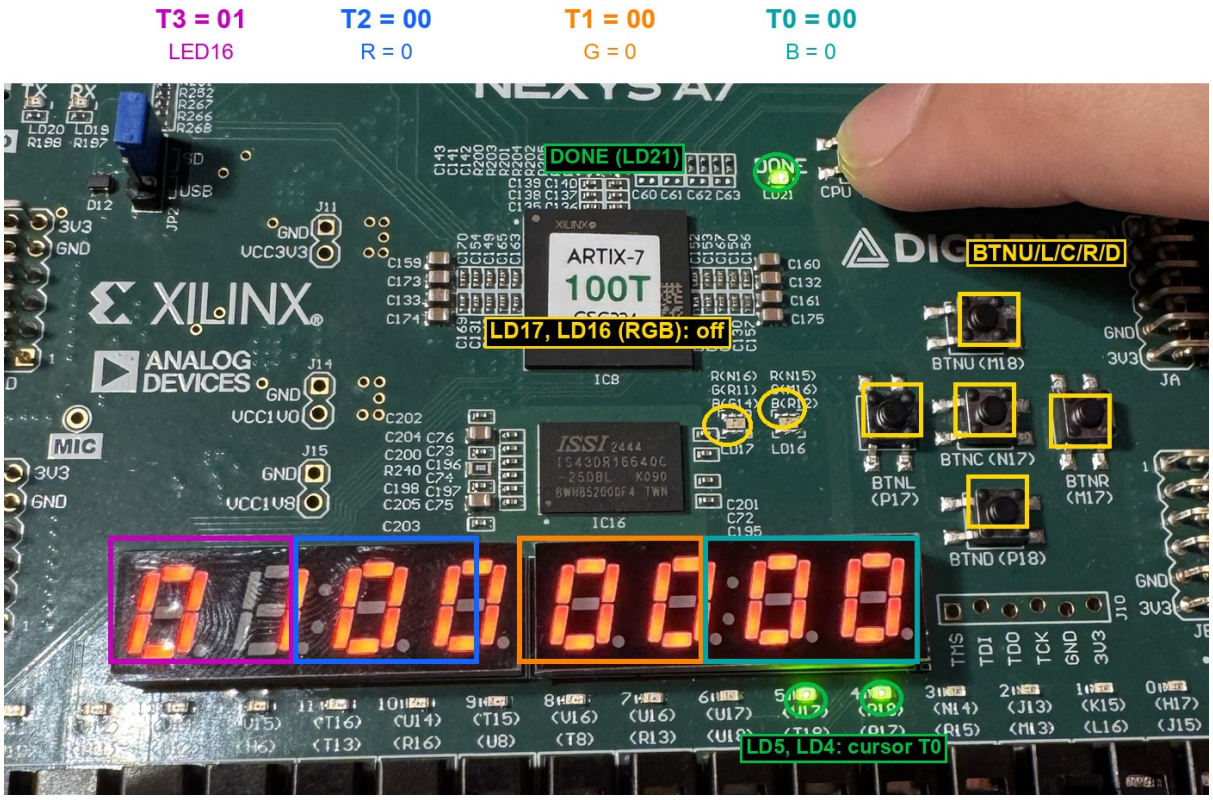
7.2. בדיקה פונקציונלית על הלוח

מקרי הבדיקה מפורטים בטבלה 17, ואיור 35 מציג את הלוח אחרי הטעינה.

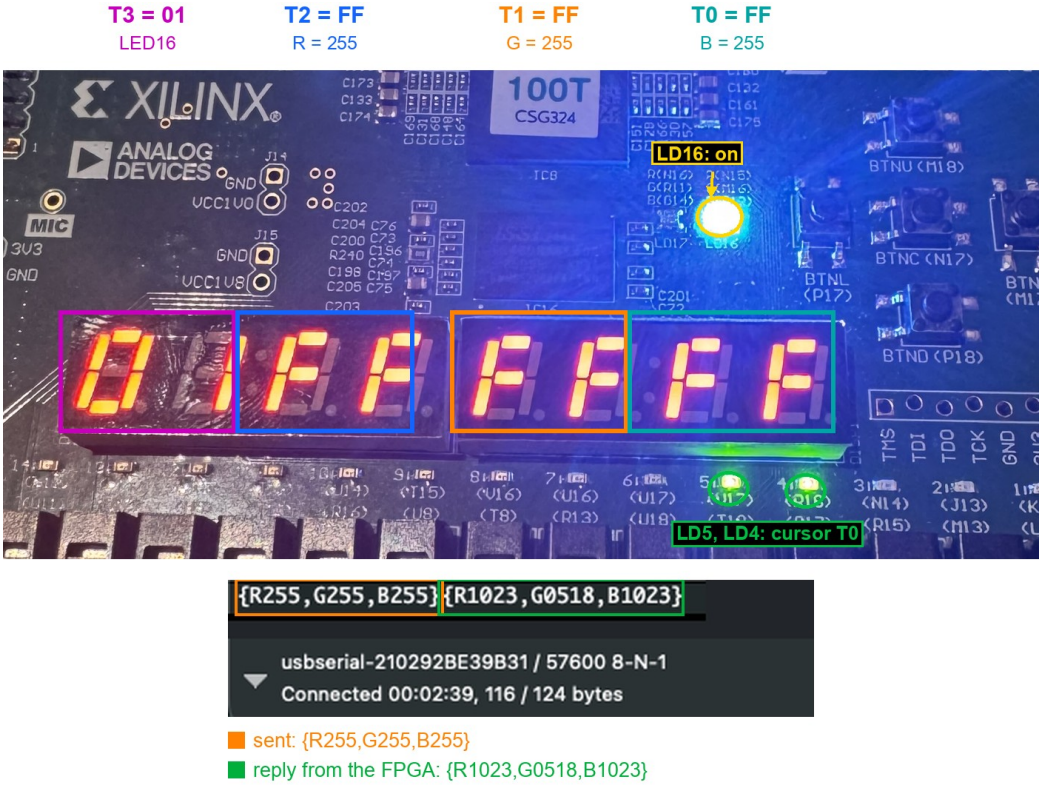
מקרים 1 עד 6, חבילות הפיקסל, צולמו עם ה-bitstream הסופי, שבו ה-CIE מנורמל לאדום. האיור של הלוח אחרי הטעינה ומקרים 7 עד 12 צולמו עם ה-bitstream הקודם, שבו ה-CIE היה מנורמל לכחול. המקרים האלה בודקים את הלחצנים, את התצוגה, את בחירת הנורה ואת פסילת חבילה שגויה, והשינוי ב-cie_scaling לא נוגע בנתיבים האלה. רק התשובות ב-TX במקרים 10 ו-12 תלויות בנרמול, והן מסומנות בטבלה. במקרה 10 הצבע הוא 0A 14 1E, כמו במקרה 6, שבו נמדדה התשובה בנרמול לאדום, {R0003,G0003,B0013}. במקרה 12 התשובה בנרמול לאדום מחושבת ממשוואה 12: {R0004,G0003,B0013}.

טבלה 17: מקרי הבדיקה על הלוח.

#	Input	Display (T3 T2 T1 T0)	RGB LED	TX reply	Result	Figure
1	{R255,G255,B255}	01 FF FF FF	LD16 on	{R1023,G0518,B1023}	Pass	36
2	{R255,G000,B000}	01 FF 00 00	LD16 red	{R1023,G0000,B0000}	Pass	37
3	{R000,G255,B000}	01 00 FF 00	LD16 green	{R0000,G0518,B0000}	Pass	38
4	{R000,G000,B255}	01 00 00 FF	LD16 blue	{R0000,G0000,B1023}	Pass	39
5	{R128,G128,B128}	01 80 80 80	LD16 on, dimmer	{R0221,G0111,B0221}	Pass	40
6	{r010,g020,b030}	01 0A 14 1E	LD16 dim blue	{R0003,G0003,B0013}	Pass	41
7	{L017} / {L016}	10 0A 14 1E / 01 0A 14 1E	LD17 / LD16	none	Pass	42
8	{R256,G000,B000}	01 0A 14 1E (kept)	unchanged	none	Pass	43
9	LEFT x4	cursor T0, T1, T2, T3, T3	unchanged	none	Pass	44
10	UP on T3, CENTER	01 0A 14 1E → 10 0A 14 1E	LD16 → LD17	{R0000,G0000,B0013} (to B)	Pass	45
11	RIGHT, UP x3, RIGHT, DOWN x3, RIGHT	10 0A 14 1E → 10 0D 11 1E	unchanged (not applied)	none	Pass	46
12	CENTER	10 0D 11 1E	LD17	{R0001,G0000,B0013} (to B)	Pass	47

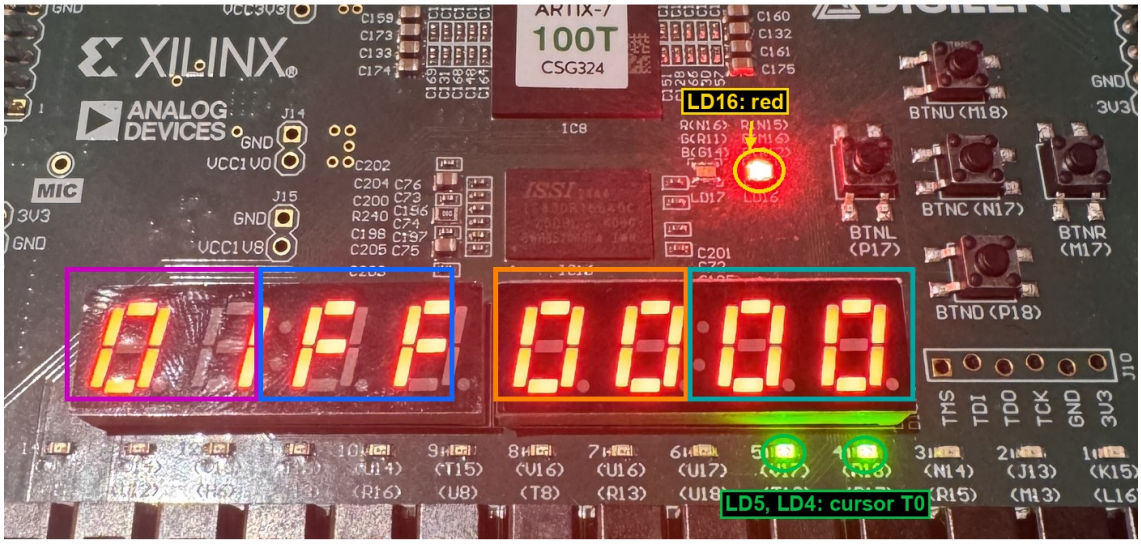


איור 35: הלוח אחרי הטעינה ולחיצה על CPU RESET: בתצוגה 00 00 00 01 (T3 = 01) בוחר את LED16, הצבע 0, 0, 0), הסמן על T0 (LD5 ו-LD4 דולקים), ושני ה-RGB LED כבויים. מסומנים חמשת הכפתורים שעורכים את הערכים.



איור 36: החבילה {R255,G255,B255}: LD16 דולק, בתצוגה 01 FF FF FF, ו-CoolTerm מציג את תשובת ה-FPGA: האדום והכחול בעוצמה מלאה (1023), והירוק מוחלש ל-518 לפי משקל ה-CIE (משוואה 12).

T3 = 01 LED16 | T2 = FF R = 255 | T1 = 00 G = 0 | T0 = 00 B = 0

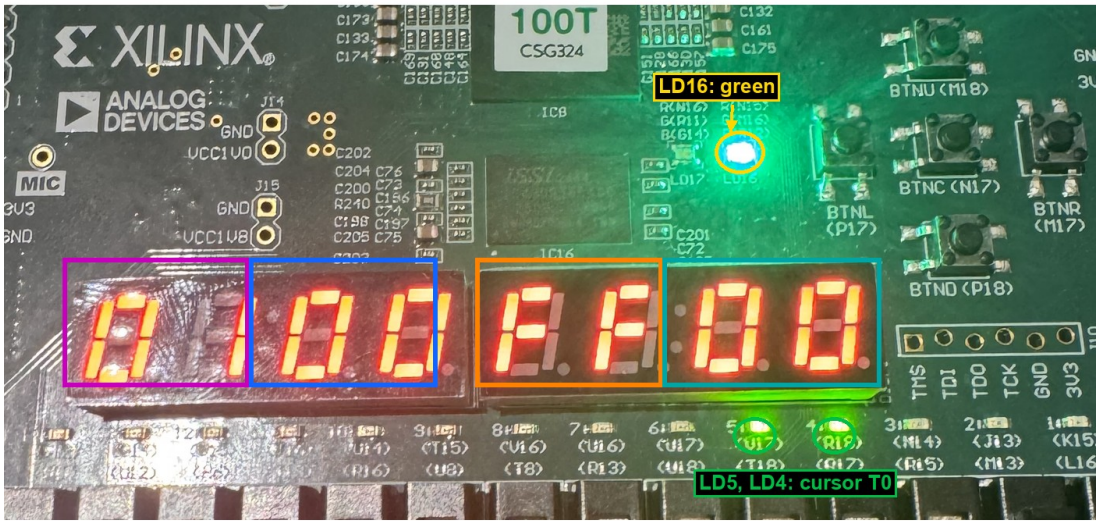



```
{R255,G255,B255}{R1023,G0518,B1023}
{R255,G000,B000}{R1023,G0000,B0000}
```

sent: {R255,G000,B000}
reply from the FPGA: {R1023,G0000,B0000}

איור 37: החבילה {R255,G000,B000}: LD16 דולק אדום בעוצמה מלאה, 1023 מתוך 1024.

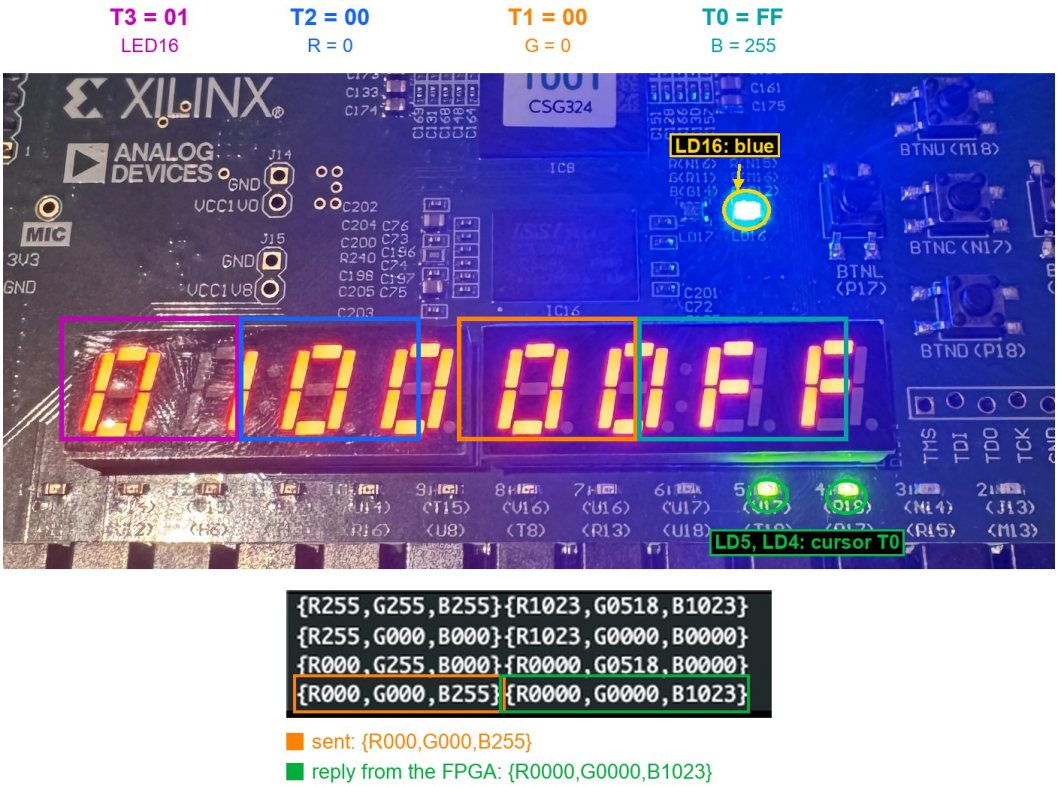
T3 = 01 LED16 | T2 = 00 R = 0 | T1 = FF G = 255 | T0 = 00 B = 0



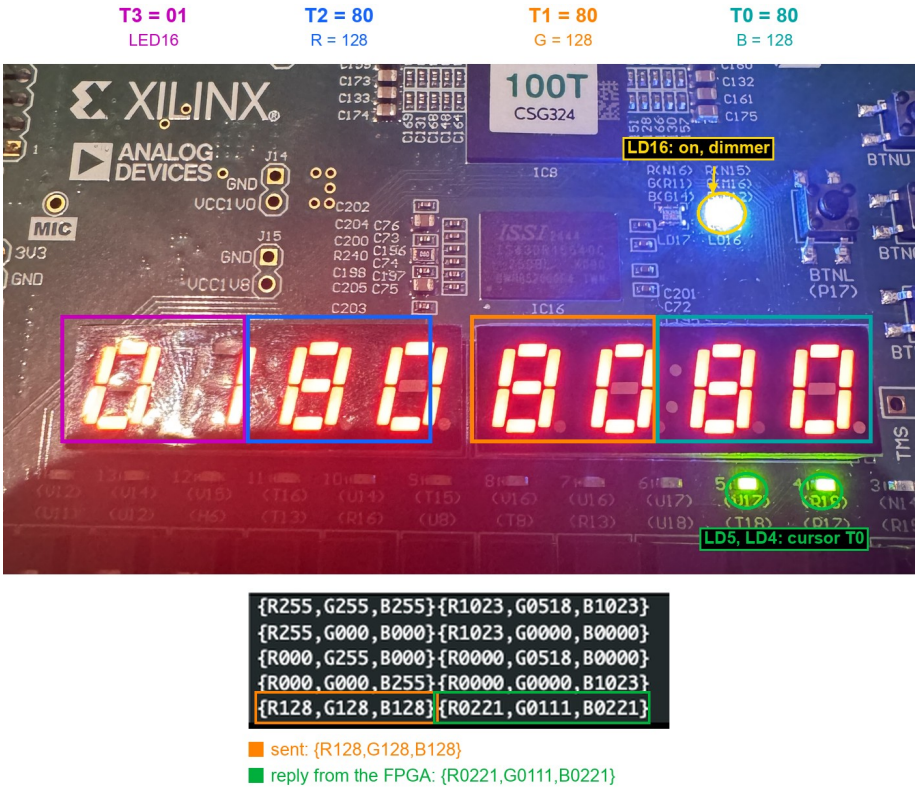

```
{R255,G255,B255}{R1023,G0518,B1023}
{R255,G000,B000}{R1023,G0000,B0000}
{R000,G255,B000}{R0000,G0518,B0000}
```

sent: {R000,G255,B000}
reply from the FPGA: {R0000,G0518,B0000}

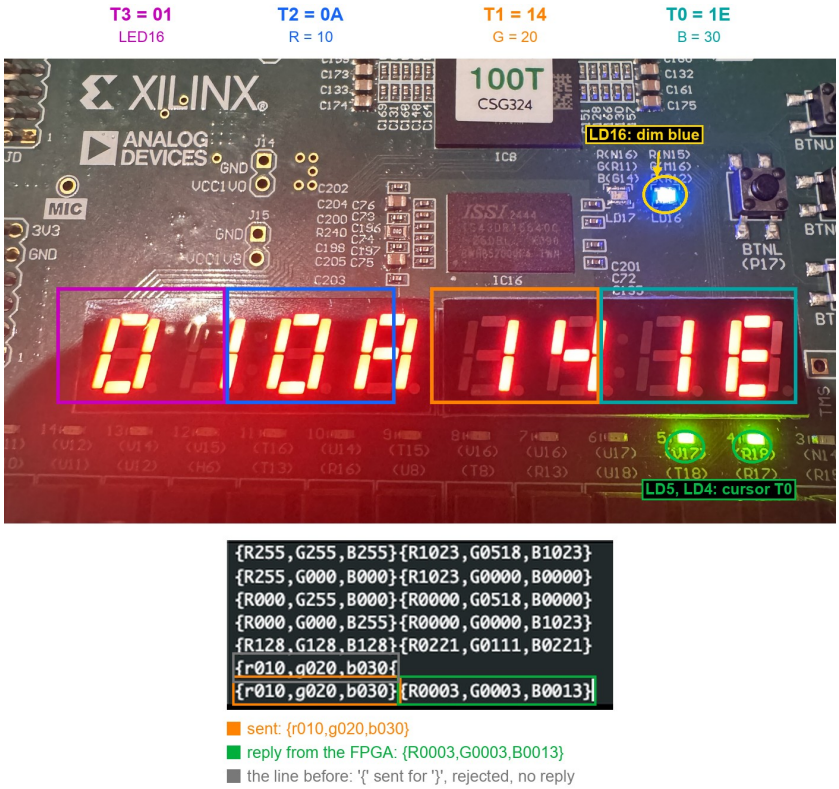
איור 38: החבילה {R000,G255,B000}: LD16 דולק ירוק, ב-518 מתוך 1024 (משקל ה-CIE של הירוק).



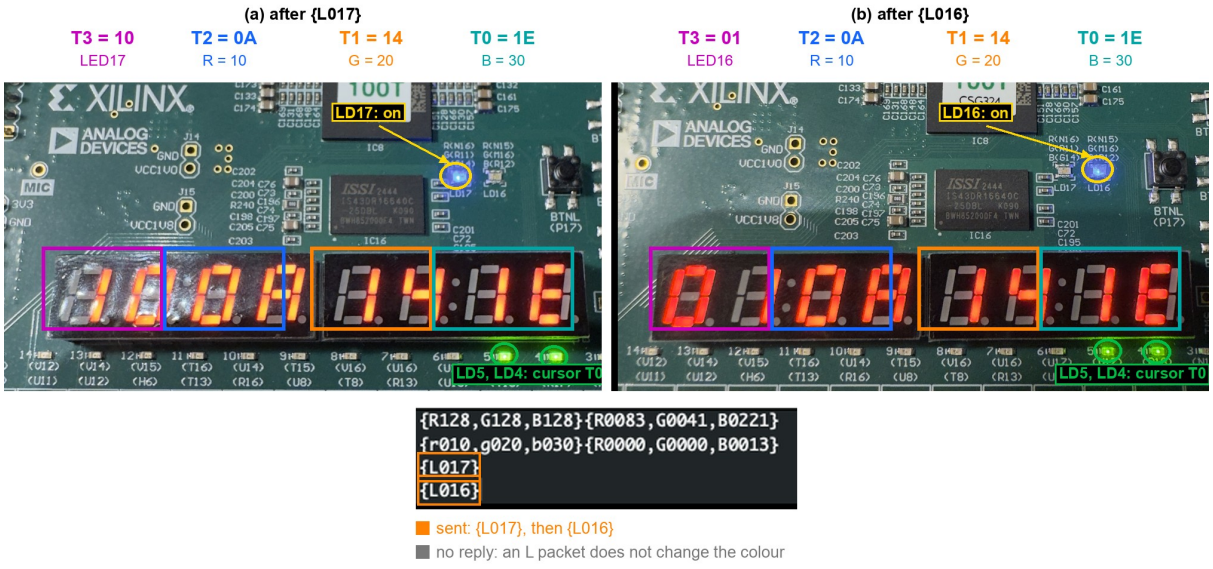
איור 39: החבילה {R000,G000,B255}: LD16 דולק כחול בעוצמה מלאה, 1023 מתוך 1024. המקדם של הכחול נחתך ל-1 (משוואה 11).



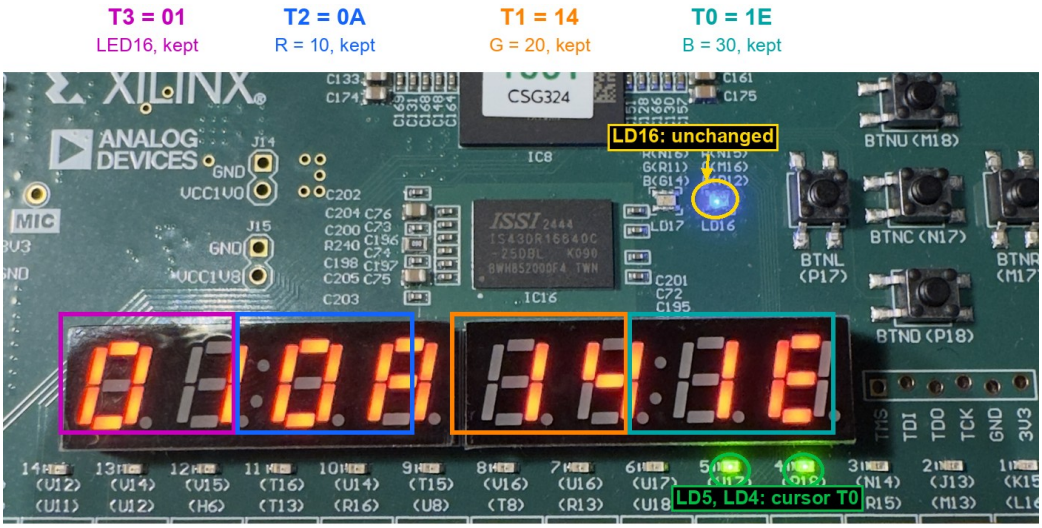
איור 40: החבילה {R128,G128,B128}: חצי מהערך ב-sRGB נותן 221 מתוך 1023 באדום ובכחול, כ-22% מהעוצמה, בגלל תיקון הגאמה, ו-111 בירוק.

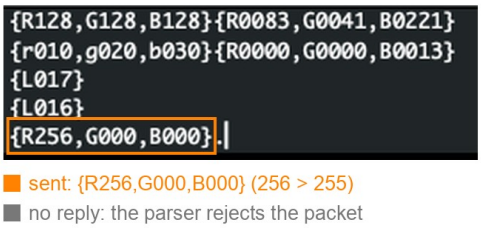


איור 41: החבילה {r010,g020,b030} באותיות קטנות: התצוגה מראה 1E 14 0A, ו-LD16 דולק כחול חלש (B = 13, R ו-G = 3 מתוך 1024). בשורה שלפניה סוגר הסיום של החבילה הוקלד כסוגר פתיחה, והחבילה נפסלה בלי תשובה.

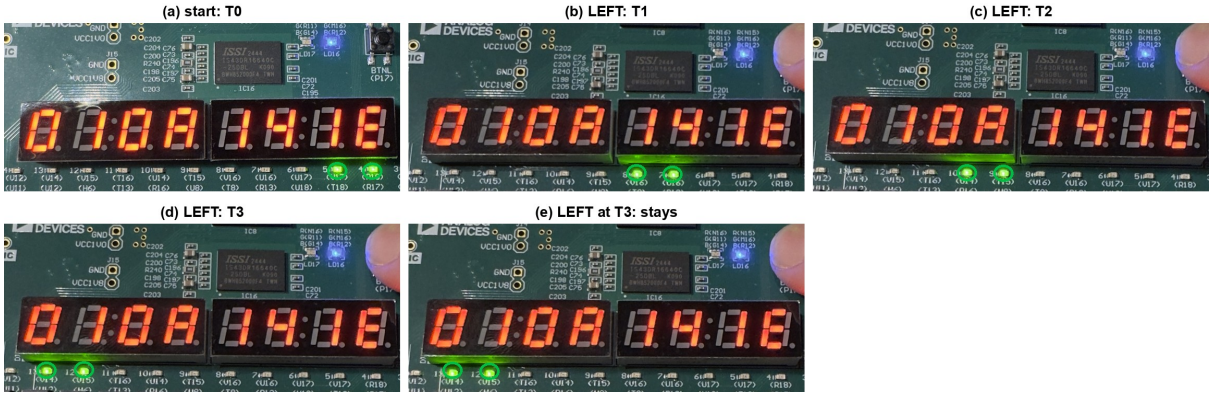


איור 42: חבילות בחירת LED: (a) אחרי {L017} הצבע עובר ל-LD17 ו-T3 = 10, ובאיור (b), אחרי {L016}, הוא חוזר ל-LD16. על חבילת L אין תשובה ב-TX.

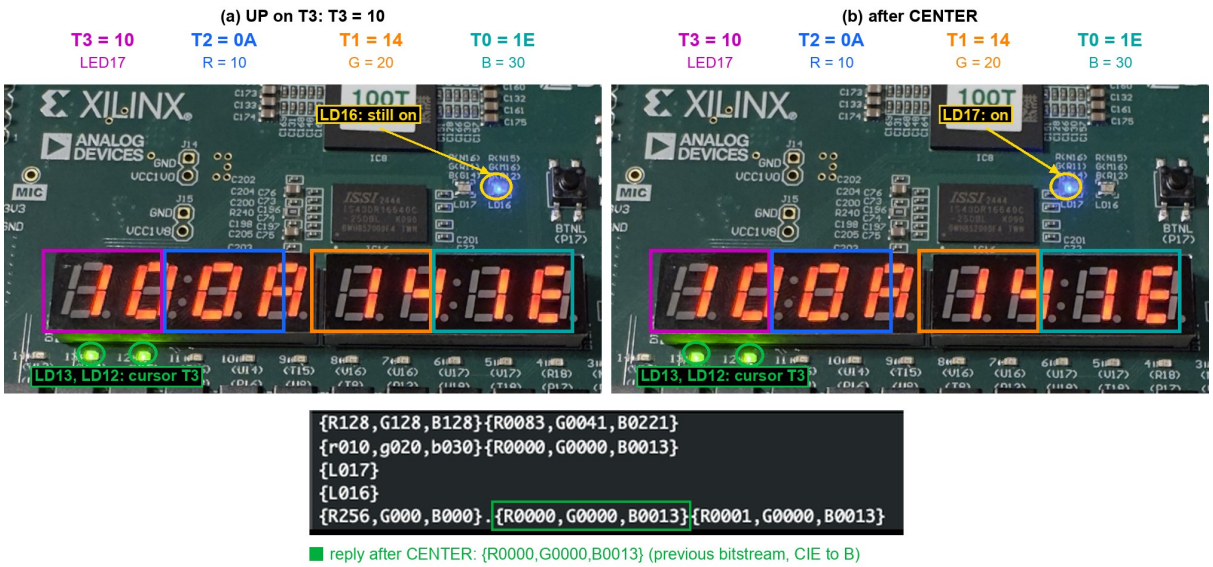




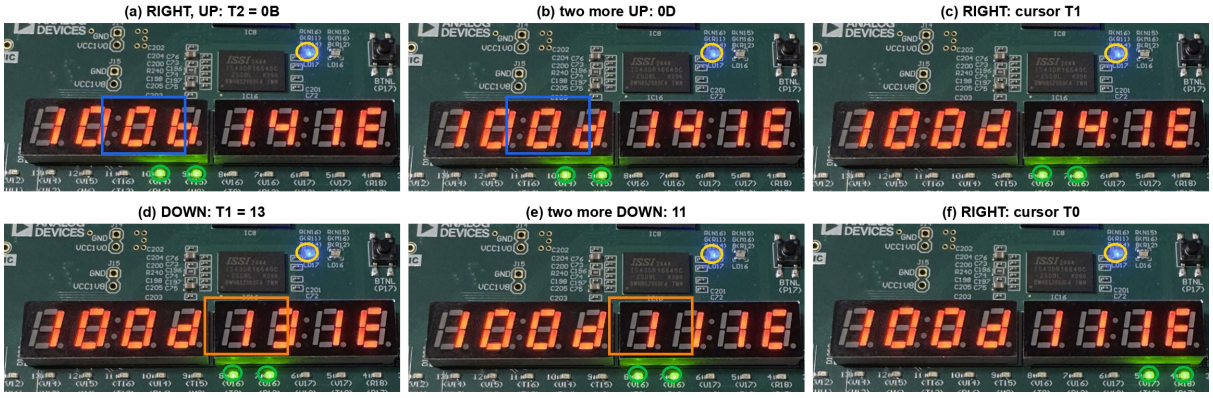
איור 43: החבילה {R256,G000,B000} (ערך גדול מ-255): התצוגה וה-LED לא משתנים, ואין תשובה ב-TX.



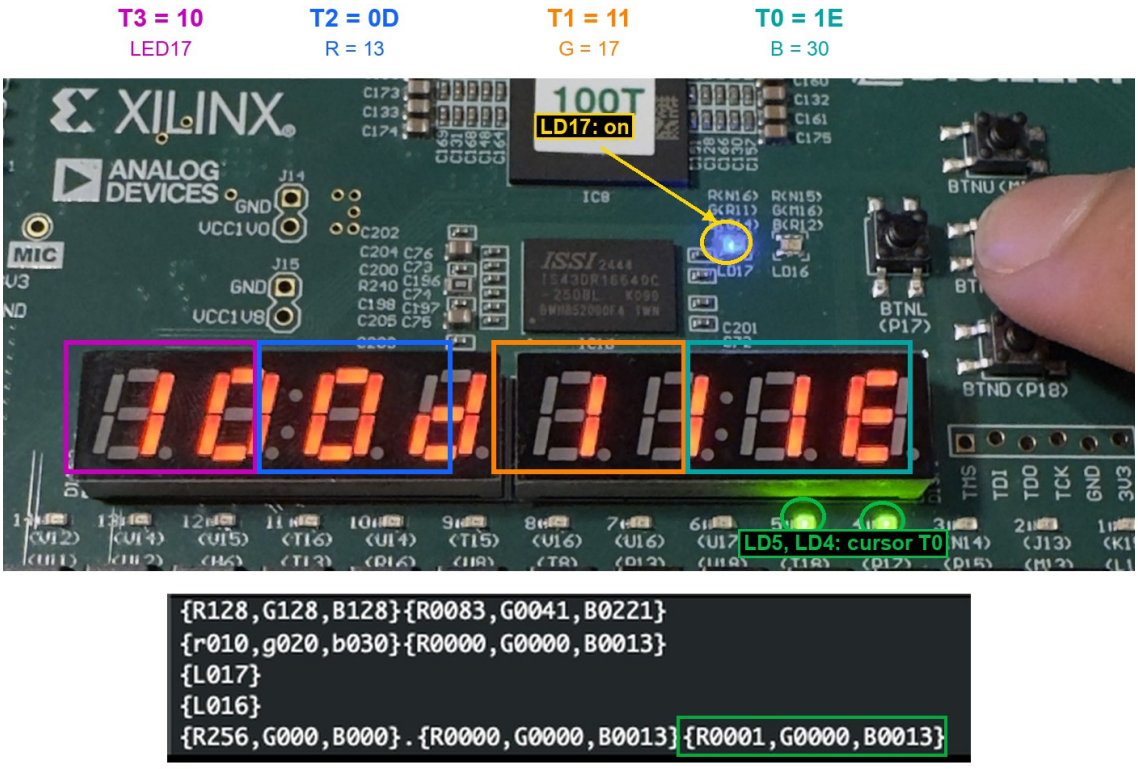
איור 44: תזוזת הסמן (נוריות ירוקות): כל LEFT מזיז את הסמן שדה אחד שמאלה, ובקצה (T3) הסמן נשאר במקומו.



איור 45: בחירת הנורה בכפתורים: (a) UP על T3 משנה את התצוגה ל-10, אבל הצבע עדיין ב-LD16, ובאיור (b), אחרי CENTER, הצבע עובר ל-LD17 וה-FPGA שולח את הערכים.



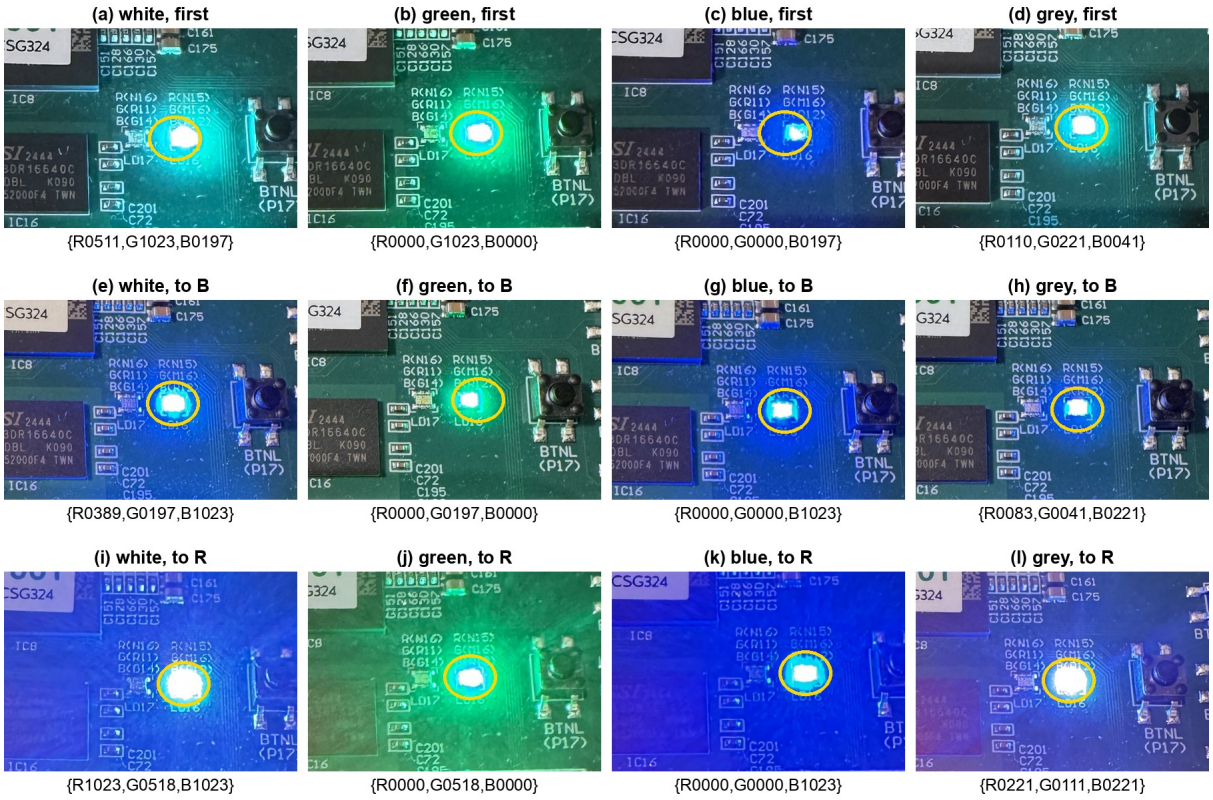
איור 46: עריכה בכפתורים: UP מעלה ו-DOWN מוריד את השדה שהסמן עליו ב-1 בכל לחיצה, אבל צבע ה-LED (העיגול הצהוב) לא משתנה עד ל-CENTER.



איור 47: החלת העריכה: אחרי CENTER ה-FPGA שולח את הערכים החדשים. ב-bitstream הקודם, עם נרמול לכחול, האדום עלה ל-1 מתוך 1024 והירוק נשאר 0, ולכן השינוי לא נראה בצילום, רק בתשובה ב-TX.

7.3. שלוש הגרסאות של ה-CIE על הלוח

שלוש הגרסאות של cie_scaling מסעיף 2.5 נבדקו על הלוח, ואיור 48 מציג את LED16 בכל אחת מהן לאותן ארבע חבילות. בעמודות של הירוק והכחול לבדם רואים את המקדמים עצמם: הירוק יורד מ-1023 בגרסה הראשונה ל-197 בנרמול לכחול ועולה ל-518 בנרמול לאדום (b, f, j), והכחול לבדו חלש רק בגרסה הראשונה, 197 מול (c, g, k) 1023. הלבן והאפור נראים ירקרקים בגרסה הראשונה (a, d) וכחולים בנרמול לכחול (e, h). בנרמול לאדום הם הקרובים ביותר ללבן מבין שלוש הגרסאות (i, l), ולא צהבהבים כפי שהנוסחה של Rec.601 צופה (סעיף 2.5). החישוב מניח שהבהירות של כל צבע ב-LED, במחזור עבודה מלא, יחסית למקדם שלו בנוסחה, וה-LED שעל הלוח לא בהכרח מקיים את ההנחה הזאת. גם איזון הלבן האוטומטי של המצלמה משנה את הגוון, ולכן ההשוואה בין הצילומים איכותית, והערכים עצמם הם התשובות ב-TX שמתחת לכל תמונה. בכל זאת, בצילומים הנרמול לאדום נותן את הלבן הבהיר והניטרלי ביותר, והתוצאה תומכת בבחירה בו.



איור 48: שלוש הגרסאות של ה-CIE על הלוח, לאותן ארבע חבילות ({R255,G255,B255}, {R000,G255,B000}, {R000,G000,B255} ו-{R128,G128,B128}): הגרסה הראשונה (a עד d), נרמול לכחול (e עד h) ונרמול לאדום, הגרסה הסופית (i עד l). מתחת לכל תמונה התשובה ב-TX.

8. החלטות תכנון, באגים ולקחים

8.1. החלטות תכנון

- סמן השדה: מונה של שני ביטים שנעצר בקצוות (display_cnt) ולא one-hot. הוא תופס פחות דלגלגים, והעצירה בקצה היא תנאי על הלחצן עצמו (!is_t0 ו-!is_t3). זוגות הנוריות של השדה הם פענוח של המונה.
- לחיצה על Right ועל Left באותו מחזור: בשרשרת ה-if של display_cnt ה-Right נבדק ראשון ולכן גובר. כשהסמן כבר בקצה הימני (T0), ה-Right חסום וה-Left מתבצע.
- בחירת הנורה היא ביט אחד (0 ל-15 ,LED16 ל-LED17), ו-T3 מציג אותו כ-{bit, !bit}, כלומר 01 או 10. ייצוג one-hot היה שומר את אותו מידע פעמיים. Up ו-Down על T3 נעצרים בקצוות, כמו בצבעים: מ-LED17 לחיצה על Up לא משנה דבר.
- אחרי reset הצבע הוא 0, 0, 0, כלומר הנורה כבויה, הנורה הנבחרת היא LED16 והסמן על T0.
- גלאי העליה של הלחצנים נמצא ב-selector, דלגלג אחד לכל לחצן, כך שהחזקה ארוכה היא צעד אחד. חזרה אוטומטית בהחזקה לא מומשה.
- SLOTS_WIDTH נגזר בכל מודול מהפרמטר SLOTS שלו, ולא מוגדר בחבילה. localparam או טיפוס בחבילה נקבעים מה-define בזמן הקומפילציה, ודריסת הפרמטר ב-chip_top לא מגיעה אליהם (לקח ממעבדה 5). לכן הפורטים שאחרי ההזזה הם וקטורים רגילים ברוחב SLOTS_WIDTH.
- ב-cie_scaling ההזזה ל-SLOTS_WIDTH באה לפני המשקלים ולא אחריהם. ב-8192 סלוטים, למשל, ערך 1 בטבלה מוזז ל-8, ואחרי משקל הירוק (0.508) נשאר 4. אילו המשקל היה מוחל קודם, אותו ערך היה מתאפס, כלומר ההזזה קודם שומרת על הצעדים הקטנים.
- CIE מנורמל לאדום: האדום והכחול בעוצמה מלאה והירוק מוחלש, כלומר בהירות לפני איזון מושלם. הנרמול לכחול, שמומש לפניו, מאוזן לגמרי אבל נותן לבן חשוך פי 2.1, והנרמול הראשון, לירוק, נתן לירוק, שהעין כבר רואה הכי חזק, יתרון נוסף (סעיפים 7.3 ,2.5 ו-8.2).
- binary_2_bcd סקוונציאלי (double dabble, הזזה אחת לכל ביט) ולא קומבינטורי. גרסה שהמירה בחלוקה (/ ו-% ב-100 ,10 ו-1000) נכשלה בתזמון ב-100MHz: מסלול מ-pwm_pixel דרך הטבלה, ה-CIE והמחלקים ועד tx_phy, עם WNS של −0.81ns אחרי המימוש. בגרסה הסקוונציאלית המחלקים נעלמו, וההמרה מסתיימת הרבה לפני שהספרות נשלחות (משוואה 16).
- start באמצע הודעה נשמר בדגל אחד (start_mid_trans), כמו תור בעומק אחד. התחלה מחדש של ההודעה הייתה שוברת את הבית שכבר יוצא ב-tx_phy, ו-FIFO היה שולח ל-PC ערכים שכבר לא מוצגים. ההודעה מתארת מצב, ולכן מספיק לשלוח את הערכים האחרונים פעם אחת.
- ב-tx_mac בית הסגירה '}' הוא קבוצה 3 אלמנט 0. כך כל קבוצה מתחילה בבית פתיחה משלה ('{' או ','), ואין שני בתים על אותו מצב של המונים.
- מקרה קצה ידוע: start שמגיע בדיוק במחזור של sent_all לא נשמר בדגל ולא מתחיל הודעה. הוחלט להשאיר את זה כך.
- מיפוי טבלת הגאמה ל-Block RAM נשאר (סעיף 5.1). הוספת keep_hierarchy על gemma_correction_i הייתה משאירה את הטבלה ב-LUTs ומעלימה את אזהרות ה-SYNTH-6, אבל התזמון עומד בדרישות עם מרווח, ואין סיבה לשנות את הקוד.

8.2. באגים שנמצאו ותוקנו

- gemma_correction: הסדר של השדות בסטראקט של הפלט היה red, blue ו-green, וההשמה של {R, G, B} הכניסה את הירוק לשדה הכחול. התיקון: סדר השדות בחבילה הוא red, green ו-blue.

- pwm: גרסה שהפכה את pwm_signal בכל השוואה (toggle) נתנה 1 קבוע ב-value=0, ושינוי ערך באמצע מחזור הפך את הקוטביות לתמיד. התיקון: השוואה cnt < value_reg.
- החבילה הגדירה localparam SLOTS_WIDTH מה-define וסטראקט ברוחב SLOTS, ודריסת הפרמטר לא הגיעה אליהם. הבאג חזר שלוש פעמים עד שההגדרות נמחקו מהחבילה (סעיף 8.1).
- selector: Up ו-Down מומשו ב-always_comb על ערך קבוע, ולכן לא היה זיכרון והערך לא נצבר. בנוסף, המוקס קרא מהערכים של ההודעה האחרונה במקום מהערכים הנערכים, וכל לחיצה חישבה את אותו ערך מחדש. התיקון: Up ו-Down כותבים לרגיסטר העריכה, והמוקס קורא ממנו.
- selector :case על וקטור של 4 ביטים מול ביט אחד, ואף ענף לא נתפס. תנאי הרוויה היה הפוך (צעד רק בגבול), ושני רגיסטרים שהחזיקו את אותו מידע על הנורה התפצלו. התיקון: מונה של שני ביטים, & is_max!, ורגיסטר אחד של ביט אחד לנורה.
- selector ו-tx_mac: tx_mac הופעל מ-pixel_valid ודגם את הערך הישן, כי pwm_pixel מתעדכן רק במחזור שאחריו. התיקון: pwm_new_data, פולס מרוגסטר שיוצא במחזור שאחרי הכתיבה.
- display_ctrl: case (1'b1) על an, שפעיל בנמוך (one-cold), בחר ספרה כבויה. התיקון: case (1'b0).
- tx_mac: קבוצה (R) 0 קראה את pixel_data[0], שהוא הכחול; ספרה נשלחה כערך של 4 ביטים (0x05) ולא כ-(0x35) ASCII; ו-tx_start יצא באותו מחזור שבו המונים התעדכנו, כך ש-tx_phy היה דוגם את הבית הקודם. התיקונים: אינדקסים מפורשים [[2, [[1 ו-[[0, {4'h3, digit}, ו-tx_start מרוגסטר במחזור שאחרי.
- cie_scaling: הגרסה הראשונה, נרמול לירוק (×½R, G×1, B×0.195), הכפילה כל צבע במשקל שלו במקום לחלק בו, והלבן נראה ירקרק בבדיקה הראשונה על הלוח (סעיף 7.3). התיקון היה נרמול לכחול, ואחרי ההשוואה בסעיף 2.5 נבחר נרמול לאדום. כל שינוי הוא שלוש שורות ב-RTL ושלושה קבועים במודל של הטסטבנץ'.
- debounce, שהועתק ממעבדה 5, נשאר עם reset אסינכרוני, היחיד בתכנון. כשהטבלה מופתה ל-Block RAM, ה-enable של ה-RAM עבר דרך רגיסטר המצבים של ה-debounce של BTNC, והתקבלו שלוש אזהרות DRC מסוג REQP-1840 (בקרה אסינכרונית של RAMB18). התיקון: reset סינכרוני, כמו בשאר המודולים, והאזהרות נעלמו.
- בשני טסטבנצ'ים שנכתבו בעזרת AI נמצאו באגי תזמון בטסטבנץ' עצמו. ב-tb_tx_mac ה-start הנוסף נשלח לפני ההמתנה ל-tx_start, פולס של מחזור אחד שכבר עבר, והטסטבנץ' נתקע עד ה-TIMEOUT של ה-watchdog. ב-tb_chip_top המקלט התחיל להאזין אחרי ששליחת ההודעה הסתיימה, אבל התשובה ב-tx מתחילה כבר בזמן ה-stop bit, ולכן הבתים נקלטו מוזזים. התיקונים: ה-start הנוסף נשלח בזמן שהבית על הקו, והשליחה והקליטה רצות במקביל (fork).

8.3. לקחים

- הטבלה הנתונה נכונה מספרית, אבל הפלט שלה ברוחב 10 ביטים מגביל את רזולוציית הצבע: בערכים הנמוכים כמה ערכי sRGB נותנים אותה רמה, ו-256 הכניסות נותנות 238 רמות בלבד. אחרי ה-CIE נשארות באדום ובכחול 238 רמות ובירוק 217, והירוק נדלק רק מ-5 sRGB. יותר סלוטים לא עוזרים: ההזזה מכפילה את אותן רמות, כך ש-1024 הוא הגבול של הטבלה, ומעליו רק רזולוציית ה-PWM עולה.
- הבדיקה על הלוח מצאה את מה שהסימולציה לא יכלה למצוא: הטסטבנץ' בדק שהמשקלים ממומשים כמו במודל, אבל רק התמונה של הלבן הראתה שכיוון הנרמול עצמו שגוי.
- כשכיוון הנרמול נכון, הבחירה ביניהם היא פשרה: כל נרמול קונה בהירות במחיר של איזון. החישוב בסעיף 2.5 הראה איפה המחיר מתחיל לעלות, והבחירה באדום נשענת עליו.

9. סיכום

במעבדה מומש ב-SystemVerilog בקר PWM לנורות ה-RGB של לוח Nexys A7-100T: פיקסל sRGB שמגיע ב-UART, או נערך בלחצנים, עובר gamma correction ו-CIE scaling, מדליק את LED16 או את LED17 בשלושה ערוצי PWM של 1024 סלוטים, ונשלח חזרה ל-PC. כל 17 הטסטבנצ'ים רצו בלי Error בלוג. הצפי מה-RTL היה 554 דלגלגים, ובפועל התקבלו 527 בסינתזה ו-535 אחרי המימוש, וכל הפרש מוסבר בפרק 6. התזמון עומד בשעון של 100MHz, עם WNS של 2.839ns אחרי המימוש. על הלוח עברו כל 12 מקרי הבדיקה. הלקח העיקרי עלה בבדיקה על הלוח: הלבן נראה ירקרק, כי הנרמול הראשון של ה-CIE הכפיל במשקלים במקום לחלק בהם. אחרי השוואה מתמטית בין הנרמולים נבחר נרמול לאדום, שנותן לבן בהיר פי 2.1 מהנרמול המאוזן לכחול (סעיף 2.5).

נספח א': קוד המקור (RTL ואילוצים)

הקוד בנספח הוא של הגרסה הסופית: cie_scaling מנורמל לאדום (סעיף 2.5). הגרסאות הקודמות, הגרסה הראשונה והנרמול לכחול, מתוארות רק בסעיפים 2.5 ו-7.3.

pkg/lab6_pkg.sv

```
package lab6_pkg;

  // Global
  // Clock frequency
  localparam CLK_FREQ = 100000000;
  localparam DATA_WIDTH = 8;
  localparam DIG_IN_WIDTH = 4;
  localparam NUM_DIGITS = 4;
  localparam BUTTON_NUM = 5;

  localparam PACKET_SIZE = 16;

  // TB Values
  localparam CLK_PERIOD = 10;

  typedef struct packed {
    logic up, down, left, right, center;
  } btn_ctf_t;

  // UART frame (tx_phy)
  localparam START_BIT_SIZE = 1;
  localparam STOP_BIT_SIZE = 1;
  localparam FRAME_WIDTH = START_BIT_SIZE + DATA_WIDTH + STOP_BIT_SIZE;

  // RX message (rx_mac -> parser)
  localparam DIGITS = 3;
  localparam MAX_GROUPS = 3;
  localparam NUM_OF_VALUES = 3;
  typedef struct packed {
    logic [DATA_WIDTH-1:0] msg_start;
    logic [DATA_WIDTH-1:0] op_2;
    logic [DIGITS-1:0][DATA_WIDTH-1:0] value_2;
    logic [DATA_WIDTH-1:0] comma0;
    logic [DATA_WIDTH-1:0] op_1;
    logic [DIGITS-1:0][DATA_WIDTH-1:0] value_1;
    logic [DATA_WIDTH-1:0] comma1;
    logic [DATA_WIDTH-1:0] op_0;
    logic [DIGITS-1:0][DATA_WIDTH-1:0] value_0;
    logic [DATA_WIDTH-1:0] msg_close;
  } packet_cfg_t;

  localparam [DATA_WIDTH-1:0] MSG_START_VAL = 8'h7B;
  localparam [DATA_WIDTH-1:0] R_VAL = 8'h52;
  localparam [DATA_WIDTH-1:0] R_SMALL_VAL = 8'h72;
  localparam [DATA_WIDTH-1:0] COMMA_VAL = 8'h2C;
  localparam [DATA_WIDTH-1:0] G_VAL = 8'h47;
  localparam [DATA_WIDTH-1:0] G_SMALL_VAL = 8'h67;
  localparam [DATA_WIDTH-1:0] B_VAL = 8'h42;
  localparam [DATA_WIDTH-1:0] B_SMALL_VAL = 8'h62;
```

```
    localparam [DATA_WIDTH-1:0] MSG_CLOSE_VAL = 8'h7D;
    localparam [DATA_WIDTH-1:0] L_VAL = 8'h4C;
    localparam [DATA_WIDTH-1:0] L_SMALL_VAL = 8'h6C;

    // RX result (parser -> gemma_correction, display_ctrl)
    localparam SCREENS_PER_SEGMENT = 2;
    typedef struct packed {
        logic [SCREENS_PER_SEGMENT-1:0][DIG_IN_WIDTH-1:0] value_2; //8
        logic [SCREENS_PER_SEGMENT-1:0][DIG_IN_WIDTH-1:0] value_1; //8
        logic [SCREENS_PER_SEGMENT-1:0][DIG_IN_WIDTH-1:0] value_0; //8
    } pixel_parsed_t;

    // gemma correction (gemma_correction -> cie_scaling)
    // GAMMA_WIDTH is the gamma table width - a constant, it does not follow SLOTS.
    // SLOTS is a chip_top parameter; from the width-extension block onward the
    // signals are plain [SLOTS_WIDTH-1:0] vectors, not a package type.
    localparam GAMMA_WIDTH = 10;
    typedef struct packed {
        logic [GAMMA_WIDTH-1:0] red;
        logic [GAMMA_WIDTH-1:0] green;
        logic [GAMMA_WIDTH-1:0] blue;
    } corrected_pixel_t;



    // Display (rx_display -> seven_segment -> display_ctrl -> pins)
    // Number of anodes for screen
    localparam AN_WIDTH = 8;
    localparam SS_DIG_WIDTH = 7;
    localparam [AN_WIDTH-1:0] AN_RESET_VAL = {{(AN_WIDTH-1){1'b1}}, 1'b0};
    localparam [SS_DIG_WIDTH-1:0] SEG_RESET_VAL = 7'd1;

    typedef struct packed {
        logic [AN_WIDTH-1:0] an;
        logic [SS_DIG_WIDTH-1:0] seg;
        logic dp;
    } display_cfg_t;

    typedef struct packed {
        logic led15, led14, led13, led12, led11, led10, led9, led8, led7, led6, led5, led4, led3, led2, led1, led0;
    } leds_cfg_t;

    // FSM states, in data-flow order
    typedef enum logic [1:0] { DEB_S0, DEB_W1, DEB_S1, DEB_W0 } deb_state_t;
    typedef enum logic { TX_IDLE, TX_TRANSMIT } tx_state_t;
    typedef enum logic [1:0] { RX_PHY_IDLE, RX_PHY_START, RX_PHY_DATA, RX_PHY_STOP } rx_phy_state_t;
    typedef enum logic [1:0] { RX_MAC_IDLE, RX_MAC_OPCODE, RX_MAC_VALUE, RX_MAC_DONE_NEXT }
rx_mac_state_t;

endpackage
```

defines/lab6_definitions.svh

```
`ifndef LAB6_DEFINITIONS_SVH
`define LAB6_DEFINITIONS_SVH

`ifdef SIM
   `define BAUD_RATE 6250000
   `define DEB_COUNT 10
   `define DISPLAY_RR 10000000
   `define SLOTS 64
`else
   `define BAUD_RATE 57600
   `define DEB_COUNT 2000000
   `define DISPLAY_RR 4000
   `define SLOTS 1024
`endif

`endif
```

src/chip_top.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module chip_top #(
   parameter BAUD_RATE = `BAUD_RATE,
   parameter DEB_COUNT = `DEB_COUNT,
   parameter DISPLAY_RR = `DISPLAY_RR,
   parameter int SLOTS = `SLOTS
) (
   input logic clk, cpu_reset,
   input logic btnu, btnd, btnl, btnr, btnc,
   input logic rx,
   output logic tx,
   output logic [AN_WIDTH-1:0] an,
   output logic [SS_DIG_WIDTH-1:0] seg,
   output logic dp,
   output logic [15:0] led,
   output logic led16_r, led16_g, led16_b,
   output logic led17_r, led17_g, led17_b
);

   btn_ctf_t io_in; assign io_in = {btnu, btnd, btnl, btnr, btnc};

   // debounce
   logic reset_n;
   btn_ctf_t io_d;
   debounce_cluster #(.DEB_COUNT(DEB_COUNT)) debounce_cluster_i(
      .clk(clk),
      .cpu_reset(cpu_reset),
      .io_in(io_in),
      .io_d(io_d),
      .cpu_reset_d(reset_n)
   );
```

```
//rx_phy
logic [DATA_WIDTH-1:0] data_rx;
logic done_rx;
rx_phy #(.BAUD_RATE(BAUD_RATE)) rx_phy_i(
    .clk(clk),
    .reset_n(reset_n),
    .en(1'b1),
    .rx(rx),
    .data(data_rx),
    .done(done_rx)
);

//rx_mac
packet_cfg_t pkt_data;
logic pkt_valid;
rx_mac rx_mac_i(
    .clk(clk),
    .reset_n(reset_n),
    .done(done_rx),
    .data_in(data_rx),
    .pkt_data(pkt_data),
    .pkt_valid(pkt_valid)
);

//parser
pixel_parsed_t pixel_parsed_data;
logic led_parsed_data;
logic pixel_valid, led_valid;
parser parser_i(
    .clk(clk),
    .reset_n(reset_n),
    .pkt_data(pkt_data),
    .pkt_valid(pkt_valid),
    .pixel_parsed_data(pixel_parsed_data),
    .led_parsed_data(led_parsed_data),
    .pixel_valid(pixel_valid),
    .led_valid(led_valid)
);

//selector
pixel_parsed_t display_pixel, pwm_pixel;
logic pwm_new_data, pwm_led, display_led;
leds_cfg_t leds_out;
selector selector_i(
    .clk(clk),
    .reset_n(reset_n),
    .btn_in(io_d),
    .parsed_pixel_value(pixel_parsed_data),
    .parsed_led_value(led_parsed_data),
    .pixel_valid(pixel_valid),
    .led_valid(led_valid),
    .display_pixel(display_pixel),
    .pwm_pixel(pwm_pixel),
    .pwm_new_data(pwm_new_data),
    .pwm_led(pwm_led),
    .display_led(display_led),
```

```
    .leds_out(leds_out)
);
assign led = leds_out;

//display_ctrl
display_cfg_t disp_cfg;
display_ctrl #(.DISPLAY_RR(DISPLAY_RR)) display_ctrl_i(
    .clk(clk),
    .reset_n(reset_n),
    .led_17_16(display_led),
    .display_pixel(display_pixel),
    .display_out(disp_cfg)
);
assign {an, seg, dp} = disp_cfg;

//gemma_correction
corrected_pixel_t corrected_pixel;
gemma_correction gemma_correction_i(
    .pixel_in(pwm_pixel),
    .pixel_out(corrected_pixel)
);

//cie_scaling
localparam int SLOTS_WIDTH = $clog2(SLOTS);
logic [2:0][SLOTS_WIDTH-1:0] cie_pixel;
cie_scaling #(.SLOTS(SLOTS)) cie_scaling_i(
    .pixel_in(corrected_pixel),
    .pixel_out(cie_pixel)
);

//pwm_cluster
logic [2:0] led16, led17;
pwm_cluster #(.SLOTS(SLOTS)) pwm_cluster_i(
    .clk(clk),
    .reset_n(reset_n),
    .value(cie_pixel),
    .pwm_led(pwm_led),
    .led16(led16),
    .led17(led17)
);
assign {led16_r, led16_g, led16_b} = led16;
assign {led17_r, led17_g, led17_b} = led17;

//tx_mac
logic done, tx_start;
logic [DATA_WIDTH-1:0] drive_data;
tx_mac #(.SLOTS(SLOTS)) tx_mac_i(
    .clk(clk),
    .reset_n(reset_n),
    .start(pwm_new_data),
    .done(done),
    .pixel_data(cie_pixel),
    .drive_data(drive_data),
    .tx_start(tx_start)
);
```

```
    //tx_phy
    tx_phy #(.BAUD_RATE(BAUD_RATE)) tx_phy_i(
        .clk(clk),
        .reset_n(reset_n),
        .start(tx_start),
        .data(drive_data),
        .tx(tx),
        .done(done)
    );

endmodule
```

src/debounce_cluster.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module debounce_cluster
#(
    parameter DEB_COUNT = `DEB_COUNT
) (
    input logic clk, cpu_reset,
    input btn_ctf_t io_in,
    output btn_ctf_t io_d,
    output logic cpu_reset_d = '0
);

    localparam DEB_COUNT_WIDTH = $clog2(DEB_COUNT);
    localparam [DEB_COUNT_WIDTH-1:0] ONE = 1;
    localparam IO_WIDTH = BUTTON_NUM;

    // Reset Debounce
    // Initial value for reset regs are 0, so that dut has power on reset
    logic reset_sync = '0;
    logic reset_meta = '0;
    always_ff @(posedge clk) begin
        {reset_sync, reset_meta} <= {reset_meta, cpu_reset};
    end

    logic reset_prev = '0;
    always_ff @(posedge clk) begin
        reset_prev <= reset_sync;
    end
    logic reset_edge;
    assign reset_edge = reset_prev ^ reset_sync;

    logic [DEB_COUNT_WIDTH-1:0] reset_cnt = '0;
    logic reset_stable;
    assign reset_stable = (reset_cnt == DEB_COUNT - 1);
    always_ff @(posedge clk) begin
        if (reset_edge) reset_cnt <= '0;
        else if (!reset_stable) reset_cnt <= reset_cnt + ONE;
    end

    always_ff @(posedge clk) begin
```

```
    if (reset_stable & !reset_edge) cpu_reset_d <= reset_sync;
  end


  //BTNC Debounce

  genvar i;
  generate
    for(i = 0; i < IO_WIDTH; i++) begin : io_deb
      debounce #(.DEB_COUNT(DEB_COUNT)) debounce_i (clk, cpu_reset_d, io_in[i], io_d[i]);
    end
  endgenerate


endmodule
```

src/debounce.sv

```
import lab6_pkg::*;

module debounce #(
  parameter DEB_COUNT = 2000000
) (
  input logic clk, reset_n, io_in,
  output logic io_d
);

  localparam COUNT_WIDTH = DEB_COUNT > 1 ? $clog2(DEB_COUNT) : 1;
  localparam [COUNT_WIDTH-1:0] ONE = 1;

  logic [COUNT_WIDTH-1:0] cnt;

  logic io_meta, io_sync;
  always_ff @(posedge clk) begin
    if (!reset_n) {io_sync, io_meta} <= '0;
    else {io_sync, io_meta} <= {io_meta, io_in};
  end


  deb_state_t state, next;
  logic cnt_max; assign cnt_max = (cnt == DEB_COUNT-1);
  always_ff @(posedge clk) begin
    if (!reset_n) cnt <= '0;
    else if (state != next) cnt <= '0;
    else if ((state == DEB_W0) || (state == DEB_W1)) cnt <= cnt + ONE;
  end

  always_ff @(posedge clk) begin
    if (!reset_n) state <= DEB_S0;
    else state <= next;
  end

  always_comb begin
    next = state;
```

```
    unique case(state)
      DEB_S0: next = (io_sync) ? DEB_W1 : DEB_S0;
      DEB_W1: next = (!io_sync) ? DEB_S0 : ((cnt_max) ? DEB_S1 : DEB_W1);
      DEB_S1: next = (!io_sync) ? DEB_W0 : DEB_S1;
      DEB_W0: next = (io_sync) ? DEB_S1 : ((cnt_max) ? DEB_S0 : DEB_W0);
      default: next = state;
    endcase
  end

  assign io_d = (state == DEB_S1) || (state == DEB_W0);

endmodule
```

src/rx_phy.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module rx_phy #(
  parameter BAUD_RATE = `BAUD_RATE
) (
  input logic clk, reset_n, en,
  input logic rx,
  output logic [DATA_WIDTH-1:0] data,
  output logic done
);

  localparam TICKS_PER_BIT = 16;
  localparam TICK_CNT = CLK_FREQ / (BAUD_RATE * TICKS_PER_BIT);


  rx_phy_state_t state, next;

  // 1. sync + fall edge
  logic rx_prev, rx_sync, rx_meta;
  always_ff @(posedge clk) begin
    if (!reset_n) {rx_prev, rx_sync, rx_meta} <= '0;
    else {rx_prev, rx_sync, rx_meta} <= {rx_sync, rx_meta, rx};
  end
  logic rx_fall; assign rx_fall = rx_prev && !rx_sync;

  // 2. Cunters
  // tick counter
  localparam TICK_CNT_BITS = (TICK_CNT > 1) ? $clog2(TICK_CNT) : 1;
  localparam [TICK_CNT_BITS-1:0] TICK_ONE = 1;
  localparam [TICK_CNT_BITS-1:0] LAST_CNT = TICK_CNT - 1;
  logic [TICK_CNT_BITS-1:0] tick_cnt;
  logic tick; assign tick = tick_cnt == LAST_CNT;

  always_ff @(posedge clk) begin
    if (!reset_n) tick_cnt <= '0;
    else if (((state == RX_PHY_IDLE) & rx_fall) | tick) tick_cnt <= '0;
    else tick_cnt <= tick_cnt + TICK_ONE;
  end
```

```
// tick16 counter
localparam TICK16_CNT_BITS = $clog2(TICKS_PER_BIT);
localparam [TICK16_CNT_BITS-1:0] TICK16_ONE = 1;
logic [TICK16_CNT_BITS-1:0] tick16_cnt;
always_ff @(posedge clk) begin
   if (!reset_n) tick16_cnt <= '0;
   else if ((state == RX_PHY_IDLE) & rx_fall) tick16_cnt <= '0;
   else if (tick) tick16_cnt <= tick16_cnt + TICK16_ONE;
end

//signals from tick16 counter
localparam SAMPLE_NUM = 3;
localparam [TICK16_CNT_BITS-1:0] SAMPLE_TICK = TICKS_PER_BIT / 2 - 1;
localparam [TICK16_CNT_BITS-1:0] START_CHECK_TICK = SAMPLE_TICK + SAMPLE_NUM;
localparam [TICK16_CNT_BITS-1:0] LAST_TICK = TICKS_PER_BIT - 1;
logic tick7; assign tick7 = (tick16_cnt == SAMPLE_TICK) & tick;
logic tick10; assign tick10 = (tick16_cnt == START_CHECK_TICK) & tick;
logic tick15; assign tick15 = (tick16_cnt == LAST_TICK) & tick;

logic [SAMPLE_NUM-1:0] start_samples;
always_ff @(posedge clk) begin
   if (!reset_n) start_samples <= '0;
   else if ((state == RX_PHY_START) & tick) start_samples <= {start_samples[SAMPLE_NUM-2:0], rx_sync};
end
logic start_check;
always_ff @(posedge clk) begin
   if (!reset_n) start_check <= '0;
   else if (state == RX_PHY_IDLE) start_check <= '0;
   else if ((!(|start_samples)) & tick10) start_check <= '1;
end

// bit counter
localparam DATA_BITS = $clog2(DATA_WIDTH);
localparam [DATA_BITS-1:0] BIT_ONE = 1;
localparam [DATA_BITS-1:0] LAST_BIT = DATA_WIDTH-1;
logic [DATA_BITS-1:0] bit_cnt;
always_ff @(posedge clk) begin
   if (!reset_n) bit_cnt <= '0;
   else if ((state == RX_PHY_START) & start_check) bit_cnt <= '0;
   else if ((state == RX_PHY_DATA) & tick15) bit_cnt <= bit_cnt + BIT_ONE;
end
logic bit7; assign bit7 = bit_cnt == LAST_BIT;

// 3. fsm
always_ff @(posedge clk) begin
   if (!reset_n) state <= RX_PHY_IDLE;
   else state <= next;
end

always_comb begin
   next = RX_PHY_IDLE;
   if (en) begin
      unique case(state)
         RX_PHY_IDLE: next = (rx_fall) ? RX_PHY_START : RX_PHY_IDLE;
         RX_PHY_START: next = (!tick15) ? RX_PHY_START : (start_check ? RX_PHY_DATA : RX_PHY_IDLE);
         RX_PHY_DATA: next = (bit7 & tick15) ? RX_PHY_STOP : RX_PHY_DATA;
```

```
            RX_PHY_STOP: next = (tick7) ? RX_PHY_IDLE : RX_PHY_STOP;
            default: next = state;
          endcase
      end
    end

    // data_reg
    always_ff @(posedge clk) begin
      if (!reset_n) data <= '0;
      else if ((state == RX_PHY_DATA) & tick7) data <= {rx_sync, data[DATA_WIDTH-1:1]};
    end

    // done reg
    always_ff @(posedge clk) begin
      if (!reset_n) done <= '0;
      else if ((state == RX_PHY_STOP) & tick7) done <= '1;
      else done <= '0;
    end



endmodule
```

src/rx_mac.sv

```
import lab6_pkg::*;

module rx_mac (
    input logic clk, reset_n,
    input logic done,
    input logic [DATA_WIDTH-1:0] data_in,
    output packet_cfg_t pkt_data,
    output logic pkt_valid
);

    rx_mac_state_t state, next;

    localparam [DATA_WIDTH-1:0] ASCII_ZERO = 8'h30;
    localparam [DATA_WIDTH-1:0] ASCII_NINE = 8'h39;
    localparam [DATA_WIDTH-1:0] ASCII_A_BIG = 8'h41;
    localparam [DATA_WIDTH-1:0] ASCII_Z_BIG = 8'h5A;
    localparam [DATA_WIDTH-1:0] ASCII_A_SMALL = 8'h61;
    localparam [DATA_WIDTH-1:0] ASCII_Z_SMALL = 8'h7A;

    logic is_big_char; assign is_big_char = (data_in >= ASCII_A_BIG) & (data_in <= ASCII_Z_BIG);
    logic is_small_char; assign is_small_char = (data_in >= ASCII_A_SMALL) & (data_in <= ASCII_Z_SMALL);

    logic is_comma; assign is_comma = (data_in == COMMA_VAL);
    logic is_start; assign is_start = (data_in == MSG_START_VAL);
    logic is_end; assign is_end = (data_in == MSG_CLOSE_VAL);
    logic is_value; assign is_value = (data_in >= ASCII_ZERO) & (data_in <= ASCII_NINE);
    logic is_opcode; assign is_opcode = is_big_char | is_small_char;
```

```
  localparam VALUE_WIDTH = $clog2(NUM_OF_VALUES);
  logic [VALUE_WIDTH-1:0] value_cnt;
  logic value_done; assign value_done = (value_cnt == NUM_OF_VALUES - 1);
  always_ff @(posedge clk) begin
    if (!reset_n) value_cnt <= '0;
    else if ((state == RX_MAC_OPCODE) & done & is_opcode) value_cnt <= '0;
    else if ((state == (RX_MAC_VALUE)) & done & is_value) value_cnt <= value_cnt + 1;
  end

  localparam GROUP_WIDTH = $clog2(MAX_GROUPS);
  logic [GROUP_WIDTH-1:0] group_cnt;
  logic max_group; assign max_group = (group_cnt == MAX_GROUPS - 1);
  always_ff @(posedge clk) begin
    if (!reset_n) group_cnt <= '0;
    else if (done & is_start) group_cnt <= '0;
    else if ((state == RX_MAC_DONE_NEXT) & done & is_comma & !max_group) group_cnt <= group_cnt + 1;
  end

  always_ff @(posedge clk) begin
    if (!reset_n) state <= RX_MAC_IDLE;
    else state <= next;
  end

  always_comb begin
    next = state;
    unique case(state)
      RX_MAC_IDLE: next = (done & is_start) ? RX_MAC_OPCODE : RX_MAC_IDLE;
      RX_MAC_OPCODE: begin
        if (done) begin
          if (is_start) next = RX_MAC_OPCODE;
          else if (is_opcode) next = RX_MAC_VALUE;
          else next = RX_MAC_IDLE;
        end
      end
      RX_MAC_VALUE: begin
        if (done) begin
          if (is_value) next = (value_done) ? RX_MAC_DONE_NEXT : RX_MAC_VALUE;
          else next = (is_start) ? RX_MAC_OPCODE : RX_MAC_IDLE;
        end
      end
      RX_MAC_DONE_NEXT: begin
        if (done) begin
          if ((!is_comma & !is_start) | (max_group & is_comma)) next = RX_MAC_IDLE;
          else if (is_start | (is_comma & !max_group)) next = RX_MAC_OPCODE;
        end
      end
      default: next = RX_MAC_IDLE;
    endcase
  end


  // MAX_GROUPS is 3, NUM_OF_VALUES = 3; 2 start and end, opcodes as max_group, values the amount of
max_group, and commans is max_group - 1
  // 2 + 3 opcode + 3*3values + 2 commas
  // => PACKET_SIZE = 2 + MAX_GROUPS + MAX_GROUPS*NUM_OF_VALUES + (MAX_GROUPS-1) = 2 + 3 + 9 +
2 = 16
```

```
    localparam CALC_PACKET_SIZE = 2 + MAX_GROUPS*(2 + NUM_OF_VALUES) - 1;
    always_ff @(posedge clk) begin
        if (!reset_n) pkt_data <= '0;
        else if (done & is_start) pkt_data <= {{(CALC_PACKET_SIZE-1)*DATA_WIDTH{1'b0}}, MSG_START_VAL};
        else if (done & (((state == RX_MAC_OPCODE) & is_opcode) | ((state == RX_MAC_VALUE) & is_value) |
((state == RX_MAC_DONE_NEXT) & ((is_comma & !max_group) | is_end)))) pkt_data <=
{pkt_data[(CALC_PACKET_SIZE-1)*DATA_WIDTH-1:0], data_in};
    end

    always_ff @(posedge clk) begin
        if (!reset_n) pkt_valid <= '0;
        else pkt_valid <= (state == RX_MAC_DONE_NEXT) & done & is_end;
    end

endmodule
```

src/parser.sv

```
import lab6_pkg::*;

module parser (
    input logic clk, reset_n,
    input packet_cfg_t pkt_data,
    input logic pkt_valid,
    output pixel_parsed_t pixel_parsed_data,
    output logic led_parsed_data,
    output logic pixel_valid, // FOR TB!
    output logic led_valid
);

    localparam MAX_VALUE = 1000; // values from 0-999 so 1000 number options
    localparam ASCII_TO_BIN_WIDTH = $clog2(MAX_VALUE);
    localparam [ASCII_TO_BIN_WIDTH-1:0] HUNDRED = 100, TEN = 10;
    localparam [ASCII_TO_BIN_WIDTH-1:0] MAX_PIXEL_VAL = 255;
    localparam [ASCII_TO_BIN_WIDTH-1:0] LED16_VAL = 16, LED17_VAL = 17;

    localparam SHORT_MSG_BYTES = 6;
    localparam PKT_WIDTH = PACKET_SIZE * DATA_WIDTH;
    localparam SHORT_MSG_PAD_LSB = SHORT_MSG_BYTES * DATA_WIDTH;


    logic valid_r; assign valid_r = (pkt_data.op_2 == R_VAL) | (pkt_data.op_2 == R_SMALL_VAL);
    logic valid_g; assign valid_g = (pkt_data.op_1 == G_VAL) | (pkt_data.op_1 == G_SMALL_VAL);
    logic valid_b; assign valid_b = (pkt_data.op_0 == B_VAL) | (pkt_data.op_0 == B_SMALL_VAL);
    logic valid_l; assign valid_l = (pkt_data.op_0 == L_VAL) | (pkt_data.op_0 == L_SMALL_VAL);


    // No {, } check because it is already checked in rx_mac, and if small message then c and v and r wont be
in place so msg not valid

    logic valid_color_op; assign valid_color_op = valid_r & valid_g & valid_b;
    logic valid_led_op; assign valid_led_op = valid_l & (pkt_data[PKT_WIDTH-1:SHORT_MSG_PAD_LSB] == '0);

    // ASCII nums = 0x30 - 0x39 so can take only bottom nibble
```

```
    logic [ASCII_TO_BIN_WIDTH-1:0] value_2, value_1, value_0;
    assign value_2 = pkt_data.value_2[2][3:0] * HUNDRED + pkt_data.value_2[1][3:0] * TEN +
pkt_data.value_2[0][3:0];
    assign value_1 = pkt_data.value_1[2][3:0] * HUNDRED + pkt_data.value_1[1][3:0] * TEN +
pkt_data.value_1[0][3:0];
    assign value_0 = pkt_data.value_0[2][3:0] * HUNDRED + pkt_data.value_0[1][3:0] * TEN +
pkt_data.value_0[0][3:0];

    logic bad_pixel_value; assign bad_pixel_value = ((value_2 > MAX_PIXEL_VAL) | (value_1 > MAX_PIXEL_VAL) |
(value_0 > MAX_PIXEL_VAL)) & valid_color_op;
    logic bad_led_value; assign bad_led_value = ((value_0 != LED16_VAL) & (value_0 != LED17_VAL)) &
valid_led_op;

    logic pixel_msg_valid_comb; assign pixel_msg_valid_comb = (valid_color_op & !bad_pixel_value);
    logic led_msg_valid_comb; assign led_msg_valid_comb = (valid_led_op & !bad_led_value);

    always_ff @(posedge clk) begin
        if (!reset_n) pixel_valid <= '0;
        else pixel_valid <= pkt_valid & pixel_msg_valid_comb;
    end

    always_ff @(posedge clk) begin
        if (!reset_n) led_valid <= '0;
        else led_valid <= pkt_valid & led_msg_valid_comb;
    end

    always_ff @(posedge clk) begin
        if (!reset_n) pixel_parsed_data <= '0;
        else if (pkt_valid & pixel_msg_valid_comb) pixel_parsed_data <= {value_2[DATA_WIDTH-1:0],
value_1[DATA_WIDTH-1:0], value_0[DATA_WIDTH-1:0]};
    end

    always_ff @(posedge clk) begin
        if (!reset_n) led_parsed_data <= '0;
        else led_parsed_data <= (pkt_valid & led_msg_valid_comb & (value_0 == LED17_VAL));
    end


endmodule
```

src/selector.sv

```
import lab6_pkg::*;

module selector (
    input logic clk, reset_n,
    input btn_ctf_t btn_in,
    input pixel_parsed_t parsed_pixel_value,
    input logic parsed_led_value,
    input logic pixel_valid, led_valid,
    output pixel_parsed_t display_pixel,
    output pixel_parsed_t pwm_pixel,
    output logic pwm_new_data,
    output logic pwm_led,
    output logic display_led,
```

```
    output leds_cfg_t leds_out
);
    // 1. need to convert btn_in levels to edge pulses
    btn_ctf_t btn_edge, btn_prev;
    genvar i;
    generate
        for (i = 0; i < 5; i++) begin : BTN_EDGE_DETECTORS
            always_ff @(posedge clk) begin
                if (!reset_n) btn_prev[i] <= '0;
                else btn_prev[i] <= btn_in[i];
            end
            assign btn_edge[i] = !btn_prev[i] & btn_in[i];
        end
    endgenerate

    localparam DISPLAY_SECTIONS = 4;
    typedef enum logic [DISPLAY_SECTIONS-1:0] {
        FIRST_T = 4'b0001,
        LAST_T = 4'b1000
    } t_e;

    localparam MAX_PIXEL_VAL = 255;
    localparam MIN_PIXEL_VAL = 0;

    logic [1:0] display_cnt;
    logic is_t0; assign is_t0 = (display_cnt == 2'b00);
    logic is_t3; assign is_t3 = (display_cnt == 2'b11);

    always_ff @(posedge clk) begin
        if (!reset_n) display_cnt <= '0;
        else if (btn_edge.right & !is_t0) display_cnt <= display_cnt - 1;
        else if (btn_edge.left & !is_t3) display_cnt <= display_cnt + 1;
    end

    // Pixel value
    logic [2:0] is_max, is_min;
    generate
        for (i = 0; i < 3; i++) begin : MIN_MAX
            assign is_max[i] = (display_pixel[DATA_WIDTH*i +: DATA_WIDTH] == MAX_PIXEL_VAL);
            assign is_min[i] = (display_pixel[DATA_WIDTH*i +: DATA_WIDTH] == MIN_PIXEL_VAL);
        end
    endgenerate

    generate
        for (i = 0; i < 3; i++) begin : UP_DOWN_VAL
            always_ff @(posedge clk) begin
                if (!reset_n) display_pixel[DATA_WIDTH*i +: DATA_WIDTH] <= '0;
                else if (pixel_valid) display_pixel[DATA_WIDTH*i +: DATA_WIDTH] <=
parsed_pixel_value[DATA_WIDTH*i +: DATA_WIDTH];
                else if (!is_max[i] & btn_edge.up & (display_cnt == i)) display_pixel[DATA_WIDTH*i +: DATA_WIDTH]
<= display_pixel[DATA_WIDTH*i +: DATA_WIDTH] + 1;
                else if (!is_min[i] & btn_edge.down & (display_cnt == i)) display_pixel[DATA_WIDTH*i +:
DATA_WIDTH] <= display_pixel[DATA_WIDTH*i +: DATA_WIDTH] - 1;
            end
        end
    endgenerate
```

```
    always_ff @(posedge clk) begin
        if (!reset_n) {pwm_pixel, pwm_new_data} <= '0;
        else if (pixel_valid) {pwm_pixel, pwm_new_data} <= {parsed_pixel_value, 1'b1};
        else if (btn_edge.center) {pwm_pixel, pwm_new_data} <= {display_pixel, 1'b1};
        else pwm_new_data <= '0;
    end

    // led value
    always_ff @(posedge clk) begin
        if (!reset_n) display_led <= '0;
        else if (led_valid) display_led <= parsed_led_value;
        else if (((btn_edge.down & display_led) | (btn_edge.up & !display_led)) & (display_cnt == 2'd3))
display_led <= !display_led;
    end

    always_ff @(posedge clk) begin
        if (!reset_n) pwm_led <= '0;
        else if (led_valid) pwm_led <= parsed_led_value;
        else if (btn_edge.center) pwm_led <= display_led;
    end

    always_comb begin
        unique case(display_cnt)
            2'd0: leds_out = 16'h0030;
            2'd1: leds_out = 16'h0180;
            2'd2: leds_out = 16'h0600;
            2'd3: leds_out = 16'h3000;
            default: leds_out = '0;
        endcase
    end

endmodule
```

src/display_ctrl.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"
module display_ctrl #(
    parameter DISPLAY_RR = `DISPLAY_RR
) (
    input logic clk, reset_n,
    input logic led_17_16,
    input pixel_parsed_t display_pixel,
    output display_cfg_t display_out
);

    // 1. tick gen for 500hz
    logic tick;
    baud_gen #(.BAUD_RATE(DISPLAY_RR)) display_tick_i (
        .clk(clk),
        .reset_n(reset_n),
        .en(1'b1),
        .baud_tick(tick)
```

```
    );

    // 2. rotate an 0 and 500hz rate
    logic [AN_WIDTH-1:0] an;
    always_ff @(posedge clk) begin
        if (!reset_n) an <= AN_RESET_VAL;
        else if (tick) an <= {an[AN_WIDTH-2:0], an[AN_WIDTH-1]};
    end

    logic[DIG_IN_WIDTH-1:0] display_data;
    always_comb begin
        unique case (1'b0)
            an[0]: display_data = display_pixel.value_0[0];
            an[1]: display_data = display_pixel.value_0[1];
            an[2]: display_data = display_pixel.value_1[0];
            an[3]: display_data = display_pixel.value_1[1];
            an[4]: display_data = display_pixel.value_2[0];
            an[5]: display_data = display_pixel.value_2[1];
            an[6]: display_data = led_17_16 ? '0 : DIG_IN_WIDTH'(1);
            an[7]: display_data = led_17_16 ? DIG_IN_WIDTH'(1) : '0;
            default: display_data = '0;
        endcase
    end

    seven_segment ss (
        .data_val(display_data),
        .ss_dig(display_out.seg)
    );

    assign display_out.an = an;
    assign display_out.dp = 1'b1;

endmodule
```

src/seven_segment.sv

```
import lab6_pkg::*;

module seven_segment (
    input logic [DIG_IN_WIDTH-1:0] data_val,
    output logic [SS_DIG_WIDTH-1:0] ss_dig
);

    always_comb begin
        unique case (data_val)
            4'h0: ss_dig = 7'b0000001;
            4'h1: ss_dig = 7'b1001111;
            4'h2: ss_dig = 7'b0010010;
            4'h3: ss_dig = 7'b0000110;
            4'h4: ss_dig = 7'b1001100;
            4'h5: ss_dig = 7'b0100100;
            4'h6: ss_dig = 7'b0100000;
            4'h7: ss_dig = 7'b0001111;
            4'h8: ss_dig = 7'b0000000;
            4'h9: ss_dig = 7'b0000100;
```

```
        4'hA: ss_dig = 7'b0001000;
        4'hB: ss_dig = 7'b1100000;
        4'hC: ss_dig = 7'b0110001;
        4'hD: ss_dig = 7'b1000010;
        4'hE: ss_dig = 7'b0110000;
        4'hF: ss_dig = 7'b0111000;
        default: ss_dig = 7'b1111111;
      endcase
  end
endmodule
```

src/gemma_correction.sv

```
import lab6_pkg::*;

module gemma_correction(
  input pixel_parsed_t pixel_in,
  output corrected_pixel_t pixel_out
);
  `include "black_magic_box.sv"
  assign pixel_out.red = gamma_table[pixel_in.value_2];
  assign pixel_out.green = gamma_table[pixel_in.value_1];
  assign pixel_out.blue = gamma_table[pixel_in.value_0];
endmodule
```

src/black_magic_box.sv

```
// Include with:  `include "black_magic_box.sv"
const logic [9:0] gamma_table [0:255] = '{
10'd0, 10'd0, 10'd1, 10'd1, 10'd1, 10'd2, 10'd2, 10'd2, 10'd2, 10'd3, 10'd3,
10'd3, 10'd4, 10'd4, 10'd4, 10'd5, 10'd5, 10'd6, 10'd6, 10'd7, 10'd7, 10'd8,
10'd8, 10'd9, 10'd9, 10'd10, 10'd11, 10'd11, 10'd12, 10'd13, 10'd13, 10'd14,
10'd15, 10'd16, 10'd16, 10'd17, 10'd18, 10'd19, 10'd20, 10'd21, 10'd22, 10'd23,
10'd24, 10'd25, 10'd26, 10'd27, 10'd28, 10'd29, 10'd30, 10'd31, 10'd33, 10'd34,
10'd35, 10'd36, 10'd38, 10'd39, 10'd40, 10'd42, 10'd43, 10'd45, 10'd46, 10'd48,
10'd49, 10'd51, 10'd52, 10'd54, 10'd56, 10'd57, 10'd59, 10'd61, 10'd63, 10'd64,
10'd66, 10'd68, 10'd70, 10'd72, 10'd74, 10'd76, 10'd78, 10'd80, 10'd82, 10'd84,
10'd86, 10'd88, 10'd91, 10'd93, 10'd95, 10'd97, 10'd100, 10'd102, 10'd105,
10'd107, 10'd109, 10'd112, 10'd115, 10'd117, 10'd120, 10'd122, 10'd125, 10'd128,
10'd130, 10'd133, 10'd136, 10'd139, 10'd142, 10'd145, 10'd147, 10'd150, 10'd153,
10'd156, 10'd160, 10'd163, 10'd166, 10'd169, 10'd172, 10'd175, 10'd179, 10'd182,
10'd185, 10'd189, 10'd192, 10'd196, 10'd199, 10'd203, 10'd206, 10'd210, 10'd213,
10'd217, 10'd221, 10'd225, 10'd228, 10'd232, 10'd236, 10'd240, 10'd244, 10'd248,
10'd252, 10'd256, 10'd260, 10'd264, 10'd268, 10'd272, 10'd277, 10'd281, 10'd285,
10'd290, 10'd294, 10'd298, 10'd303, 10'd307, 10'd312, 10'd317, 10'd321, 10'd326,
10'd331, 10'd335, 10'd340, 10'd345, 10'd350, 10'd355, 10'd360, 10'd365, 10'd370,
10'd375, 10'd380, 10'd385, 10'd390, 10'd395, 10'd401, 10'd406, 10'd411, 10'd417,
10'd422, 10'd427, 10'd433, 10'd439, 10'd444, 10'd450, 10'd455, 10'd461, 10'd467,
10'd473, 10'd479, 10'd484, 10'd490, 10'd496, 10'd502, 10'd508, 10'd514, 10'd521,
10'd527, 10'd533, 10'd539, 10'd546, 10'd552, 10'd558, 10'd565, 10'd571, 10'd578,
10'd584, 10'd591, 10'd598, 10'd604, 10'd611, 10'd618, 10'd625, 10'd631, 10'd638,
```

```
10'd645, 10'd652, 10'd659, 10'd666, 10'd674, 10'd681, 10'd688, 10'd695, 10'd702,
10'd710, 10'd717, 10'd725, 10'd732, 10'd740, 10'd747, 10'd755, 10'd763, 10'd770,
10'd778, 10'd786, 10'd794, 10'd802, 10'd809, 10'd817, 10'd826, 10'd834, 10'd842,
10'd850, 10'd858, 10'd866, 10'd875, 10'd883, 10'd891, 10'd900, 10'd908, 10'd917,
10'd925, 10'd934, 10'd943, 10'd952, 10'd960, 10'd969, 10'd978, 10'd987, 10'd996,
10'd1005, 10'd1014, 10'd1023
};
```

src/cie_scaling.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"
module cie_scaling #(
    parameter SLOTS = `SLOTS,
    localparam int SLOTS_WIDTH = $clog2(SLOTS)
)(
    input corrected_pixel_t pixel_in,
    output logic [2:0][SLOTS_WIDTH-1:0] pixel_out
);

    /*
        CIE / luminance scaling, normalized to RED (brightness over perfect balance).
        REC.601: Y = 0.299R + 0.587G + 0.114B  - the eye sees green ~2x red and ~5x blue.
        scale_c = 0.299 / weight_c

        1. Red:   1
        2. Green: 0.299/0.587 = 0.509 ~ 0.5078 = 1/2 + 1/128   (-0.3%)
        3. Blue:  0.299/0.114 = 2.62  -> cannot exceed 100% duty, clamped to 1
                (blue stays ~2.6x weaker in perception than red/green; documented in the report)

        Example, SLOTS = 1024, input 1023:
        Red(255) = 1023, Green(255) = 518, Blue(255) = 1023
        Shifts and adds only, no multipliers.
    */
    typedef struct packed {
        logic [SLOTS_WIDTH-1:0] red;
        logic [SLOTS_WIDTH-1:0] green;
        logic [SLOTS_WIDTH-1:0] blue;

    } cie_pixel_t;
    cie_pixel_t sh_pixel_in;
    generate
        if (SLOTS_WIDTH > GAMMA_WIDTH) begin : SH_UP
            localparam SHIFT = (SLOTS_WIDTH - GAMMA_WIDTH);
            assign sh_pixel_in.red = (pixel_in.red << SHIFT);
            assign sh_pixel_in.green = (pixel_in.green << SHIFT);
            assign sh_pixel_in.blue = (pixel_in.blue << SHIFT);
        end else begin : SH_DOWN
            localparam SHIFT = (GAMMA_WIDTH - SLOTS_WIDTH);
            assign sh_pixel_in.red = (pixel_in.red >> SHIFT);
            assign sh_pixel_in.green = (pixel_in.green >> SHIFT);
            assign sh_pixel_in.blue = (pixel_in.blue >> SHIFT);
        end
```

```
    endgenerate

    assign pixel_out[2] = sh_pixel_in.red;
    assign pixel_out[1] = (sh_pixel_in.green >> 1) + (sh_pixel_in.green >> 7);
    assign pixel_out[0] = sh_pixel_in.blue;

endmodule
```

src/pwm_cluster.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module pwm_cluster #(
    parameter int SLOTS = `SLOTS,
    localparam int SLOTS_WIDTH = $clog2(SLOTS)
) (
    input logic clk, reset_n,
    input logic [2:0][SLOTS_WIDTH-1:0] value,   // [2] = R, [1] = G, [0] = B
    input logic pwm_led,                        // 0 -> LED16, 1 -> LED17
    output logic [2:0] led16,                   // {r, g, b}
    output logic [2:0] led17
);

    // one pwm per channel
    logic [2:0] pwm_signal;
    genvar i;
    generate
        for (i = 0; i < 3; i++) begin : pwm_gen
            pwm #(.SLOTS(SLOTS)) pwm_i(
                .clk(clk),
                .reset_n(reset_n),
                .value(value[i]),
                .pwm_signal(pwm_signal[i])
            );
        end
    endgenerate

    always_comb begin
        if (pwm_led) begin
            led16 = '0;
            led17 = pwm_signal;
        end else begin
            led16 = pwm_signal;
            led17 = '0;
        end
    end

endmodule
```

src/pwm.sv

```
import lab6_pkg::*;

// SLOTS is passed down by name from chip_top (1024 default, 512 .. 8192);
module pwm #(
    parameter int SLOTS = 1024,
    localparam int SLOTS_WIDTH = $clog2(SLOTS)
) (
    input logic clk, reset_n,
    input logic [SLOTS_WIDTH-1:0] value,
    output logic pwm_signal
);

    logic [SLOTS_WIDTH-1:0] cnt;
    logic cnt_wrap; assign cnt_wrap = (cnt == SLOTS - 1);
    always_ff @(posedge clk) begin
        if (!reset_n) cnt <= '0;
        else if (cnt_wrap) cnt <= '0;
        else cnt <= cnt + 1;
    end

    logic [SLOTS_WIDTH-1:0] value_reg;
    always_ff @(posedge clk) begin
        if (!reset_n) value_reg <= '0;
        else if (cnt_wrap) value_reg <= value;
    end

    always_ff @(posedge clk) begin
        if (!reset_n) pwm_signal <= '0;
        else pwm_signal <= (cnt < value_reg);
    end

endmodule
```

src/tx_mac.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tx_mac #(
    parameter SLOTS = `SLOTS,
    localparam SLOTS_WIDTH = $clog2(SLOTS)
) (
    input logic clk, reset_n,
    input logic start, done,
    input logic [2:0][SLOTS_WIDTH-1:0] pixel_data, // [2] = R, [1] = G, [0] = B, same order as cie_scaling
    output logic [DATA_WIDTH-1:0] drive_data,
    output logic tx_start
);
    localparam GROUP_NUM = 4;
    localparam GROUP_WIDTH = $clog2(GROUP_NUM);

    localparam ELEMENT_NUM = 6;
    localparam ELEMENT_WIDTH = $clog2(ELEMENT_NUM);
```

```
    tx_state_t state, next;

    logic [GROUP_WIDTH-1:0] group_cnt;
    logic [ELEMENT_WIDTH-1:0] element_cnt;
    logic last_group; assign last_group = (group_cnt == GROUP_NUM-1);
    logic last_element; assign last_element = (element_cnt == ELEMENT_NUM-1);
    logic msg_close; assign msg_close = (element_cnt == '0) & last_group;
    logic sent_all; assign sent_all = (state == TX_TRANSMIT) & msg_close & done;

    logic start_mid_trans;

    // element counter
    always_ff @(posedge clk) begin
        if (!reset_n) element_cnt <= '0;
        else if ((last_element & done & (state == TX_TRANSMIT)) | ((state == TX_IDLE) & start) | (sent_all &
start_mid_trans)) element_cnt <= '0;
        else if ((state == TX_TRANSMIT) & done & !sent_all) element_cnt <= element_cnt + 1;
    end

    // group counter
    always_ff @(posedge clk) begin
        if (!reset_n) group_cnt <= '0;
        else if (((state == TX_IDLE) & start) | sent_all) group_cnt <= '0;
        else if ((state == TX_TRANSMIT) & done & last_element) group_cnt <= group_cnt + 1;
    end

    // pending flag
    always_ff @(posedge clk) begin
        if (!reset_n) start_mid_trans <= '0;
        else if (((state == TX_IDLE) & start) | ((state == TX_TRANSMIT) & sent_all)) start_mid_trans <= '0;
        else if ((state == TX_TRANSMIT) & start) start_mid_trans <= '1;
    end

    // state register
    always_ff @(posedge clk) begin
        if (!reset_n) state <= TX_IDLE;
        else state <= next;
    end

    // next state
    always_comb begin
        next = state;
        unique case(state)
            TX_IDLE: next = (start) ? TX_TRANSMIT : TX_IDLE;
            TX_TRANSMIT: next = (!sent_all) ? TX_TRANSMIT : ((start_mid_trans) ? TX_TRANSMIT : TX_IDLE);
            default: next = TX_IDLE;
        endcase
    end

    // start pulse to tx_phy, one per byte
    always_ff @(posedge clk) begin
        if (!reset_n) tx_start <= '0;
        else tx_start <= ((state == TX_IDLE) & start) | (sent_all & start_mid_trans) | ((state == TX_TRANSMIT) &
done & !sent_all);
    end
```

```
  logic [DATA_WIDTH-1:0] group_opener, opcode;
  logic [SLOTS_WIDTH-1:0] group_val;
  always_comb begin
    unique case(group_cnt)
      2'd0: begin
        group_opener = MSG_START_VAL;
        opcode = R_VAL;
        group_val = pixel_data[2];
      end
      2'd1: begin
        group_opener = COMMA_VAL;
        opcode = G_VAL;
        group_val = pixel_data[1];
      end
      2'd2: begin
        group_opener = COMMA_VAL;
        opcode = B_VAL;
        group_val = pixel_data[0];
      end
      2'd3: begin
        group_opener = MSG_CLOSE_VAL;
        {opcode, group_val} = '0;
      end
      default: {group_opener, opcode, group_val} = '0;
    endcase
  end

  // bcd conversion starts with the opener byte of each group; the digits are needed 2 bytes later
  logic bcd_start; assign bcd_start = tx_start & (element_cnt == 3'd0);
  logic bcd_done;
  logic [NUM_DIGITS-1:0][DIG_IN_WIDTH-1:0] bcd_val;
  binary_2_bcd #(.SLOTS(SLOTS)) bin2bcd (
    .clk(clk),
    .reset_n(reset_n),
    .start(bcd_start),
    .bin_in(group_val),
    .bcd_out(bcd_val),
    .done(bcd_done)
  );

  always_comb begin
    unique case(element_cnt)
      3'd0: drive_data = group_opener;
      3'd1: drive_data = opcode;
      3'd2, 3'd3, 3'd4, 3'd5: drive_data = {4'h3, bcd_val[(ELEMENT_NUM-1) - element_cnt]};
      default: drive_data = '0;
    endcase
  end

endmodule
```

src/binary_2_bcd.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module binary_2_bcd #(
    parameter int SLOTS = `SLOTS,
    localparam int SLOTS_WIDTH = $clog2(SLOTS)
)(
    input logic clk, reset_n,
    input logic start,
    input logic [SLOTS_WIDTH-1:0] bin_in,
    output logic [NUM_DIGITS-1:0][DIG_IN_WIDTH-1:0] bcd_out,
    output logic done
);

    // Double Dabble algorithim (same as lab 3).
    // If a digit >= 5 then we need to add 3 to that digit before the shift.
    // SLOTS_WIDTH shifts, MSB first. bin_in is copied to a shift register on start so there is no variable index.
    // No +3 check on thousands: max value is 9999 so thousands can only pass 4 on the last shift.


    localparam [DIG_IN_WIDTH-1:0] DABBLE_LIMIT = 4'd5;
    localparam [DIG_IN_WIDTH-1:0] DABBLE_ADD = 4'd3;
    localparam int CNT_WIDTH = $clog2(SLOTS_WIDTH + 1);
    localparam [CNT_WIDTH-1:0] CNT_ONE = 1;
    localparam [CNT_WIDTH-1:0] LAST_SHIFT = SLOTS_WIDTH;

    // busy: high from start until the last shift
    logic busy;
    logic [CNT_WIDTH-1:0] counter;
    logic shift_done; assign shift_done = (counter == LAST_SHIFT);
    logic shifting; assign shifting = busy & !shift_done;

    always_ff @(posedge clk) begin
        if (!reset_n) busy <= '0;
        else if (start) busy <= '1;
        else if (shift_done) busy <= '0;
    end

    always_ff @(posedge clk) begin
        if (!reset_n) counter <= '0;
        else if (start) counter <= '0;
        else if (shifting) counter <= counter + CNT_ONE;
    end

    logic [DIG_IN_WIDTH-1:0] ones_reg, tens_reg, hundreds_reg, thousands_reg;
    logic [DIG_IN_WIDTH-1:0] ones, tens, hundreds;
    assign ones = (ones_reg >= DABBLE_LIMIT) ? ones_reg + DABBLE_ADD : ones_reg;
    assign tens = (tens_reg >= DABBLE_LIMIT) ? tens_reg + DABBLE_ADD : tens_reg;
    assign hundreds = (hundreds_reg >= DABBLE_LIMIT) ? hundreds_reg + DABBLE_ADD : hundreds_reg;

    // dabble + shift: bin_in MSB enters ones, each digit's MSB moves to the next digit
    logic [SLOTS_WIDTH-1:0] bin_shift;
    always_ff @(posedge clk) begin
        if (!reset_n) begin
```

```
            bin_shift <= '0;
            {thousands_reg, hundreds_reg, tens_reg, ones_reg} <= '0;
        end
        else if (start) begin
            bin_shift <= bin_in;
            {thousands_reg, hundreds_reg, tens_reg, ones_reg} <= '0;
        end
        else if (shifting) begin
            bin_shift <= {bin_shift[SLOTS_WIDTH-2:0], 1'b0};
            ones_reg <= {ones[DIG_IN_WIDTH-2:0], bin_shift[SLOTS_WIDTH-1]};
            tens_reg <= {tens[DIG_IN_WIDTH-2:0], ones[DIG_IN_WIDTH-1]};
            hundreds_reg <= {hundreds[DIG_IN_WIDTH-2:0], tens[DIG_IN_WIDTH-1]};
            thousands_reg <= {thousands_reg[DIG_IN_WIDTH-2:0], hundreds[DIG_IN_WIDTH-1]};
        end
    end

    // done: one cycle pulse after the last shift
    always_ff @(posedge clk) begin
        if (!reset_n) done <= '0;
        else done <= busy & shift_done;
    end

    assign bcd_out = {thousands_reg, hundreds_reg, tens_reg, ones_reg};

endmodule
```

src/tx_phy.sv

```
`include "lab6_definitions.svh"
import lab6_pkg::*;

module tx_phy
#(
    parameter BAUD_RATE = `BAUD_RATE
) (
    input logic clk, reset_n, start,
    input logic [DATA_WIDTH - 1:0] data,
    output logic tx, done
);

    tx_state_t state, next;

    logic tick;
    baud_gen #(.BAUD_RATE(BAUD_RATE)) baud_gen_i (
        .clk(clk),
        .reset_n(reset_n),
        .en((state == TX_TRANSMIT)),
        .baud_tick(tick)
    );

    localparam FRAME_BITS_SIZE = $clog2(FRAME_WIDTH);
    localparam [FRAME_BITS_SIZE-1:0] ONE = 1;

    logic [FRAME_BITS_SIZE-1:0] cnt;
    logic [FRAME_WIDTH-1:0] sh_reg;
```

```
  assign done = (cnt == FRAME_WIDTH - 1) & tick;
  logic sh_cnt_en, load;
  assign sh_cnt_en = (state == TX_TRANSMIT) & tick;
  assign load = (state == TX_IDLE) & start;
  always_ff @(posedge clk) begin
    if (!reset_n) cnt <= '0;
    else if (done | (state == TX_IDLE)) cnt <= '0;
    else if (sh_cnt_en) cnt <= cnt + ONE;
  end

  always_ff @(posedge clk) begin
    if (!reset_n) sh_reg <= '1;
    else if (load) sh_reg <= {1'b1, data, 1'b0};
    else if (sh_cnt_en) sh_reg <= {1'b1, sh_reg[FRAME_WIDTH - 1:1]};
  end

  always_ff @(posedge clk) begin
    if (!reset_n) state <= TX_IDLE;
    else state <= next;
  end

  always_comb begin
    next = state;
    unique case(state)
      TX_IDLE: next = start ? TX_TRANSMIT : TX_IDLE;
      TX_TRANSMIT: next = done ? TX_IDLE : TX_TRANSMIT;
      default: next = state;
    endcase
  end

  assign tx = sh_reg[0];


endmodule
```

src/baud_gen.sv

```
`include "lab6_definitions.svh"
module baud_gen #(
   parameter BAUD_RATE = `BAUD_RATE
) (
   input logic clk, reset_n, en,
   output logic baud_tick
);
   import lab6_pkg::CLK_FREQ;
   localparam COUNT = CLK_FREQ / BAUD_RATE;
   localparam COUNT_BITS = $clog2(COUNT);
   localparam [COUNT_BITS-1:0] ONE = 1;

   logic [COUNT_BITS - 1 : 0] cnt;
   assign baud_tick = (cnt == COUNT - ONE);
   always_ff @(posedge clk) begin
      if (!reset_n) cnt <= '0;
      else if (baud_tick | !en) cnt <= '0;
      else cnt <= cnt + ONE;
   end

endmodule
```

NexysA7-100T_Master.xdc (האילוצים הפעילים בלבד)

```
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35
Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}];
set_false_path -from [get_ports {cpu_reset btnu btnd btnl btnr btnc rx}]
set_false_path -to [get_ports {tx led[*] led16_r led16_g led16_b led17_r led17_g led17_b an[*] seg[*] dp}]
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { led[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { led[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { led[2] }]; #IO_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { led[3] }]; #IO_L8P_T1_D11_14 Sch=led[3]
set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { led[4] }]; #IO_L7P_T1_D09_14 Sch=led[4]
set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { led[5] }]; #IO_L18N_T2_A11_D27_14
Sch=led[5]
set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { led[6] }]; #IO_L17P_T2_A14_D30_14
Sch=led[6]
set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { led[7] }]; #IO_L18P_T2_A12_D28_14
Sch=led[7]
set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { led[8] }]; #IO_L16N_T2_A15_D31_14
Sch=led[8]
set_property -dict { PACKAGE_PIN T15   IOSTANDARD LVCMOS33 } [get_ports { led[9] }]; #IO_L14N_T2_SRCC_14 Sch=led[9]
set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports { led[10] }]; #IO_L22P_T3_A05_D21_14
Sch=led[10]
set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { led[11] }];
#IO_L15N_T2_DQS_DOUT_CSO_B_14 Sch=led[11]
set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports { led[12] }]; #IO_L16P_T2_CSI_B_14
Sch=led[12]
set_property -dict { PACKAGE_PIN V14   IOSTANDARD LVCMOS33 } [get_ports { led[13] }]; #IO_L22N_T3_A04_D20_14
Sch=led[13]
set_property -dict { PACKAGE_PIN V12   IOSTANDARD LVCMOS33 } [get_ports { led[14] }]; #IO_L20N_T3_A07_D23_14
Sch=led[14]
set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { led[15] }]; #IO_L21N_T3_DQS_A06_D22_14
Sch=led[15]
set_property -dict { PACKAGE_PIN R12   IOSTANDARD LVCMOS33 } [get_ports { led16_b }]; #IO_L5P_T0_D06_14
Sch=led16_b
```

```
set_property -dict { PACKAGE_PIN M16   IOSTANDARD LVCMOS33 } [get_ports { led16_g }]; #IO_L10P_T1_D14_14 Sch=led16_g
set_property -dict { PACKAGE_PIN N15   IOSTANDARD LVCMOS33 } [get_ports { led16_r }]; #IO_L11P_T1_SRCC_14 Sch=led16_r
set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [get_ports { led17_b }]; #IO_L15N_T2_DQS_ADV_B_15 Sch=led17_b
set_property -dict { PACKAGE_PIN R11   IOSTANDARD LVCMOS33 } [get_ports { led17_g }]; #IO_0_14 Sch=led17_g
set_property -dict { PACKAGE_PIN N16   IOSTANDARD LVCMOS33 } [get_ports { led17_r }]; #IO_L11N_T1_SRCC_14 Sch=led17_r
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { seg[6] }]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { seg[5] }]; #IO_25_14 Sch=cb
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { seg[4] }]; #IO_25_15 Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { seg[3] }]; #IO_L17P_T2_A26_15 Sch=cd
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { seg[2] }]; #IO_L13P_T2_MRCC_14 Sch=ce
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { seg[1] }]; #IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { seg[0] }]; #IO_L4P_T0_D04_14 Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { dp }]; #IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { an[0] }]; #IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { an[1] }]; #IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { an[2] }]; #IO_L24P_T3_A01_D17_14 Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { an[3] }]; #IO_L19P_T3_A22_15 Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { an[4] }]; #IO_L8N_T1_D12_14 Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { an[5] }]; #IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { an[6] }]; #IO_L23P_T3_35 Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { an[7] }]; #IO_L23N_T3_A02_D18_14 Sch=an[7]
set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { cpu_reset }]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { btnc }]; #IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { btnu }]; #IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { btnl }]; #IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { btnr }]; #IO_L10N_T1_D15_14 Sch=btnr
set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { btnd }]; #IO_L9N_T1_DQS_D13_14 Sch=btnd
set_property -dict { PACKAGE_PIN C4    IOSTANDARD LVCMOS33 } [get_ports { rx }]; #IO_L7P_T1_AD6P_35 Sch=uart_txd_in
set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports { tx }]; #IO_L11N_T1_SRCC_35 Sch=uart_rxd_out
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

נספח ב': קוד הטסטבנצ'ים

הטסטבנצ'ים לפי הסדר של פרק 4:

sim/tb_debounce.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_debounce;
  timeunit 1ns;
  timeprecision 1ps;
  localparam DEB_WAIT = (`DEB_COUNT + 2 + 1) * 10; // 2 for sync + 1 for state reg
  bit clk = '0;
  always #5 clk = !clk;

  logic reset_n = '0;
  logic io_in = '0;
  logic io_d;

  debounce #(.DEB_COUNT(`DEB_COUNT)) debounce_i (
    .clk(clk),
    .reset_n(reset_n),
    .io_in(io_in),
    .io_d(io_d)
  );

  initial $monitor("Time: %0t | io_in=%0b | io_d=%0b", $time, io_in, io_d);

  task reset_dut();
    $display("Time: %0t | TASK: Reset DUT", $time);
    reset_n = '0; #(CLK_PERIOD);
    assert(!io_d);
    reset_n = '1;
  endtask

  bit toggle_value;

  task toggle_io(input bit t_val);
    $display("Time: %0t | TASK: Toggle io to: %0b", $time, t_val);
    io_in = t_val;
    toggle_value = t_val;
  endtask

  task wait_after_toggle(input int wait_time);
    $display("Time: %0t | TASK: Wait for: %0d ns", $time, wait_time);
    #(wait_time);
  endtask


  initial begin
    reset_dut();

    // 1. Regular
    $display("Time: %0t | Toggle io to 1", $time);
    toggle_io(1); wait_after_toggle(DEB_WAIT); assert(io_d);
```

```
        $display("Time: %0t | Toggle io to 0", $time);
        toggle_io(0); wait_after_toggle(DEB_WAIT); assert(!io_d);

        #(CLK_PERIOD);

        //2. Debounce Teset
        $display("Time: %0t | Debounce teset", $time);
        toggle_io(1); wait_after_toggle(DEB_WAIT / 3); assert(!io_d);
        toggle_io(0); wait_after_toggle(DEB_WAIT / 6); assert(!io_d);
        toggle_io(1); wait_after_toggle(DEB_WAIT / 10); assert(!io_d);
        toggle_io(0); wait_after_toggle(DEB_WAIT / 5); assert(!io_d);
        toggle_io(1); wait_after_toggle(DEB_WAIT); assert(io_d);

        #(CLK_PERIOD);

        //3. Reset mid
        toggle_io(0); wait_after_toggle(DEB_WAIT / 2); assert(io_d);
        reset_dut(); assert(!io_d);
        toggle_io(1); wait_after_toggle(DEB_WAIT); assert(io_d);

        #(CLK_PERIOD);

        $finish;
    end

    property p_io_d_reset_val;
        @(posedge clk)
        (!reset_n) |=> (!io_d);
    endproperty

    assert property(p_io_d_reset_val);

endmodule
```

sim/tb_debounce_cluster.sv

```
import lab6_pkg::*;

module tb_debounce_cluster;
    timeunit 1ns;
    timeprecision 1ps;

    bit clk = '0;
    always #5 clk = !clk;

    logic cpu_reset = '0;
    logic cpu_reset_d;
    btn_ctf_t io_in = '0;
    btn_ctf_t io_d;

    debounce_cluster debounce_cluster_i (
        .clk(clk),
        .cpu_reset(cpu_reset),
        .io_in(io_in),
```

```
        .io_d(io_d),
        .cpu_reset_d(cpu_reset_d)
    );

    localparam DEB_COUNT = `DEB_COUNT;
    localparam SYNC_WAIT = 2 * CLK_PERIOD;
    localparam COUNT_WAIT = DEB_COUNT * CLK_PERIOD;
    localparam LATCH_WAIT = 1 * CLK_PERIOD;

    localparam TOTAL_WAIT_FOR_CHANGE = SYNC_WAIT + COUNT_WAIT + LATCH_WAIT;
    localparam RESET_WAIT_FOR_CHANGE = DEB_COUNT + 1;

    initial $monitor("Time: %5d | cpu_reset=%b | cpu_reset_d=%b | io_in=%b | io_d=%b", $time, cpu_reset,
cpu_reset_d, io_in, io_d);

    logic [BUTTON_NUM-1:0] io_cfg = {{(BUTTON_NUM-1){1'b0}}, {1'b1}};
    integer i;
    initial begin
        cpu_reset = '1; #(TOTAL_WAIT_FOR_CHANGE);

        // show that every button has a debounce
        for (i = 0; i < BUTTON_NUM; i++) begin
            io_in = io_cfg << i; #(TOTAL_WAIT_FOR_CHANGE);
        end

        // show reset debounce
        $display("Time: %5d | RESET Test", $time);
        cpu_reset = '0; #(TOTAL_WAIT_FOR_CHANGE);
        cpu_reset = '1; #(TOTAL_WAIT_FOR_CHANGE * 2);

        // io_cfg is now 10000..
        // show debounce efect only working when pressed (changed for more than COUNT_WAIT)
        $display("Time: %5d | DEBOUNCE Test", $time);
        io_in = '0; #(COUNT_WAIT/4);
        io_in = '1; #(COUNT_WAIT/6);
        io_in = '0; #(COUNT_WAIT/2);
        io_in = '1; #(COUNT_WAIT - 1);
        io_in = '0; #(COUNT_WAIT);
        #(SYNC_WAIT + CLK_PERIOD + LATCH_WAIT);
        $finish;
    end

    property reset_cnt_reset;
    @(posedge clk)
    ($past(debounce_cluster_i.reset_edge)) |-> debounce_cluster_i.reset_cnt == '0;
    endproperty
    assert property(reset_cnt_reset);

    property reset_cnt_no_wrap;
        @(posedge clk)
```

```
      ($past(debounce_cluster_i.reset_stable) & !$past(debounce_cluster_i.reset_edge)) |->
debounce_cluster_i.reset_cnt == DEB_COUNT-1;
   endproperty
   assert property(reset_cnt_no_wrap);



endmodule
```

sim/tb_rx_phy.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"
module tb_rx_phy;

   bit clk = '0;
   always #5 clk = !clk;

   logic reset_n = '0;
   logic en = '0;
   logic rx = '1;
   logic [DATA_WIDTH-1:0] data;
   logic done;

   rx_phy dut (
      .clk(clk),
      .reset_n(reset_n),
      .en(en),
      .rx(rx),
      .data(data),
      .done(done)
   );

   task reset_dut();
      $display("Time: %0t | TASK: Reset DUT", $time);
      reset_n = '0; #(CLK_PERIOD);
      assert(data === '0);
      assert(done === '0);
      reset_n = '1;
   endtask

   function logic [FRAME_WIDTH-1:0] get_frame (input logic [DATA_WIDTH-1:0] data);
      return {1'b1, data, 1'b0};
   endfunction

   localparam BIT_WAIT_CYCLES = CLK_FREQ / `BAUD_RATE;
   localparam BIT_WAIT = BIT_WAIT_CYCLES * CLK_PERIOD;
   localparam MID_BIT_WAIT = BIT_WAIT / 2;
   localparam SYNC_WAIT = 3 * CLK_PERIOD; // From waves


   bit sent_frame;
   task send_frame(input [DATA_WIDTH-1:0] d_in);
      logic [FRAME_WIDTH-1:0] frame;
```

```
    sent_frame = '1;
    frame = get_frame(d_in);
    $display("Time: %0t | TASK: Sending frame: %b", $time, frame);
    for (int i = 0; i < FRAME_WIDTH; i++) begin
        $display("Time: %0t | TASK: Sending bit[%0d] | rx = %b", $time, i, frame[i]);
        rx = frame[i];
        if (i == FRAME_WIDTH-1) begin
            #(MID_BIT_WAIT + SYNC_WAIT);
            assert(done);
            assert(data === d_in);
            #(BIT_WAIT - (MID_BIT_WAIT + SYNC_WAIT));
        end else #(BIT_WAIT);
    end
    sent_frame = '0;
    #(CLK_PERIOD);
endtask

task send_frame_not_en(input [DATA_WIDTH-1:0] d_in, input logic [DATA_WIDTH-1:0] prev_d_in);
    logic [FRAME_WIDTH-1:0] frame;
    sent_frame = '1;
    frame = get_frame(d_in);
    $display("Time: %0t | TASK: Sending frame: %b", $time, frame);
    for (int i = 0; i < FRAME_WIDTH; i++) begin
        $display("Time: %0t | TASK: Sending bit[%0d] | rx = %b", $time, i, frame[i]);
        rx = frame[i];
        if (i == FRAME_WIDTH-1) begin
            #(MID_BIT_WAIT + SYNC_WAIT);
            assert(!done);
            assert(data === {d_in[3:0], prev_d_in[7:4]});
            #(BIT_WAIT - (MID_BIT_WAIT + SYNC_WAIT));
        end else if (i == 5) begin
            en_dut('0);
            en_dut('1);
            #(BIT_WAIT - 2*CLK_PERIOD);
        end else #(BIT_WAIT);
    end
    sent_frame = '0;
    #(CLK_PERIOD);
endtask

task en_dut(input bit e);
    $display("Time: %0t | TASK: %s DUT", $time, e ? "Enable" : "Disable");
    en = e; #(CLK_PERIOD);
endtask

initial begin
    reset_dut();
    en_dut('1);
    send_frame(8'hA5);
    send_frame(8'h5A);
    send_frame_not_en(8'h84, 8'h5A);
    send_frame(8'hFF);
    send_frame(8'h00);

    $finish;
end
```

```
    property p_done_one_cyle;
        @(posedge clk)
        disable iff(!reset_n)
        ($past(done)) |-> (!done);
    endproperty
    assert property(p_done_one_cyle);

    // done once per frame
    int cnt;

    always_ff @(posedge clk) begin
        if (!reset_n) cnt <= '0;
        else if (sent_frame & done) cnt <= cnt + 1;
        else if (!sent_frame) cnt <= '0;
        assert(cnt <= 1);
    end

    property p_done_low_not_in_frame;
        @(posedge clk)
        disable iff(!reset_n)
        (!sent_frame) |-> (done == '0);
    endproperty
    assert property(p_done_low_not_in_frame);



endmodule
```

sim/tb_rx_mac.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_rx_mac;

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic done = '0;
    logic [DATA_WIDTH-1:0] data_in = '0;
    packet_cfg_t pkt_data;
    logic pkt_valid;

    rx_mac dut (
        .clk(clk),
        .reset_n(reset_n),
        .done(done),
        .data_in(data_in),
        .pkt_data(pkt_data),
        .pkt_valid(pkt_valid)
    );
```

```
task reset_dut();
   $display("Time: %0t | TASK: Reset DUT", $time);
   reset_n = '0; #(CLK_PERIOD);
   assert(!pkt_valid);
   assert(pkt_data === '0);
   reset_n = '1;
endtask

// full flow : {RXXX,GXXX,BXXX}
task drive_full_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
   for (int i = 0; i < 16; i++) begin
      case(i)
         0: data_in = MSG_START_VAL;
         1: data_in = R_VAL;
         2: data_in = row_val[2];
         3: data_in = row_val[1];
         4: data_in = row_val[0];
         5, 10: data_in = COMMA_VAL;
         6: data_in = G_VAL;
         7: data_in = col_val[2];
         8: data_in = col_val[1];
         9: data_in = col_val[0];
         11: data_in = B_VAL;
         12: data_in = val[2];
         13: data_in = val[1];
         14: data_in = val[0];
         15: data_in = MSG_CLOSE_VAL;
         default: data_in = MSG_START_VAL;
      endcase
      done = '1; #(CLK_PERIOD);
      if (i == 15) begin
         assert(pkt_valid === '1);
         //$display("Time: %0t | %h", $time, build_valid_pkt(row_val, col_val, val));
         assert(pkt_data === build_exp_pkt(row_val, col_val, val));
      end
      done = '0; #(CLK_PERIOD);
   end
endtask

task drive_full_input_bad(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
   for (int i = 0; i < 17; i++) begin
      case(i)
         0: data_in = MSG_START_VAL;
         1: data_in = R_VAL;
         2: data_in = row_val[2];
         3: data_in = row_val[1];
         4: data_in = row_val[0];
         5, 10: data_in = COMMA_VAL;
         6: data_in = G_VAL;
         7: data_in = col_val[2];
         8: data_in = col_val[1];
         9: data_in = col_val[0];
         11: data_in = B_VAL;
         12: data_in = val[2];
         13: data_in = val[1];
```

```
            14: data_in = val[0];
            15: data_in = MSG_START_VAL;
            16: data_in = MSG_CLOSE_VAL;
            default: data_in = MSG_START_VAL;
        endcase
        done = '1; #(CLK_PERIOD);
        if (i == 15) begin
            assert(pkt_valid === '0);
            //$display("Time: %0t | %h", $time, build_exp_pkt(row_val, col_val, val));
            assert(pkt_data !== build_exp_pkt(row_val, col_val, val));
        end
        done = '0; #(CLK_PERIOD);
    end
endtask

task drive_restart_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
    for (int i = 0; i < 20; i++) begin
        case(i)
            0, 4: data_in = MSG_START_VAL;
            1, 5: data_in = R_VAL;
            2, 6: data_in = row_val[2];
            3, 7: data_in = row_val[1];
            8: data_in = row_val[0];
            9, 14: data_in = COMMA_VAL;
            10: data_in = G_VAL;
            11: data_in = col_val[2];
            12: data_in = col_val[1];
            13: data_in = col_val[0];
            15: data_in = B_VAL;
            16: data_in = val[2];
            17: data_in = val[1];
            18: data_in = val[0];
            19: data_in = MSG_CLOSE_VAL;
            default: data_in = MSG_START_VAL;
        endcase
        done = '1; #(CLK_PERIOD);
        if (i == 19) begin
            assert(pkt_valid === '1);
            assert(pkt_data === build_exp_pkt(row_val, col_val, val));
        end
        done = '0; #(CLK_PERIOD);
    end
endtask

task drive_one_group_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val);
    for (int i = 0; i < 6; i++) begin
        case(i)
            0: data_in = MSG_START_VAL;
            1: data_in = R_VAL;
            2: data_in = row_val[2];
            3: data_in = row_val[1];
            4: data_in = row_val[0];
            5: data_in = MSG_CLOSE_VAL;
            default: data_in = MSG_START_VAL;
        endcase
        done = '1; #(CLK_PERIOD);
```

```
        if (i == 5) begin
          assert(pkt_valid === '1);
          assert(pkt_data === {{10*DATA_WIDTH{1'b0}}, MSG_START_VAL, R_VAL, row_val,
MSG_CLOSE_VAL});
        end
        done = '0; #(CLK_PERIOD);
      end
    endtask

    task drive_four_groups_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
      for (int i = 0; i < 21; i++) begin
        case(i)
          0: data_in = MSG_START_VAL;
          1, 16: data_in = R_VAL;
          2, 17: data_in = row_val[2];
          3, 18: data_in = row_val[1];
          4, 19: data_in = row_val[0];
          5, 10, 15: data_in = COMMA_VAL;
          6: data_in = G_VAL;
          7: data_in = col_val[2];
          8: data_in = col_val[1];
          9: data_in = col_val[0];
          11: data_in = B_VAL;
          12: data_in = val[2];
          13: data_in = val[1];
          14: data_in = val[0];
          20: data_in = MSG_CLOSE_VAL;
          default: data_in = MSG_START_VAL;
        endcase
        done = '1; #(CLK_PERIOD);
        if (i >= 15) begin
          assert(pkt_valid === '0);
        end
        done = '0; #(CLK_PERIOD);
      end
    endtask

    function packet_cfg_t build_exp_pkt(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
      packet_cfg_t p;
      p.msg_start = MSG_START_VAL;
      p.op_2      = R_VAL;
      p.value_2   = row_val;
      p.comma0    = COMMA_VAL;
      p.op_1      = G_VAL;
      p.value_1   = col_val;
      p.comma1    = COMMA_VAL;
      p.op_0      = B_VAL;
      p.value_0   = val;
      p.msg_close = MSG_CLOSE_VAL;
      return p;
    endfunction

    initial begin
      reset_dut();

      drive_full_input(32'h313233, 32'h343536, 32'h373839);
```

```
    #(CLK_PERIOD * 5);

    drive_full_input_bad(32'h090807, 32'h060504, 32'h030201);
    #(CLK_PERIOD * 5);

    drive_restart_input(32'h323535, 32'h303030, 32'h313238);
    #(CLK_PERIOD * 5);

    drive_one_group_input(32'h313233);
    #(CLK_PERIOD * 5);

    drive_four_groups_input(32'h313233, 32'h343536, 32'h373839);
    #(CLK_PERIOD * 5);

    $finish;
  end

endmodule
```

sim/tb_parser.sv

```
import lab6_pkg::*;

module tb_parser;

  bit clk = '0;
  always #5 clk = !clk;

  logic reset_n = '0;
  packet_cfg_t pkt_data = '0;
  logic pkt_valid = '0;
  pixel_parsed_t pixel_parsed_data;
  logic led_parsed_data;
  logic pixel_valid, led_valid;

  parser dut (
    .clk(clk),
    .reset_n(reset_n),
    .pkt_data(pkt_data),
    .pkt_valid(pkt_valid),
    .pixel_parsed_data(pixel_parsed_data),
    .led_parsed_data(led_parsed_data),
    .pixel_valid(pixel_valid),
    .led_valid(led_valid)
  );

  task reset_dut();
    $display("Time: %0t | TASK: Reset DUT", $time);
    reset_n = '0; #(CLK_PERIOD);
    assert(pixel_parsed_data === '0);
    assert(pixel_valid === '0);
    assert(led_parsed_data === '0);
    assert(led_valid === '0);
```

```
    reset_n = '1;
  endtask


  function logic [DIGITS-1:0][DATA_WIDTH-1:0] hex_to_ascii(input logic [SCREENS_PER_SEGMENT-
1:0][DIG_IN_WIDTH-1:0] x);
    logic [DIGITS-1:0][DATA_WIDTH-1:0] temp;
    temp[2] = "0" + x / 100;
    temp[1] = "0" + (x/10) % 10;
    temp[0] = "0" + x % 10;
    return temp;
  endfunction

  function packet_cfg_t convert_parsed_to_packet(input logic [2:0][DIGITS-1:0][DATA_WIDTH-1:0] x);
    return {"{", pkt_data.op_2 , x[2], ",", pkt_data.op_1 , x[1], ",", pkt_data.op_0 , x[0], "}"};
  endfunction

  function packet_cfg_t full_conversion_to_packet(input pixel_parsed_t x);
    logic [DIGITS-1:0][DATA_WIDTH-1:0] r,g,b;
    r = hex_to_ascii(x.value_2);
    g = hex_to_ascii(x.value_1);
    b = hex_to_ascii(x.value_0);
    return convert_parsed_to_packet({r, g, b});
  endfunction

  task drive_pkt_data(input packet_cfg_t pkt);
    $display("Time: %0t | TASK: Drive pkt_data with: %h", $time, pkt);
    pkt_data = pkt;
  endtask;

  task drive_pkt_valid();
    $display("Time: %0t | TASK: Drive pkt_valid to 1 cycle", $time);
    pkt_valid = '1; #(CLK_PERIOD);
    pkt_valid = '0;
  endtask


  task drive_flow (input packet_cfg_t pkt);
    drive_pkt_data(pkt);
    drive_pkt_valid();
    assert(pixel_valid === '1);
    assert(pkt_data === full_conversion_to_packet(pixel_parsed_data));
    #(CLK_PERIOD);
  endtask

  task drive_false_flow(input packet_cfg_t pkt);
    pixel_parsed_t temp;
    temp = pixel_parsed_data;
    drive_pkt_data(pkt);
    drive_pkt_valid();
    assert(pixel_valid === '0);
    assert(pixel_parsed_data === temp);
    #(CLK_PERIOD);
  endtask
```

```
    initial begin
        reset_dut();

        // First check regular
        drive_flow("{R123,G045,B200}");
        drive_flow("{R255,G255,B255}");

        drive_false_flow("{A111,B222,X444}");

        drive_false_flow("{R256,G000,B000}");
        drive_false_flow("{R000,G256,B000}");
        drive_false_flow("{R000,G000,B256}");

        drive_flow("{r005,g050,b055}");
        drive_flow("{R123,g089,B111}");

        $finish;


    end




endmodule
```

sim/tb_selector.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_selector;
    timeunit 1ns;
    timeprecision 1ps;

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    btn_ctf_t btn_in = '0;
    pixel_parsed_t parsed_pixel_value = '0;
    logic parsed_led_value = '0;
    logic pixel_valid = '0;
    logic led_valid = '0;
    pixel_parsed_t display_pixel, pwm_pixel;
    logic pwm_new_data, pwm_led, display_led;
    leds_cfg_t leds_out;

    selector dut (
        .clk(clk),
        .reset_n(reset_n),
        .btn_in(btn_in),
        .parsed_pixel_value(parsed_pixel_value),
        .parsed_led_value(parsed_led_value),
```

```
        .pixel_valid(pixel_valid),
        .led_valid(led_valid),
        .display_pixel(display_pixel),
        .pwm_pixel(pwm_pixel),
        .pwm_new_data(pwm_new_data),
        .pwm_led(pwm_led),
        .display_led(display_led),
        .leds_out(leds_out)
    );

    localparam leds_cfg_t T0_LEDS = 16'h0030;
    localparam leds_cfg_t T1_LEDS = 16'h0180;
    localparam leds_cfg_t T2_LEDS = 16'h0600;
    localparam leds_cfg_t T3_LEDS = 16'h3000;

    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD);
        assert(display_pixel === '0) else $error("display_pixel = %h after reset", display_pixel);
        assert(pwm_pixel === '0) else $error("pwm_pixel = %h after reset", pwm_pixel);
        assert(display_led === '0) else $error("display_led = %b after reset", display_led);
        assert(pwm_led === '0) else $error("pwm_led = %b after reset", pwm_led);
        assert(pwm_new_data === '0) else $error("pwm_new_data = %b after reset", pwm_new_data);
        assert(leds_out === T0_LEDS) else $error("leds_out = %h after reset, expected %h", leds_out, T0_LEDS);
        reset_n = '1;
    endtask

    // one press = level up for one clock, then back down (the edge detector inside makes one pulse)
    task press(input btn_ctf_t b);
        $display("Time: %0t | TASK: Press %b (up down left right center)", $time, b);
        btn_in = b; #(CLK_PERIOD);
        btn_in = '0; #(CLK_PERIOD);
    endtask

    task hold(input btn_ctf_t b, input int cycles);
        $display("Time: %0t | TASK: Hold %b for %0d cycles", $time, b, cycles);
        btn_in = b; #(CLK_PERIOD * cycles);
        btn_in = '0; #(CLK_PERIOD);
    endtask

    task send_pixel(input pixel_parsed_t p);
        $display("Time: %0t | TASK: UART pixel %h", $time, p);
        parsed_pixel_value = p; pixel_valid = '1; #(CLK_PERIOD);
        pixel_valid = '0;
        assert(pwm_new_data === '1) else $error("pwm_new_data = %b after pixel_valid, expected 1",
pwm_new_data);
        #(CLK_PERIOD);
        assert(pwm_new_data === '0) else $error("pwm_new_data = %b, expected a one cycle pulse",
pwm_new_data);
    endtask

    task send_led(input logic l);
        $display("Time: %0t | TASK: UART led %b", $time, l);
        parsed_led_value = l; led_valid = '1; #(CLK_PERIOD);
        led_valid = '0; #(CLK_PERIOD);
    endtask
```

```
  task check_pixels(input pixel_parsed_t exp_display, input pixel_parsed_t exp_pwm);
    assert(display_pixel === exp_display) else $error("display_pixel = %h, expected %h", display_pixel,
exp_display);
    assert(pwm_pixel === exp_pwm) else $error("pwm_pixel = %h, expected %h", pwm_pixel, exp_pwm);
  endtask

  task check_leds(input logic exp_display, input logic exp_pwm);
    assert(display_led === exp_display) else $error("display_led = %b, expected %b", display_led,
exp_display);
    assert(pwm_led === exp_pwm) else $error("pwm_led = %b, expected %b", pwm_led, exp_pwm);
  endtask

  task check_cursor(input leds_cfg_t exp);
    assert(leds_out === exp) else $error("leds_out = %h, expected %h", leds_out, exp);
  endtask

  btn_ctf_t UP    = '{up: 1, default: 0};
  btn_ctf_t DOWN  = '{down: 1, default: 0};
  btn_ctf_t LEFT  = '{left: 1, default: 0};
  btn_ctf_t RIGHT = '{right: 1, default: 0};
  btn_ctf_t CENTER = '{center: 1, default: 0};

  initial begin
    reset_dut();

    // 1. UART pixel loads both copies
    send_pixel(24'h10_20_30);
    check_pixels(24'h10_20_30, 24'h10_20_30);

    // 2. up on T0 (blue): only display changes, no pwm_new_data
    press(UP);
    check_pixels(24'h10_20_31, 24'h10_20_30);
    assert(pwm_new_data === '0) else $error("pwm_new_data = 1 after up");

    // 3. cursor walks left T0 -> T3 and stops there
    press(LEFT); check_cursor(T1_LEDS);
    press(UP);   check_pixels(24'h10_21_31, 24'h10_20_30);
    press(LEFT); check_cursor(T2_LEDS);
    press(DOWN); check_pixels(24'h0F_21_31, 24'h10_20_30);
    press(LEFT); check_cursor(T3_LEDS);
    press(LEFT); check_cursor(T3_LEDS);

    // 4. led field: up -> 1, up again stays 1, down -> 0
    press(UP);   check_leds('1, '0);
    press(UP);   check_leds('1, '0);
    press(DOWN); check_leds('0, '0);
    press(UP);   check_leds('1, '0);

    // 5. center copies display -> pwm, with a pwm_new_data pulse
    btn_in = CENTER; #(CLK_PERIOD);
    btn_in = '0;
    assert(pwm_new_data === '1) else $error("pwm_new_data = %b after center, expected 1",
pwm_new_data);
    #(CLK_PERIOD);
```

```
        assert(pwm_new_data === '0) else $error("pwm_new_data = %b, expected a one cycle pulse",
pwm_new_data);
        check_pixels(24'h0F_21_31, 24'h0F_21_31);
        check_leds('1, '1);

        // 6. cursor walks right T3 -> T0 and stops there
        press(RIGHT); press(RIGHT); press(RIGHT); check_cursor(T0_LEDS);
        press(RIGHT); check_cursor(T0_LEDS);

        // 7. saturation: 255 + 1 stays 255, 0 - 1 stays 0
        send_pixel(24'hFF_00_FF);
        press(UP);   check_pixels(24'hFF_00_FF, 24'hFF_00_FF);
        press(LEFT); press(DOWN); check_pixels(24'hFF_00_FF, 24'hFF_00_FF);
        press(LEFT); press(UP);   check_pixels(24'hFF_00_FF, 24'hFF_00_FF);

        // 8. holding a button is one step (edge detect)
        press(DOWN); check_pixels(24'hFE_00_FF, 24'hFF_00_FF);
        hold(DOWN, 10); check_pixels(24'hFD_00_FF, 24'hFF_00_FF);

        // 9. UART led loads both copies, no pwm_new_data
        send_led('0);
        check_leds('0, '0);
        assert(pwm_new_data === '0) else $error("pwm_new_data = 1 after led_valid");

        #(CLK_PERIOD); $finish;
    end

endmodule
```

sim/tb_display_ctrl.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_display_ctrl;

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic led_17_16 = '0;
    pixel_parsed_t display_pixel = '0;
    display_cfg_t display_out;

    display_ctrl dut (
        .clk(clk),
        .reset_n(reset_n),
        .led_17_16(led_17_16),
        .display_pixel(display_pixel),
        .display_out(display_out)
    );

    localparam CYCLES_PER_AN = CLK_FREQ / `DISPLAY_RR;
    localparam AN_WAIT = CLK_PERIOD * CYCLES_PER_AN;
```

```
  task reset_dut();
    $display("Time: %0t | Task: Reset DUT", $time);
    reset_n = '0; #(CLK_PERIOD);
    assert(display_out.an === AN_RESET_VAL);
    assert(display_out.seg === SEG_RESET_VAL);
    assert(display_out.dp === 1'b1);
    reset_n = '1;
  endtask

  function [AN_WIDTH-1:0] exp_an(input int i);
    return ~(1 << i);
  endfunction

  task drive(input pixel_parsed_t p, input logic led);
    $display("Time: %0t | TASK: Driving inputs: display_pixel=%h, led_17_16=%b", $time, p, led);
    display_pixel = p;
    led_17_16 = led;
    for (int i = 0; i < AN_WIDTH; i++) begin
      assert(display_out.an === exp_an(i));
      #(AN_WAIT);
    end
  endtask

  initial begin
    reset_dut();

    drive(24'h123456, '0);
    drive(24'hABCDEF, '1);

    #(CLK_PERIOD * 5);
    $finish;
  end

endmodule
```

sim/tb_seven_segment.sv

```
import lab6_pkg::*;

module tb_seven_segment;
  timeunit 1ns;
  timeprecision 1ps;
  logic [DIG_IN_WIDTH-1:0] data_val;
  logic [SS_DIG_WIDTH-1:0] ss_dig;

  seven_segment ss_i(
    .data_val(data_val),
    .ss_dig(ss_dig)
  );

  initial $monitor("Time: %5d | data_val=%h | ss_dig=%b", $time, data_val, ss_dig);

  integer i;
```

```
    initial begin
        data_val = '0; #(CLK_PERIOD);
        for (i = 0; i < 2**DIG_IN_WIDTH; i++) begin
            data_val = i[DIG_IN_WIDTH-1:0]; #(CLK_PERIOD);
        end

        $finish;
    end

endmodule
```

sim/tb_gemma_correction.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_gemma_correction;
    timeunit 1ns;
    timeprecision 1ps;
    pixel_parsed_t pixel_in;
    corrected_pixel_t pixel_out;

    localparam real MAX_PIXEL_STRENGTH = 255.0;

    gemma_correction dut (
        .pixel_in(pixel_in),
        .pixel_out(pixel_out)
    );

    task drive_c(input pixel_parsed_t c);
        corrected_pixel_t temp;
        pixel_in = c; #(CLK_PERIOD);
        temp = {full_convertion(c.value_2), full_convertion(c.value_1), full_convertion(c.value_0)};
        $display("Time: %0t | pixel_out.red=%d | calc_pixel.red=%d", $time, pixel_out.red, temp.red);
        assert(pixel_out.red === temp.red);
    endtask

    function real cal_c(input [DATA_WIDTH-1:0] val);
        return val / MAX_PIXEL_STRENGTH;
    endfunction

    function real lin_c(input real c);
        if (c <= 0.04045) return c / 12.92;
        else return $pow(((c + 0.055) / 1.055), 2.4);
    endfunction

    function logic [GAMMA_WIDTH-1:0] table_val(input real l);
        return GAMMA_WIDTH'(int'(l * 1023));
    endfunction

    function [GAMMA_WIDTH-1:0] full_convertion(input [DATA_WIDTH-1:0] val);
        return table_val(lin_c(cal_c(val)));
    endfunction
```

```
    initial begin
        for (int red = 0; red < 256; red++) begin
            drive_c({DATA_WIDTH'(red), DATA_WIDTH'(0), DATA_WIDTH'(0)});
        end
        $finish;
    end



endmodule
```

sim/tb_cie_scailing.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"
module tb_cie_scailing;
    timeunit 1ns;
    timeprecision 1ps;
    localparam SLOTS_WIDTH = $clog2(`SLOTS);
    localparam real GREEN_SCALE = 0.587;
    localparam real RED_SCALE = 0.299;
    localparam real BLUE_SCALE = 0.114;

    typedef real real_rgb_t [2:0];

    corrected_pixel_t pixel_in; //[GAMMA_WIDTH-1:0]
    logic [2:0][SLOTS_WIDTH-1:0] pixel_out;

    cie_scaling #(.SLOTS(`SLOTS)) dut (
        .pixel_in(pixel_in),
        .pixel_out(pixel_out)
    );

    function real_rgb_t scale_pixel_in(input corrected_pixel_t p);
        real_rgb_t temp;
        temp[2] = p.red * (RED_SCALE/RED_SCALE) * ((`SLOTS-1.0) / (2**GAMMA_WIDTH - 1));
        temp[1] = p.green * (RED_SCALE/GREEN_SCALE) * ((`SLOTS-1.0) / (2**GAMMA_WIDTH - 1));
        temp[0] = p.blue * 1.0 /* RED_SCALE/BLUE_SCALE = 2.62 > 1, clamped */ * ((`SLOTS-1.0) /
(2**GAMMA_WIDTH - 1));
        return temp;
    endfunction

    /*
        AI MADE:
        Tolerance: the shift-add constant deviates from Rec.601 by -0.3% (green, 0.5078 vs 0.509); blue is
clamped to 1,
        plus up to 3 floors and the shift losing the last slot. Checked by brute force over every value and
        SLOTS 512..8192: max |rtl - ideal| = 2% of the value + 3.
    */
    localparam real TOL_REL = 0.02;
    localparam real TOL_ABS = 3.0;

    task scale_check(input logic[2:0][SLOTS_WIDTH-1:0] p_out);
        real_rgb_t exp;
```

```
        real diff, tol;
        $display("Time: %0t | TASK: Check p_out within 2 precent of actual real value", $time);
        exp = scale_pixel_in(pixel_in);
        $display("Time: %0t | pixel_out.red=%0d exp_red=%0.2f | pixel_out.green=%0d exp_green=%0.2f |
pixel_out.blue=%0d exp_blue=%0.2f", $time, p_out[2], exp[2], p_out[1], exp[1], p_out[0], exp[0]);
        for (int c = 0; c < 3; c++) begin
            diff = p_out[c] - exp[c];
            tol = TOL_REL * exp[c] + TOL_ABS;
            assert ((diff <= tol) && (-diff <= tol)) else $error("chan %0d in=%0d: got %0d, ideal %0.2f, diff %0.2f >
tol %0.2f", c, pixel_in[c*GAMMA_WIDTH +: GAMMA_WIDTH], p_out[c], exp[c], diff, tol);
        end
    endtask

    task drive_rgb_pixel(input corrected_pixel_t p);
        $display("Time: %0t | TASK: Drive pixel_in: red=%0d green=%0d blue=%0d", $time, p.red, p.green,
p.blue);
        pixel_in = p; #(CLK_PERIOD);
        scale_check(pixel_out);
    endtask

    initial begin
        // 1. every value, same on all channels
        for (int v = 0; v < 2**GAMMA_WIDTH; v++) drive_rgb_pixel({v[GAMMA_WIDTH-1:0], v[GAMMA_WIDTH-
1:0], v[GAMMA_WIDTH-1:0]});

        $finish;
    end

endmodule
```

sim/tb_pwm.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_pwm;
    timeunit 1ns;
    timeprecision 1ps;
    bit clk = '0;
    always #5 clk = !clk;

    localparam SLOTS_WIDTH = $clog2(`SLOTS);

    logic reset_n = '0;
    logic [SLOTS_WIDTH-1:0] value = '0;
    logic pwm_signal;


    pwm #(.SLOTS(`SLOTS)) dut (
        .clk(clk),
        .reset_n(reset_n),
        .value(value),
        .pwm_signal(pwm_signal)
    );
```

```
logic [SLOTS_WIDTH:0] pwm_one_cnt;
logic [SLOTS_WIDTH:0] pwm_zero_cnt;
logic [SLOTS_WIDTH-1:0] pwm_cnt;
bit last_cycle; assign last_cycle = (pwm_cnt == SLOTS_WIDTH'(`SLOTS - 1));
bit new_cycle; assign new_cycle = (pwm_cnt == '0);
always_ff @(posedge clk) begin
   if (!reset_n | last_cycle) pwm_cnt <= '0;
   else pwm_cnt <= pwm_cnt + 1;
end
always_ff @(posedge clk) begin
   if (!reset_n) pwm_one_cnt <= '0;
   else if (new_cycle) pwm_one_cnt <= pwm_signal;
   else if ((pwm_signal == '1)) pwm_one_cnt <= pwm_one_cnt + 1;
end
always_ff @(posedge clk) begin
   if (!reset_n) pwm_zero_cnt <= '0;
   else if (new_cycle) pwm_zero_cnt <= !pwm_signal;
   else if ((pwm_signal == '0)) pwm_zero_cnt <= pwm_zero_cnt + 1;
end

task reset_dut();
   $display("Time: %0t | TASK: Reset DUT", $time);
   reset_n = '0; #(CLK_PERIOD);
   reset_n = '1;
endtask

task wait_cycle();
   $display("Time: %0t | TASK: Waiting for new cycle to start", $time);
   while (!last_cycle) #(CLK_PERIOD); #(CLK_PERIOD);
endtask

task drive_value(input [SLOTS_WIDTH-1:0] val);
   $display("Time: %0t | TASK: Drive value: %d to pwm", $time, val);
   while (!last_cycle) #(CLK_PERIOD);
   value = val;
   check_pwm_signal();
endtask

task check_pwm_signal();
   $display("Time: %0t | TASK: Checking PWM signal", $time);
   wait_cycle();
   wait_cycle();
   $display("Time: %0t | pwm_one_count=%d | value=%d", $time, pwm_one_cnt, value);
   assert(pwm_one_cnt === value);
   assert(pwm_zero_cnt === `SLOTS - value);
endtask

initial begin
   reset_dut();
   drive_value(SLOTS_WIDTH'(`SLOTS / 2));
   drive_value('0);
```

```
        drive_value(SLOTS_WIDTH'(`SLOTS - 1));

        #(CLK_PERIOD); $finish;
    end


endmodule
```

sim/tb_pwm_cluster.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_pwm_cluster;
    timeunit 1ns;
    timeprecision 1ps;

    localparam SLOTS = `SLOTS;
    localparam SLOTS_WIDTH = $clog2(SLOTS);

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic [2:0][SLOTS_WIDTH-1:0] value = '0;
    logic pwm_led = '0;
    logic [2:0] led16, led17;

    pwm_cluster #(.SLOTS(SLOTS)) dut (
        .clk(clk),
        .reset_n(reset_n),
        .value(value),
        .pwm_led(pwm_led),
        .led16(led16),
        .led17(led17)
    );

    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD);
        assert(led16 === '0) else $error("led16 = %b after reset", led16);
        assert(led17 === '0) else $error("led17 = %b after reset", led17);
        reset_n = '1;
    endtask

    // count ones on all six pins over SLOTS consecutive cycles (any window of a full period gives the duty)
    task measure(output int ones16 [2:0], output int ones17 [2:0]);
        for (int i = 0; i < 3; i++) begin ones16[i] = 0; ones17[i] = 0; end
        for (int c = 0; c < SLOTS; c++) begin
            for (int i = 0; i < 3; i++) begin ones16[i] += led16[i]; ones17[i] += led17[i]; end
            #(CLK_PERIOD);
        end
    endtask
```

```
    task drive(input logic [2:0][SLOTS_WIDTH-1:0] v, input logic sel);
        int ones16 [2:0], ones17 [2:0];
        $display("Time: %0t | TASK: Drive value r=%0d g=%0d b=%0d, pwm_led=%b", $time, v[2], v[1], v[0], sel);
        value = v; pwm_led = sel;
        #(CLK_PERIOD * 2 * SLOTS);              // the pwm latches at its period boundary
        measure(ones16, ones17);
        for (int i = 0; i < 3; i++) begin
            if (sel) begin
                assert(ones17[i] === v[i]) else $error("led17[%0d] duty = %0d, expected %0d", i, ones17[i], v[i]);
                assert(ones16[i] === 0) else $error("led16[%0d] duty = %0d, expected 0 (led17 selected)", i,
ones16[i]);
            end else begin
                assert(ones16[i] === v[i]) else $error("led16[%0d] duty = %0d, expected %0d", i, ones16[i], v[i]);
                assert(ones17[i] === 0) else $error("led17[%0d] duty = %0d, expected 0 (led16 selected)", i,
ones17[i]);
            end
        end
    endtask

    initial begin
        reset_dut();

        // 1. three different duties on LED16, LED17 off
        drive({SLOTS_WIDTH'(SLOTS / 2), SLOTS_WIDTH'(SLOTS / 4), SLOTS_WIDTH'(1)}, '0);

        // 2. same values moved to LED17
        drive({SLOTS_WIDTH'(SLOTS / 2), SLOTS_WIDTH'(SLOTS / 4), SLOTS_WIDTH'(1)}, '1);

        // 3. edges: 0 and SLOTS-1
        drive({SLOTS_WIDTH'(0), SLOTS_WIDTH'(SLOTS - 1), SLOTS_WIDTH'(0)}, '1);
        drive({SLOTS_WIDTH'(SLOTS - 1), SLOTS_WIDTH'(0), SLOTS_WIDTH'(SLOTS - 1)}, '0);

        #(CLK_PERIOD); $finish;
    end

endmodule
```

sim/tb_binary_2_bvd.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_binary_2_bcd;
    timeunit 1ns;
    timeprecision 1ps;
    localparam SLOTS_WIDTH = $clog2(`SLOTS);

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic start = '0;
    logic [SLOTS_WIDTH-1:0] bin_in = '0;
    logic [NUM_DIGITS-1:0][DIG_IN_WIDTH-1:0] bcd_out;
    logic done;
```

```
    binary_2_bcd #(.SLOTS(`SLOTS)) dut (
        .clk(clk),
        .reset_n(reset_n),
        .start(start),
        .bin_in(bin_in),
        .bcd_out(bcd_out),
        .done(done)
    );

    task reset_dut ();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD); reset_n = '1;
    endtask

    function logic[NUM_DIGITS-1:0][DIG_IN_WIDTH-1:0] bin2bcd(input logic [SLOTS_WIDTH-1:0] b);
        for (int i = 0 ; i < NUM_DIGITS; i ++) begin
            bin2bcd[i] = (b / 10**i) % 10;
        end
    endfunction

    bit converting = '0;
    bit [SLOTS_WIDTH+1:0] cnt;
    always_ff @(posedge clk) begin
        if (!reset_n | done) cnt <='0;
        else if (converting & !start) cnt <= cnt + 1;
    end

    task drive_bin_in(input logic [SLOTS_WIDTH-1:0] b);
        $display("Time: %0t | TASK: Drive bin_in with: %b", $time, b);
        bin_in = b; start = '1; converting = '1; #(CLK_PERIOD);
        start = '0;
        do #(CLK_PERIOD); while (!done);
        converting = '0;
        assert(cnt - 1 === SLOTS_WIDTH);
        $display("Time: %0t | cnt=%d , SLOTS_WIDTH=%d", $time, cnt, SLOTS_WIDTH);
        #(CLK_PERIOD);
        assert(bcd_out === bin2bcd(b));
    endtask



    initial begin
        reset_dut();

        for (int i = 0; i < `SLOTS; i ++) begin
            drive_bin_in(i);
        end

        $finish;
    end

endmodule
```

sim/tb_tx_mac.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_tx_mac;
  timeunit 1ns;
  timeprecision 1ps;

  localparam SLOTS = `SLOTS;
  localparam SLOTS_WIDTH = $clog2(SLOTS);
  localparam MSG_BYTES = 19;
  localparam PHY_CYCLES = 20;                  // fake tx_phy: done this many clocks after tx_start

  bit clk = '0;
  always #5 clk = !clk;

  logic reset_n = '0;
  logic start = '0;
  logic done = '0;
  logic [2:0][SLOTS_WIDTH-1:0] pixel_data = '0;
  logic [DATA_WIDTH-1:0] drive_data;
  logic tx_start;

  tx_mac #(.SLOTS(SLOTS)) dut (
    .clk(clk),
    .reset_n(reset_n),
    .start(start),
    .done(done),
    .pixel_data(pixel_data),
    .drive_data(drive_data),
    .tx_start(tx_start)
  );

  task reset_dut();
    $display("Time: %0t | TASK: Reset DUT", $time);
    reset_n = '0; #(CLK_PERIOD);
    assert(tx_start === '0) else $error("tx_start = %b after reset", tx_start);
    reset_n = '1;
  endtask

  function string exp_msg(input logic [2:0][SLOTS_WIDTH-1:0] p);
    return $sformatf("{R%04d,G%04d,B%04d}", p[2], p[1], p[0]);
  endfunction

  task pulse_start();
    $display("Time: %0t | TASK: start", $time);
    start = '1; #(CLK_PERIOD);
    start = '0;
  endtask

  // fake tx_phy: wait for tx_start, take the byte, answer with done PHY_CYCLES later
  // inject = an extra start while the byte is on the line (tx_start is a 1-cycle pulse, so it can't come before the wait)
  task phy_byte(output logic [DATA_WIDTH-1:0] b, input bit inject);
    while (tx_start !== 1'b1) #(CLK_PERIOD);
```

```
    b = drive_data;
    if (inject) pulse_start();
    #(CLK_PERIOD * PHY_CYCLES);
    done = '1; #(CLK_PERIOD);
    done = '0;
endtask

// collect one message; start_at / start_at2 = byte index where an extra start is injected (-1 = none)
task recv_msg(output string s, input int start_at, input int start_at2);
    logic [DATA_WIDTH-1:0] b;
    s = "";
    for (int i = 0; i < MSG_BYTES; i++) begin
        phy_byte(b, (i == start_at) || (i == start_at2));
        s = {s, string'(b)};
    end
    $display("Time: %0t | TASK: Received %s", $time, s);
endtask

task check_msg(input logic [2:0][SLOTS_WIDTH-1:0] p, input int start_at, input int start_at2);
    string got;
    recv_msg(got, start_at, start_at2);
    assert(got == exp_msg(p)) else $error("got %s, expected %s", got, exp_msg(p));
endtask

task check_idle(input int cycles);
    for (int i = 0; i < cycles; i++) begin
        assert(tx_start === '0) else $error("tx_start = 1 while idle");
        #(CLK_PERIOD);
    end
endtask

initial begin
    reset_dut();

    // 1. one message
    pixel_data = {SLOTS_WIDTH'(SLOTS / 2), SLOTS_WIDTH'(SLOTS - 1), SLOTS_WIDTH'(3)};
    pulse_start();
    check_msg(pixel_data, -1, -1);
    check_idle(50);

    // 2. zero values
    pixel_data = '0;
    pulse_start();
    check_msg(pixel_data, -1, -1);
    check_idle(50);

    // 3. start in the middle of a message: the message finishes, then a second one follows (pending)
    pixel_data = {SLOTS_WIDTH'(1), SLOTS_WIDTH'(2), SLOTS_WIDTH'(3)};
    pulse_start();
    check_msg(pixel_data, 5, -1);
    check_msg(pixel_data, -1, -1);
    check_idle(50);

    // 4. two starts in the middle of one message: only ONE extra message (pending is a flag, not a counter)
    pulse_start();
    check_msg(pixel_data, 3, 9);
```

```
        check_msg(pixel_data, -1, -1);
        check_idle(50);

        #(CLK_PERIOD); $finish;
    end

    // watchdog
    initial begin
        #(5ms);
        $error("TIMEOUT");
        $finish;
    end

endmodule
```

sim/tb_tx_phy.sv

```
import lab6_pkg::*;

module tb_tx_phy;
    timeunit 1ns;
    timeprecision 1ps;
    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic start = '0;
    logic [DATA_WIDTH-1:0] data = '0;
    logic tx, done;

    tx_phy t_phy_i(
        .clk(clk),
        .reset_n(reset_n),
        .start(start),
        .data(data),
        .tx(tx),
        .done(done)
    );

    // How many clks per bit?
    // CLK_FREQ / BAUD_RATE
    localparam CLKS_PER_BIT = CLK_FREQ / `BAUD_RATE;
    localparam BIT_WAIT = CLKS_PER_BIT * CLK_PERIOD;

    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD); assert((tx === '1) & (done === '0));
        reset_n = '1;
    endtask

    logic [FRAME_WIDTH-1:0] frame;
    task send_data(input logic [DATA_WIDTH-1:0] data_2_drive);
        $display("Time: %0t | TASK: Send data: %b", $time, data_2_drive);
        data = data_2_drive; start = '1; #(CLK_PERIOD);
        start = '0;
```

```
      frame = {1'b1, data_2_drive, 1'b0};
      for (int i=0; i < FRAME_WIDTH; i++) begin
         if (i == 0) #(BIT_WAIT - 1*CLK_PERIOD); // To take in to consideration the first #(CLK_PERIOD) after
start
         else #(BIT_WAIT);
         $display("Time: %0t | TASK: DUT_tx=%b, TB_tx=%b", $time, tx, frame[i]);
         assert(frame[i] === tx);
      end
      assert(done);
      #(CLK_PERIOD);
   endtask

   task check_idle(input int clk_cycles);
      $display("Time: %0t | TASK: Check it tx=1 while idle", $time);
      for (int i = 0; i < clk_cycles; i++) begin
         #(CLK_PERIOD);
         assert((tx === '1) & (done === '0));
      end
   endtask

   initial $monitor("Time: %0t | tx=%b | done=%b",
                    $time, tx, done);

   initial begin
      $display("Time: %0t | Reset DUT", $time);
      reset_dut();

      // Drive 0xA5;
      $display("Time: %0t | Driving: %b", $time, 8'hA5);
      send_data(8'hA5);

      // Drive 0x5A;
      $display("Time: %0t | Driving: %b", $time, 8'h5A);
      send_data(8'h5A);

      //Drive 0x00
      $display("Time: %0t | Driving: %b", $time, 8'h00);
      send_data(8'h00);

      //Drive 0xFF
      $display("Time: %0t | Driving: %b", $time, 8'hFF);
      send_data(8'hff);

      check_idle(100);

      $finish;
   end


   property p_load;
      @(posedge clk)
      disable iff(!reset_n)
      ($past(start)) |-> (t_phy_i.sh_reg == {1'b1, data, 1'b0});
   endproperty

   property p_tx_one_done;
```

```
    @(posedge clk)
    disable iff(!reset_n)
    ($past(done)) |-> (tx);
  endproperty

  assert property(p_load);
  assert property(p_tx_one_done);

endmodule
```

sim/tb_baud_gen.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_baud_gen;
  timeunit 1ns;
  timeprecision 1ps;
  localparam BAUD_COUNT = (CLK_FREQ/`BAUD_RATE);
  localparam TICK_WAIT = (BAUD_COUNT - 1) * CLK_PERIOD;
  localparam HALF_WAIT = (BAUD_COUNT / 2) * CLK_PERIOD;

  bit clk = '0;
  always #5 clk =!clk;

  bit reset_n, en, baud_tick;

  baud_gen #(.BAUD_RATE(`BAUD_RATE)) baud_gen_i (
    .clk(clk),
    .reset_n(reset_n),
    .en(en),
    .baud_tick(baud_tick)
  );

  task reset_dut(input bit en_reset);
    $display("Time: %0t | TASK: Reset DUT", $time);
    reset_n = '0;
    if (en_reset) en = '0;
    #(CLK_PERIOD);
    reset_n = '1;
  endtask

  task en_baud_gen(input int wait_time, input bit lower_en);
    $display("Time: %0t | TASK: Raise enable for %0d ns", $time, wait_time);
    en = '1; #(wait_time);
    if (lower_en) begin
      $display("Time: %0t | TASK: Lower enable", $time);
      en = '0;
    end
  endtask

  initial $monitor("Time: %0t | en=%0b | baud_tick=%0b", $time, en, baud_tick);

  initial begin
    reset_dut('1);
```

```
        assert(!baud_tick);


        en_baud_gen(TICK_WAIT, '1);
        assert(baud_tick); #(CLK_PERIOD);

        en_baud_gen(HALF_WAIT, '1);
        assert(!baud_tick); #(CLK_PERIOD);

        $display("Time: %0t | Reset mid en", $time);
        en_baud_gen(HALF_WAIT, '0);
        assert(!baud_tick);
        reset_dut('0);
        assert(!baud_tick);
        en_baud_gen(TICK_WAIT, '1);
        assert(baud_tick);
        #(CLK_PERIOD); $finish;

    end


    int count;
    always_ff @(posedge clk) begin
        if (!reset_n | !en | (count == BAUD_COUNT-1)) count <= '0;
        else if (en) count <= count + 1;
    end

    bit valid_en; assign valid_en = (count == BAUD_COUNT-1);

    property p_tick_after_count;
        @(posedge clk)
        disable iff (!reset_n)
        (valid_en) |-> (baud_tick);
    endproperty
    assert property(p_tick_after_count);

    property p_baud_tick_one_cycle;
        @(posedge clk)
        disable iff(!reset_n)
        ($past(baud_tick)) |-> (!baud_tick);
    endproperty
    assert property(p_baud_tick_one_cycle);

    property p_reset_press;
        @(posedge clk)
        (!reset_n) |=> (baud_tick == '0);
    endproperty
    assert property(p_reset_press);



endmodule
```

sim/tb_chip_top.sv

```
import lab6_pkg::*;
`include "lab6_definitions.svh"

module tb_chip_top;
  timeunit 1ns;
  timeprecision 1ps;

  localparam SLOTS = `SLOTS;
  localparam SLOTS_WIDTH = $clog2(SLOTS);
  localparam CLKS_PER_BIT = CLK_FREQ / `BAUD_RATE;
  localparam BIT_WAIT = CLKS_PER_BIT * CLK_PERIOD;
  localparam DEB_WAIT = (`DEB_COUNT + 5) * CLK_PERIOD;        // debounce + sync + edge
  localparam MSG_BYTES = 19;

  bit clk = '0;
  always #5 clk = !clk;

  logic cpu_reset = '1;
  logic btnu = '0, btnd = '0, btnl = '0, btnr = '0, btnc = '0;
  logic rx = '1;
  logic tx;
  logic [AN_WIDTH-1:0] an;
  logic [SS_DIG_WIDTH-1:0] seg;
  logic dp;
  logic [15:0] led;
  logic led16_r, led16_g, led16_b;
  logic led17_r, led17_g, led17_b;

  chip_top #(
    .BAUD_RATE(`BAUD_RATE),
    .DEB_COUNT(`DEB_COUNT),
    .DISPLAY_RR(`DISPLAY_RR),
    .SLOTS(SLOTS)
  ) dut (
    .clk(clk),
    .cpu_reset(cpu_reset),
    .btnu(btnu), .btnd(btnd), .btnl(btnl), .btnr(btnr), .btnc(btnc),
    .rx(rx),
    .tx(tx),
    .an(an), .seg(seg), .dp(dp),
    .led(led),
    .led16_r(led16_r), .led16_g(led16_g), .led16_b(led16_b),
    .led17_r(led17_r), .led17_g(led17_g), .led17_b(led17_b)
  );

  // ---------------- model: sRGB -> table (1023) -> shift to SLOTS -> CIE ----------------
  function int table_val(input logic [DATA_WIDTH-1:0] s);
    real c, l;
    c = s / 255.0;
    if (c <= 0.04045) l = c / 12.92;
    else l = $pow((c + 0.055) / 1.055, 2.4);
    return int'(l * ((2**GAMMA_WIDTH) - 1));
  endfunction
```

```
    function int to_slots(input int t);
        if (SLOTS_WIDTH >= GAMMA_WIDTH) return t << (SLOTS_WIDTH - GAMMA_WIDTH);
        else return t >> (GAMMA_WIDTH - SLOTS_WIDTH);
    endfunction

    function int exp_red(input logic [DATA_WIDTH-1:0] s); return to_slots(table_val(s)); endfunction
    function int exp_green(input logic [DATA_WIDTH-1:0] s);
        int v;
        v = to_slots(table_val(s));
        return (v >> 1) + (v >> 7);
    endfunction
    function int exp_blue(input logic [DATA_WIDTH-1:0] s); return to_slots(table_val(s)); endfunction

    // ---------------- reset ----------------
    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        cpu_reset = '0; #(DEB_WAIT);
        cpu_reset = '1; #(DEB_WAIT);
        assert(tx === '1) else $error("tx = %b after reset, expected idle 1", tx);
        assert({led16_r, led16_g, led16_b, led17_r, led17_g, led17_b} === '0) else $error("rgb leds not off after
reset");
        assert(led === 16'h0030) else $error("led = %h after reset, expected cursor on T0 (0030)", led);
    endtask

    // ---------------- UART in ----------------
    task send_byte(input logic [DATA_WIDTH-1:0] b);
        logic [FRAME_WIDTH-1:0] frame;
        frame = {1'b1, b, 1'b0};
        for (int i = 0; i < FRAME_WIDTH; i++) begin
            rx = frame[i]; #(BIT_WAIT);
        end
    endtask

    task send_msg(input string s);
        $display("Time: %0t | TASK: Send %s", $time, s);
        for (int i = 0; i < s.len(); i++) send_byte(s[i]);
        #(CLK_PERIOD * 20);                             // rx_mac -> parser -> selector
    endtask

    // ---------------- UART out: collect one message from tx ----------------
    task recv_byte(output logic [DATA_WIDTH-1:0] b);
        while (tx !== 1'b0) #(CLK_PERIOD);                  // start bit
        #(BIT_WAIT / 2);
        for (int i = 0; i < DATA_WIDTH; i++) begin
            #(BIT_WAIT); b[i] = tx;
        end
        #(BIT_WAIT);                                    // stop bit
    endtask

    task recv_msg(output string s);
        logic [DATA_WIDTH-1:0] b;
        s = "";
        for (int i = 0; i < MSG_BYTES; i++) begin
            recv_byte(b);
            s = {s, string'(b)};
        end
```

```
    $display("Time: %0t | TASK: Received %s", $time, s);
  endtask

  task check_tx(input logic [DATA_WIDTH-1:0] r, g, b);
    string got, exp;
    exp = $sformatf("{R%04d,G%04d,B%04d}", exp_red(r), exp_green(g), exp_blue(b));
    recv_msg(got);
    assert(got == exp) else $error("tx message %s, expected %s", got, exp);
  endtask

  // ---------- PWM duty: ones in SLOTS consecutive cycles, all six rgb pins at once ----------
  logic [5:0] rgb; assign rgb = {led16_r, led16_g, led16_b, led17_r, led17_g, led17_b};

  task check_rgb(input int exp [5:0]);
    int ones [5:0];
    #(CLK_PERIOD * 2 * SLOTS);                    // let the pwm latch at a period boundary
    for (int i = 0; i < 6; i++) ones[i] = 0;
    for (int c = 0; c < SLOTS; c++) begin
      for (int i = 0; i < 6; i++) ones[i] += rgb[i];
      #(CLK_PERIOD);
    end
    for (int i = 0; i < 6; i++)
      assert(ones[i] === exp[i]) else $error("rgb[%0d] (%s) duty = %0d/%0d, expected %0d", i,
        i == 5 ? "led16_r" : i == 4 ? "led16_g" : i == 3 ? "led16_b" : i == 2 ? "led17_r" : i == 1 ? "led17_g" :
"led17_b",
        ones[i], SLOTS, exp[i]);
  endtask

  task check_led16(input logic [DATA_WIDTH-1:0] r, g, b);
    check_rgb('{5: exp_red(r), 4: exp_green(g), 3: exp_blue(b), 2: 0, 1: 0, 0: 0});
  endtask

  task check_led17(input logic [DATA_WIDTH-1:0] r, g, b);
    check_rgb('{5: 0, 4: 0, 3: 0, 2: exp_red(r), 1: exp_green(g), 0: exp_blue(b)});
  endtask

  // ---------- buttons (levels through the debounce) ----------
  typedef enum logic [2:0] { BTN_U, BTN_D, BTN_L, BTN_R, BTN_C } btn_e;

  task press(input btn_e b);
    $display("Time: %0t | TASK: Press %s", $time, b.name());
    case (b)
      BTN_U: btnu = '1;
      BTN_D: btnd = '1;
      BTN_L: btnl = '1;
      BTN_R: btnr = '1;
      default: btnc = '1;
    endcase
    #(DEB_WAIT);
    {btnu, btnd, btnl, btnr, btnc} = '0;
    #(DEB_WAIT);
  endtask

  initial begin
    reset_dut();
```

```
    // tx answers ~16 clocks after the last stop bit starts, so the receiver runs next to the sender (fork)

    // 1. white: pixel to LED16, feedback on tx
    fork
      send_msg("{R255,G255,B255}");
      check_tx(255, 255, 255);
    join
    check_led16(255, 255, 255);

    // 2. red only, lowercase opcodes
    fork
      send_msg("{r255,g000,b000}");
      check_tx(255, 0, 0);
    join
    check_led16(255, 0, 0);

    // 3. move to LED17: colour moves, no tx for an L packet
    send_msg("{L017}");
    check_led17(255, 0, 0);
    assert(tx === '1) else $error("tx busy after an L packet");

    // 4. invalid packet: nothing changes
    send_msg("{R256,G000,B000}");
    check_led17(255, 0, 0);
    assert(tx === '1) else $error("tx busy after an invalid packet");

    // 5. buttons: left x2 -> cursor on T2 (red), down, center -> new value applied + tx
    press(BTN_L); press(BTN_L);
    assert(led === 16'h0600) else $error("led = %h, expected cursor on T2 (0600)", led);
    press(BTN_D);
    check_led17(255, 0, 0);                    // not applied yet
    fork
      press(BTN_C);
      check_tx(254, 0, 0);
    join
    check_led17(254, 0, 0);

    // 6. back to LED16, off
    send_msg("{L016}");
    fork
      send_msg("{R000,G000,B000}");
      check_tx(0, 0, 0);
    join
    check_led16(0, 0, 0);

    #(CLK_PERIOD * 10); $finish;
  end

  // watchdog
  initial begin
    #(50ms);
    $error("TIMEOUT");
    $finish;
  end

endmodule
```